

DSA Patterns - పూర్తి తెలుగు గైడ్ (SDE2 Interview కి Pattern-First విధానం)

ఇది **problem-by-problem doc కాదు**. ఇది **pattern-first doc** — "ఈ problem ఏ pattern కి చెందుతుంది?" అని 60 సెక్షన్లలో గుర్తించే నైపుణ్యం నేర్పే document.

ఎందుకు ఇది అవసరం: LeetCode లో 3000+ problems ఉన్నాయి. అన్నీ solve చేయడం అసాధ్యం, అనవసరం. కానీ ~20 patterns తెలిస్తే — interview లో మీరు ఎప్పుడూ చూడని problem వచ్చినా, దాన్ని ఒక తెలిసిన pattern లోకి మార్చగలరు. **SDE2 interview లో గెలిచేది ఇదే నైపుణ్యం** — గుర్తుపెట్టుకున్న solutions కాదు, **pattern recognition**.

ప్రతి pattern కి ఉండేవి: ఎప్పుడు వాడాలో చెప్పే **recognition signals** (problem statement లోని పదాలు) · real-life analogy · **universal template code** (JavaScript) · variants · complexity · ఈ pattern కి చెందిన problems జాబితా · gotchas · interview దృష్టి.

Companion docs: [DSA_00_Foundations_Telugu.md](#) (complexity, recursion, data structures ఇక్కడ నుండి మొదలుపెట్టండి — ఇది తెలియకపోతే ఈ doc కట్టండి), [DSA_01..10_Telugu.md](#) (LeetCode Top-150 problem-by-problem drill — ఈ doc లో నేర్చుకున్న patterns ని అక్కడ practice చేయండి).

లక్ష్యం: "ఈ problem ఎలా solve చేయాలి?" అని ఆలోచించడం మానేసి, "ఇది ఏ pattern?" అని ఆలోచించడం అలవాటు చేయడం.

విషయ సూచిక (Table of Contents)

Part 0 — Pattern Recognition యొక్క శాస్త్రం

0. Pattern Recognition Framework — problem చూసి 60 సెక్షన్లలో pattern గుర్తించడం ఎలా 0.1. Constraints → Algorithm Map (n విలువ చూసి complexity ని deduce చేయడం) ★

Part 1 — Array & String Patterns (interviews లో 40%)

1. Two Pointers (opposite ends)
2. Fast & Slow Pointers (same direction, cycle detection)
3. Sliding Window (fixed & variable) ★
4. Prefix Sum & Difference Array
5. Kadane's / Running Optimum (max subarray family)
6. Cyclic Sort (index-as-hash)
7. Merge Intervals & Sweep Line
8. In-place Array Manipulation (rotation, partition, marking)

Part 2 — Hashing & Counting

9. Hash Map Patterns (frequency, seen-set, index-map, grouping, complement)
10. Monotonic Stack ★ (next greater/smaller)

Part 3 — Searching & Sorting Patterns

- 11. Binary Search — Universal Template + Variants ★
- 12. Binary Search on Answer (search space, not array) ★★
- 13. Top-K & Heap Patterns (k-largest, merge-k, two heaps)
- 14. Sorting-based Patterns (custom comparators, greedy sorting)

Part 4 — Linked List Patterns

- 15. In-place Reversal (reverse, k-group, palindrome)
- 16. Fast-Slow on Lists (middle, cycle, nth-from-end)
- 17. Dummy Node & Pointer Manipulation

Part 5 — Tree Patterns

- 18. Tree DFS — Top-Down vs Bottom-Up (ఇది అర్థమైతే 80% tree problems అయిపోయాయి) ★
- 19. Tree BFS — Level Order Family
- 20. BST Patterns (inorder property, validation, kth element)
- 21. Lowest Common Ancestor & Path Problems
- 22. Tree Construction & Serialization

Part 6 — Graph Patterns

- 23. Graph Representation & Traversal (BFS/DFS templates)
- 24. Grid/Matrix as Graph (islands, flood fill, multi-source BFS) ★
- 25. Topological Sort (dependency ordering)
- 26. Union-Find (Disjoint Set Union)
- 27. Shortest Path (BFS, Dijkstra, Bellman-Ford — ఏది ఎప్పుడు)

Part 7 — Recursion, Backtracking & Divide-Conquer

- 28. Backtracking Universal Template (subsets, permutations, combinations) ★
- 29. Divide & Conquer
- 30. Recursion → Memoization Transformation ★

Part 8 — Dynamic Programming (interviews లో అత్యంత భయపెట్టేది)

- 31. DP Recognition & State Design Framework ★★
- 32. 1D DP (Fibonacci family, house robber, decode ways)
- 33. 2D DP — Grid Paths
- 34. String DP (LCS, edit distance, palindromes)
- 35. Knapsack Family (0/1, unbounded, subset sum, partition)
- 36. LIS Family (longest increasing subsequence + patience sorting)
- 37. DP on Trees, Intervals & Bitmask (advanced)

Part 9 — Greedy, Bit & Math

38. Greedy — ఎప్పుడు పని చేస్తుంది, ఎలా నిరూపించాలి

39. Bit Manipulation Patterns

40. Math & Number Theory Patterns

Part 10 — Design & Interview Craft

41. Design Patterns (LRU cache, iterators, data structure design)

42. Interview Communication Framework (UMPIRE) + ఎలా think-aloud చేయాలి

43. Pattern Decision Tree (ఒక్క page cheat-sheet) ★

44. 8-Week Preparation Plan + Revision Strategy

Part 0 — Pattern Recognition యొక్క శాస్త్రం

0. Pattern Recognition Framework

వివరణ — interview లో నిజంగా ఏం జరుగుతుంది

Interviewer ఒక problem ఇచ్చాడు. మీకు 45 నిమిషాలు. చాలామంది ఇలా చేస్తారు:

✗ తప్పు విధానం:
problem చదవడం → "అయ్యో, ఇది నేను చూడలేదు" → panic →
brute force రాయడం → optimize చేయడానికి ప్రయత్నం → సమయం అయిపోవడం

✓ సరైన విధానం:
problem చదవడం → **సంకేతాలు (signals) వెతకడం** → pattern గుర్తించడం →
ఆ pattern యొక్క template గుర్తు చేసుకోవడం → ఈ problem కి adapt చేయడం → code

మొదటి 5 నిమిషాలు code రాయకూడదు. ఆ 5 నిమిషాలే మిగతా 40 నిమిషాలను నిర్ణయిస్తాయి.

Real-life Scenario

Pattern recognition = వైద్యుని diagnosis. అనుభవం లేని వ్యక్తి "జ్వరం వచ్చింది, ఏం చేయాలి?" అని ప్రతిసారి కొత్తగా ఆలోచిస్తాడు. అనుభవజ్ఞుడైన వైద్యుడు **లక్షణాలు (symptoms) వింటాడు** — "జ్వరం + దగ్గు + 3 రోజులు" → "ఇది ఈ category" → తగిన పరీక్ష → చికిత్స.

Problem statement లోని పదాలే symptoms. "sorted array" + "O(1) space" → two pointers. "contiguous subarray" → sliding window. "shortest path in unweighted graph" → BFS. ఈ mapping ని బట్టి పెట్టడమే **DSA preparation యొక్క అసలు పని.**

🔍 Recognition Signals — పూర్తి పట్టిక (ఇది ప్రింట్ చేసుకోండి)

Problem లో ఈ పదాలు కనిపిస్తే...	Pattern
"sorted array" + "pair/triplet" + " $O(1)$ space"	Two Pointers (opposite ends)
"contiguous subarray/substring" + "max/min/longest/shortest"	Sliding Window ★
"subarray sum equals K" (negatives ఉన్నాయి)	Prefix Sum + HashMap
"cycle" in linked list / "middle node" / "nth from end"	Fast & Slow Pointers
"numbers 1 to n" / "missing/duplicate" + " $O(1)$ space"	Cyclic Sort
"overlapping intervals" / "merge" / "meeting rooms"	Merge Intervals
"next greater/smaller element" / "histogram" / "temperature"	Monotonic Stack ★
"sorted" + "find/search" + $O(\log n)$	Binary Search
"minimize the maximum" / "maximum of minimum" / "minimum capacity/speed/days"	Binary Search on Answer ★★
"top K" / "K largest/smallest/frequent" / "Kth"	Heap (లేదా QuickSelect)
"median of stream"	Two Heaps
"merge K sorted"	Heap
"tree" + "path/depth/sum/property"	Tree DFS
"tree" + "level" / "left/right view" / "zigzag" / "minimum depth"	Tree BFS
"BST" + "kth/validate/sorted"	BST inorder
"grid" + "islands/regions/flood"	Grid DFS/BFS ★
"shortest path" + unweighted	BFS
"shortest path" + weighted, non-negative	Dijkstra
"prerequisites" / "dependencies" / "build order" / "course schedule"	Topological Sort
"connected components" / "friend circles" / "redundant connection"	Union-Find
"all possible/combinations/permutations/subsets"	Backtracking ★
"generate all valid ..." (parentheses, IPs, words)	Backtracking
"maximum/minimum/count of ways" + choices at each step + overlapping subproblems	Dynamic Programming ★★
"longest increasing subsequence" (subsequence, subarray కాదు)	LIS / DP
"can we partition/select items with limit"	Knapsack DP

Problem లో ఈ పదాలు కనిపిస్తే...	Pattern
"two strings" + "common/edit/transform"	String DP (2D)
"prefix" / "autocomplete" / "word dictionary"	Trie
"always pick the best now" + sorted input works	Greedy (నిరూపించాలి!)
"XOR" / "single number" / "power of 2" / "count bits"	Bit Manipulation
"O(1) get and put"	HashMap + Doubly Linked List (LRU)

60-సెకన్ల recognition drill

1. Input రకం ఏమిటి? array / string / tree / graph / linked list / number
2. Output ఏమిటి? value / index / boolean / list / count / all-solutions
3. కీలక constraint? sorted? O(1) space? n పరిమాణం? in-place?
4. ఏ కీలక పదాలు? (పై పట్టిక)
5. Brute force ఏమిటి? దాని complexity?
6. అక్కడ waste అవుతున్నది? → అదే optimization కి దారి

Brute force → optimization మార్గాలు (ఇది ఒక universal ట్రిక్)

$O(n^2)$ nested loops	→ hash map (lookup $O(1)$)	→ $O(n)$
$O(n^2)$ nested loops	→ sorting + two pointers	→ $O(n \log n)$
$O(n^2)$ subarrays	→ sliding window	→ $O(n)$
$O(n^2)$ subarray sums	→ prefix sum	→ $O(n)$
$O(2^n)$ recursion	→ memoization (overlapping)	→ $O(n \cdot \text{states})$
$O(n)$ linear search	→ binary search (sorted/monotonic)	→ $O(\log n)$
Repeated min/max	→ heap	→ $O(n \log k)$
Repeated connectivity	→ union-find	→ $\sim O(\alpha(n))$
Repeated prefix lookups	→ trie	→ $O(\text{len})$

Interview లో ఈ వాక్యం చెప్పండి: "Brute force is $O(n^2)$ because I'm recomputing X for every pair. If I store Y in a hash map / use a window, I can avoid that recomputation." — ఇది interviewer కి మీరు ఎందుకు అని ఆలోచిస్తున్నారని చూపిస్తుంది.

Key Points

- Interview లో గెలిచేది **pattern recognition**, memorized solutions కాదు.
- మొదటి 5 నిమిషాలు — **signals వెతకడం**, code కాదు.
- **Signals table** = మీ diagnostic manual.
- **Brute force లో ఏం వృథా అవుతోంది?** → అదే optimization కి దారి.

0.1. Constraints → Algorithm Map ★

వివరణ — ఇది interview యొక్క దాగిన సూచన

Problem statement లో $1 \leq n \leq 10^5$ అని రాసి ఉంటుంది. అది యాదృచ్ఛికం కాదు — అది interviewer ఇచ్చే సూచన (hint).

Modern machine సెకనుకి $\sim 10^8$ operations చేస్తుంది. Interview limit సాధారణంగా 1-2 సెకన్లు.

n యొక్క పరిమాణం	అమోదయోగ్య complexity	సాధారణంగా ఏ pattern
$n \leq 10$	$O(n!)$, $O(2^n \cdot n)$	permutations, brute force backtracking
$n \leq 20$	$O(2^n)$	subsets, bitmask DP
$n \leq 100$	$O(n^3)$	Floyd-Warshall, 3D DP, interval DP
$n \leq 1,000$	$O(n^2)$	2D DP, nested loops, LCS/edit distance
$n \leq 10^5$	$O(n \log n)$	sorting, heap, binary search, divide & conquer
$n \leq 10^6$	$O(n)$	two pointers, sliding window, hashing, prefix sum
$n \leq 10^9$	$O(\log n)$ or $O(\sqrt{n})$	binary search on answer, math, bit tricks
$n \leq 10^{18}$	$O(\log n)$	binary exponentiation, math (arrays అసాధ్యం!)

ఇలా వాడాలి (interview లో ప్రత్యక్షంగా)

Interviewer: "n up to 10^5 "

మీరు: " 10^5 అంటే $O(n^2) = 10^{10}$ — చాలా ఎక్కువ.

కాబట్టి నాకు $O(n \log n)$ లేదా $O(n)$ కావాలి.

అంటే sorting + two pointers, లేదా hash map, లేదా sliding window వైపు చూడాలి."

ఇది చెప్పడం interviewer కి భారీ signal — మీరు complexity ని అంచనా వేసి పని చేస్తున్నారని.

ఇతర దాగిన సూచనలు

Constraint	సూచన
"O(1) extra space"	two pointers, in-place marking, cyclic sort, bit tricks (hash map ❌)
"array is sorted"	binary search, two pointers
"values are 1 to n"	cyclic sort, index-as-hash marking
"return all possible..."	backtracking (output ఖరీదు కాబట్టి n చిన్నది)
"modulo 10^9+7 "	DP counting (సంఖ్య పెద్దది అవుతుంది కాబట్టి)
" $0 \leq \text{values} \leq 100$ " (చిన్న range)	counting sort, bucket, frequency array
"in-place"	pointer manipulation, swap
"streaming / can't store all"	heap, reservoir sampling, two pointers
"distinct values"	set operations
"negative numbers ఉన్నాయి"	sliding window ❌ (prefix sum + hashmap ✅)
"return index, not value"	sorting చేస్తే indices పోతాయి → hash map

Key Points

- **Constraints = interviewer ఇచ్చే hint.**
- $n \leq 10^5 \rightarrow O(n \log n)$; $n \leq 10^6 \rightarrow O(n)$; $n \leq 20 \rightarrow O(2^n)$.
- "O(1) space", "sorted", "values 1..n", "all possible" – ప్రతి ఒక్కటి ఒక pattern వైపు చూపిస్తుంది.
- **Interview లో ఈ reasoning ని బయటకీ చెప్పాలి.**

Part 1 — Array & String Patterns

1. Two Pointers (Opposite Ends)

🔍 ఎప్పుడు వాడాలి — Recognition signals

- ✓ Array ****sorted**** ఉంది + pair/triplet వెతకాలి
- ✓ "0(1) space" అని చెప్పారు
- ✓ రెండు చివరల నుండి compare చేయాల్సిన అవసరం (palindrome)
- ✓ Container/**area** maximize చేయాలి
- ✓ **Reverse** / partition **in-place**

వివరణ

రెండు pointers — ఒకటి మొదట్లో (**left**), ఒకటి చివర్లో (**right**) — **లోపలికి కదులుతాయి**. ప్రతి step లో ఏదో ఒక pointer కదులుతుంది కాబట్టి $O(n)$.

ఎందుకు పని చేస్తుంది (sorted arrays లో): **left** ని కుడికి కదిపితే sum పెరుగుతుంది, **right** ని ఎడమకి కదిపితే sum తగ్గుతుంది. ఈ **monotonic ప్రవర్తన** మనకి దిక్కుచి — ప్రతి step లో ఏ దిశలో వెళ్ళాలో ఖచ్చితంగా తెలుస్తుంది.

Real-life Scenario

Two pointers = బరువు తూచే త్రాసు. ఒక వైపు తేలికైన వస్తువులు (array మొదలు), ఇంకో వైపు బరువైనవి (చివర). మొత్తం బరువు ఎక్కువైతే — **బరువైన వైపు నుండి ఒకటి తీసేయి** (**right--**). తక్కువైతే — **తేలికైన వైపు నుండి ఒకటి తీసేసి** ముందుకు వెళ్ళు (**left++**). Sorted కాబట్టి, ఏ దిశలో వెళ్ళాలో సందేహం ఉండదు.

📄 Universal Template

```
function twoPointers(arr, target) {
    let left = 0, right = arr.length - 1;

    while (left < right) {
        // ⚠️ <= కాదు (ఒకే element రెండుసార్లు వద్దు)
        const sum = arr[left] + arr[right];

        if (sum === target) return [left, right];
        else if (sum < target) left++; // ఎక్కువ కావాలి → పెద్ద numbers వైపు
        else right--; // తక్కువ కావాలి → చిన్న numbers వైపు
    }
    return [-1, -1];
}
```

Variant 1 — Palindrome check (skip characters)

```
function isPalindrome(s) {
  let l = 0, r = s.length - 1;
  while (l < r) {
    while (l < r && !isAlnum(s[l])) l++; // ⚠️ లోపలి loops లో కూడా l < r
    while (l < r && !isAlnum(s[r])) r--;
    if (s[l].toLowerCase() !== s[r].toLowerCase()) return false;
    l++; r--;
  }
  return true;
}
const isAlnum = (c) => /[a-z0-9]/i.test(c);
```

Variant 2 — 3Sum (sort + fix one + two pointers) ★

```
function threeSum(nums) {
  nums.sort((a, b) => a - b); // 🔑  $O(n \log n)$  - two pointers కి sorted అవసరం
  const res = [];

  for (let i = 0; i < nums.length - 2; i++) {
    if (nums[i] > 0) break; // ✂️ sorted → ఇక sum > 0
    if (i > 0 && nums[i] === nums[i - 1]) continue; // 🔑 duplicate triplets నివారణ

    let l = i + 1, r = nums.length - 1;
    while (l < r) {
      const sum = nums[i] + nums[l] + nums[r];
      if (sum === 0) {
        res.push([nums[i], nums[l], nums[r]]);
        while (l < r && nums[l] === nums[l + 1]) l++; // 🔑 duplicates skip
        while (l < r && nums[r] === nums[r - 1]) r--;
        l++; r--;
      } else if (sum < 0) l++;
      else r--;
    }
  }
  return res;
}
// Time:  $O(n^2)$  · Space:  $O(1)$  (output తప్ప)
```

3Sum = "ఒకటి fix చేసి, మిగతా దానికీ 2Sum" — ఇది 4Sum, kSum కి కూడా విస్తరిస్తుంది (recursion తో).

Variant 3 — Container With Most Water (greedy two pointers)

```
function maxArea(height) {
  let l = 0, r = height.length - 1, max = 0;
  while (l < r) {
    max = Math.max(max, Math.min(height[l], height[r]) * (r - l));
    // 🔑 కీలక అంతర్దృష్టి: ఎత్తు తక్కువ ఉన్న వైపు కదపాలి
    // (ఎక్కువ ఉన్నది కదిపితే - width తగ్గుతుంది, height పెరగదు → ఎప్పటికీ మెరుగుపడదు)
    if (height[l] < height[r]) l++;
  }
}
```

```

    else r--;
  }
  return max;
}

```

Variant 4 — Trapping Rain Water (two pointers + running max)

```

function trap(height) {
  let l = 0, r = height.length - 1;
  let leftMax = 0, rightMax = 0, water = 0;

  while (l < r) {
    if (height[l] < height[r]) {
      // 🗝️ height[l] < height[r] కాబట్టి కుడి వైపు ఖచ్చితంగా ఎత్తైనది ఉంది
      // → leftMax మాత్రమే నిర్ణయిస్తుంది
      leftMax = Math.max(leftMax, height[l]);
      water += leftMax - height[l];
      l++;
    } else {
      rightMax = Math.max(rightMax, height[r]);
      water += rightMax - height[r];
      r--;
    }
  }
  return water;
}
// O(n) time, O(1) space – DP approach O(n) space కంటే మెరుగు

```

ఈ pattern కి చెందిన problems

Easy: [Valid Palindrome](#) · [Two Sum II \(sorted\)](#) · [Reverse String](#) · [Merge Sorted Array](#)
[Squares of a Sorted Array](#) · [Is Subsequence](#) · [Remove Duplicates](#)
 Medium: [3Sum](#) · [3Sum Closest](#) · [4Sum](#) · [Container With Most Water](#) · [Sort Colors](#)
[Partition Labels](#) · [Boats to Save People](#) · [Longest Palindromic Substring \(expand\)](#)
 Hard: [Trapping Rain Water](#)

Gotchas (సాధారణ తప్పులు)

- **while (left <= right)** — ఒకే element ని రెండుసార్లు వాడతారు (2Sum లో తప్పు).
- **Sort చేయడం మర్చిపోవడం** — two pointers logic sorted array మీద ఆధారపడుతుంది.
- **Duplicates handle చేయకపోవడం** (3Sum) — duplicate triplets.
- **Index return చేయాల్సి వచ్చినప్పుడు sort చేయడం** — indices పోతాయి! (అప్పుడు hash map వాడాలి).
- **Palindrome లో inner while loops కి l < r guard లేకపోవడం** → out of bounds.
- **Container problem లో పెద్ద height ఉన్న వైపు కదపడం** — greedy proof అర్థం కాకపోవడం.

Interview దృష్టి

- "ಎಂದುಕು two pointers పని చేస్తుంది?" → sorted + monotonic behaviour → ప్రతి step లో ఒక అవకాశాన్ని safely తొలగించు.
- "Two Sum unsorted అయితే?" → hash map $O(n)$; sorted అయితే two pointers $O(1)$ space.
- "Container problem లో తక్కువ ఎత్తు వైపు ఎందుకు కదపాలి?" → ఎక్కువ ఉన్నది కదిపితే area ఎప్పటికీ పెరగదని నిరూపించడం.

2. Fast & Slow Pointers

🔍 Recognition signals

- ✓ Linked list లో "cycle" ఉందా అని కనుక్కోవాలి
- ✓ "middle node" కావాలి (ఒకే traversal లో)
- ✓ "nth node from end"
- ✓ "happy number" / సంఖ్యల sequence లో cycle
- ✓ Constant space లో cycle detection

వివరణ

రెండు pointers ఒకే దిశలో వేర్వేరు వేగంతో కదులుతాయి — slow ఒక అడుగు, fast రెండు అడుగులు. **Floyd's Cycle Detection (tortoise & hare)** అని పిలుస్తారు.

Real-life Scenario

Circular track మీద ఇద్దరు పరుగెత్తుతున్నారు — ఒకరు నెమ్మది, ఒకరు రెట్టింపు వేగం. **Track వృత్తాకారం అయితే** (cycle), వేగవంతుడు ఎప్పుడో ఒకప్పుడు నెమ్మదివాడిని **వెనుక నుండి వచ్చి కలుస్తాడు** (lap). **Track సూటిగా ఉంటే** (no cycle), వేగవంతుడు చివరికి చేరి ఆగిపోతాడు — ఎప్పటికీ కలవరు.

📄 Template 1 — Cycle detection

```
function hasCycle(head) {
  let slow = head, fast = head;

  while (fast && fast.next) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow === fast) return true;
  }
  return false;
}
//  $O(n)$  time,  $O(1)$  space
```

// ⚠️ రెండూ check (fast రెండు అడుగులు వేస్తుంది)

// కలిశారు → cycle ✓

// fast చివరికి చేరింది → cycle లేదు

📄 Template 2 — Cycle start ని కనుగొనడం (గణిత అందం 🌟)

```
function detectCycleStart(head) {  
    let slow = head, fast = head;  
  
    // Phase 1: meeting point కనుగొనడం  
    while (fast && fast.next) {  
        slow = slow.next;  
        fast = fast.next.next;  
        if (slow === fast) {  
            // Phase 2: ఒకరిని head కి తీసుకెళ్లి, ఇద్దరూ ఒకే వేగంతో కదిలితే –  
            // cycle start దగ్గర కలుస్తారు!  
            let p = head;  
            while (p !== slow) { p = p.next; slow = slow.next; }  
            return p;  
        }  
    }  
    return null;  
}
```

ఎందుకు పని చేస్తుంది (proof – interview లో అడుగుతారు):

head నుండి cycle start వరకు దూరం = a
cycle start నుండి meeting point వరకు = b
meeting point నుండి cycle start వరకు (మిగతా cycle) = c

Slow ప్రయాణించింది: a + b
Fast ప్రయాణించింది: a + b + (b + c) [ఒక lap ఎక్కువ]
Fast = 2 × Slow → a + 2b + c = 2(a + b) → c = a 🌟

అంటే: meeting point నుండి cycle start వరకు దూరం = head నుండి cycle start వరకు దూరం!

📄 Template 3 — Middle node

```
function findMiddle(head) {  
    let slow = head, fast = head;  
    while (fast && fast.next) { slow = slow.next; fast = fast.next.next; }  
    return slow; // even length అయితే రెండో middle  
}  
// మొదటి middle కావాలంటే: while (fast.next && fast.next.next)
```

📄 Template 4 — Nth node from end (two pointers, gap technique)

```
function removeNthFromEnd(head, n) {  
    const dummy = { next: head }; // 🔑 head తొలగించాల్సి వస్తే ఉపయోగం  
    let fast = dummy, slow = dummy;  
  
    for (let i = 0; i <= n; i++) fast = fast.next; // gap = n+1  
    while (fast) { fast = fast.next; slow = slow.next; }  
  
    slow.next = slow.next.next; // slow ఇప్పుడు తొలగించాల్సిన దానికి ముందు
```

```
    return dummy.next;
}
```

Variant — Linked list కాని చోట (Happy Number)

```
function isHappy(n) {
    const next = (x) => String(x).split('').reduce((s, d) => s + d * d, 0);
    let slow = n, fast = n;
    do {
        slow = next(slow);
        fast = next(next(fast));
    } while (slow !== fast);
    return slow === 1; // cycle 1 దగ్గర అగితే happy ✅
}
// 🔑 "sequence + cycle" ఉన్న ఏ problem క్షణా ఈ pattern వర్తిస్తుంది (linked list అవసరం లేదు)
```

ఈ pattern కి చెందిన problems

Easy: [Linked List Cycle](#) · [Middle of Linked List](#) · [Happy Number](#) · [Palindrome Linked List](#)
Medium: [Linked List Cycle II](#) · [Remove Nth From End](#) · [Reorder List](#) · [Find Duplicate Number](#) ★
[Circular Array Loop](#)

Find the Duplicate Number (LC 287) — array ని linked list గా చూడటం ($i \rightarrow \text{nums}[i]$) — ఈ pattern యొక్క అత్యంత తెలివైన అనువర్తనం. $O(1)$ space లో duplicate కనుగొనడం.

Gotchas

- **while (fast && fast.next)** — **fast.next** check లేకపోతే **fast.next.next** crash.
- **slow = head, fast = head.next** తో మొదలుపెట్టడం — cycle start logic పాడవుతుంది (రెండూ **head** నుండే).
- **Middle definition** — even length లో మొదటిదా రెండోదా? (problem చదవాలి).
- **Dummy node లేకుండా head deletion** → edge case fail.

Interview దృష్టి

- "Cycle start ఎందుకు అలా వస్తుంది?" → ఫై proof ($c = a$) — ఇది చెప్పే బలమైన impression.
- "Hash set తో కూడా చేయొచ్చు కదా?" → అవును $O(n)$ space; fast-slow $O(1)$ space ✅.
- "Find Duplicate Number ని ఎలా cycle problem గా మార్చావు?" → $i \rightarrow \text{nums}[i]$ mapping.

3. Sliding Window ★

🔍 Recognition signals (అత్యంత ముఖ్యమైన pattern)

- ✅ "contiguous subarray" లేదా "substring" (subsequence కాదు!)
- ✅ "longest / shortest / maximum / minimum" + condition
- ✅ "at most K" / "exactly K" / "containing all characters"
- ✅ Fixed size window ("size k యొక్క")
- ⚠️ Negative numbers ఉంటే sliding window సాధారణంగా పని చేయదు → prefix sum

వివరణ

Brute force లో అన్ని subarrays చూస్తే $O(n^2)$. **Sliding window** – window ని ఒక చివర పెంచుతూ, అవసరమైనప్పుడు ఇంకో చివర కుదిస్తూ $O(n)$ లో పని ముగిస్తుంది. ప్రతి element గరిష్ఠంగా రెండుసార్లు touch అవుతుంది (ఒకసారి right తో, ఒకసారి left తో).

Real-life Scenario

Sliding window = రైలు కిటికీ నుండి చూడటం. రైలు కదులుతున్నప్పుడు – కొత్త దృశ్యం కుడి వైపు నుండి వస్తుంది ($right++$), పాత దృశ్యం ఎడమ వైపు నుండి పోతుంది ($left++$). మీరు ప్రతిసారి మొత్తం ప్రయాణాన్ని మళ్ళీ చూడరు – కిటికీలో ఉన్నది మాత్రమే track చేస్తారు. అదే $O(n)$.

📄 Template 1 – Fixed size window

```
function maxSumSubarray(arr, k) {
    let windowSum = 0, maxSum = -Infinity;

    for (let i = 0; i < arr.length; i++) {
        windowSum += arr[i]; // కుడి వైపు నుండి add

        if (i >= k - 1) { // window పూర్తయింది
            maxSum = Math.max(maxSum, windowSum);
            windowSum -= arr[i - k + 1]; // ఎడమ వైపు నుండి remove
        }
    }
    return maxSum;
}
// O(n) time, O(1) space
```

📄 Template 2 – Variable size window (అత్యంత ముఖ్యం ★)

```
function variableWindow(arr) {
    let left = 0, result = 0;
    const state = new Map(); // window state (frequency/sum/count)

    for (let right = 0; right < arr.length; right++) {
        // 📌 EXPAND – కుడి element ని window లోకి
```

```

state.set(arr[right], (state.get(arr[right]) ?? 0) + 1);

// 2 SHRINK - condition ఉల్లంఘించినంతసేపు (⚠ while, if కాదు!)
while (/* window invalid */) {
  state.set(arr[left], state.get(arr[left]) - 1);
  if (state.get(arr[left]) === 0) state.delete(arr[left]);
  left++;
}

// 3 UPDATE - valid window కి result
result = Math.max(result, right - left + 1);
}
return result;
}

```

ఈ మూడు దశలు (expand → shrink → update) గుర్తుపెట్టుకుంటే, 80% sliding window problems solve చేయగలరు.

ఉదాహరణ 1 — Longest substring without repeating characters

```

function lengthOfLongestSubstring(s) {
  const lastSeen = new Map();
  let left = 0, max = 0;

  for (let right = 0; right < s.length; right++) {
    const ch = s[right];
    // 🔑 duplicate కనిపిస్తే - left ని ఆ character తర్వాతికి *దూకించడం* (ఒక్కొక్కటిగా కాదు)
    if (lastSeen.has(ch) && lastSeen.get(ch) >= left) {
      left = lastSeen.get(ch) + 1;
    }
    lastSeen.set(ch, right);
    max = Math.max(max, right - left + 1);
  }
  return max;
}
// O(n) time, O(min(n, charset)) space

```

ఉదాహరణ 2 — Minimum window substring (Hard, కానీ template సూటిగా)

```

function minWindow(s, t) {
  if (t.length > s.length) return "";

  const need = new Map();
  for (const c of t) need.set(c, (need.get(c) ?? 0) + 1);

  let required = need.size; // ఎన్ని distinct chars satisfy కావాలి
  let formed = 0; // ఎన్ని satisfy అయ్యాయి
  const window = new Map();
  let left = 0, minLen = Infinity, minStart = 0;

  for (let right = 0; right < s.length; right++) {
    const c = s[right];

```

```

window.set(c, (window.get(c) ?? 0) + 1);
if (need.has(c) && window.get(c) === need.get(c)) formed++; // 🔑 === (>= కాదు)

while (formed === required) { // valid window → కుదించడానికి ప్రయత్నం
    if (right - left + 1 < minLen) { minLen = right - left + 1; minStart = left; }

    const lc = s[left];
    window.set(lc, window.get(lc) - 1);
    if (need.has(lc) && window.get(lc) < need.get(lc)) formed--;
    left++;
}
}
return minLen === Infinity ? "" : s.slice(minStart, minStart + minLen);
}

```

★ "Exactly K" ట్రిక్ (అత్యంత ఉపయోగకరమైన అంతర్దృష్టి)

```

// "exactly K distinct" ని నేరుగా sliding window తో చేయడం కష్టం.
// ✅ ట్రిక్: exactly(K) = atMost(K) - atMost(K-1)

function subarraysWithKDistinct(nums, k) {
    return atMostK(nums, k) - atMostK(nums, k - 1);
}

function atMostK(nums, k) {
    const count = new Map();
    let left = 0, res = 0;

    for (let right = 0; right < nums.length; right++) {
        count.set(nums[right], (count.get(nums[right]) ?? 0) + 1);

        while (count.size > k) {
            count.set(nums[left], count.get(nums[left]) - 1);
            if (count.get(nums[left]) === 0) count.delete(nums[left]);
            left++;
        }
        res += right - left + 1; // 🔑 ఈ right తో ముగిసే valid subarrays సంఖ్య
    }
    return res;
}

```

res += right - left + 1 — ఇది "ఈ right తో ముగిసే subarrays ఎన్ని?" అని లెక్కిస్తుంది. **Counting sliding window problems** అన్నిటికీ ఇదే ట్రిక్.

⚠️ Sliding window ఎప్పుడు పని చేయదు

```

// ❌ Negative numbers ఉంటే — window ని కుదిస్తే sum పెరగొచ్చు (monotonicity పోతుంది)
// ఉదా: [1, -1, 5], target sum = 5 → window shrink logic విఫలం

// ✅ పరిష్కారం: prefix sum + hash map (Pattern 4)

```

ఈ pattern కి చెందిన problems

Fixed: Maximum Average Subarray · Find All Anagrams · Permutation in String
Sliding Window Maximum (+ monotonic deque)

Variable: Longest Substring Without Repeating · Minimum Size Subarray Sum
Longest Repeating Character Replacement · Fruit Into Baskets
Max Consecutive Ones III · Longest Substring with At Most K Distinct

Hard: Minimum Window Substring · Substring with Concatenation of All Words
Subarrays with K Different Integers (exactly-K ట్రిక్)

Gotchas

- **while** బదులు **if** shrink లో → window invalid గా మిగిలిపోతుంది.
- **Negative numbers** తో **sliding window** → prefix sum వాడాలి.
- **"subsequence" ని "substring" అనుకోవడం** – subsequence కి sliding window పని చేయదు (DP).
- **Window size లెక్కింపు** – **right - left + 1** (+1 మర్చిపోవడం సాధారణం).
- **Map count 0 అయినప్పుడు delete చేయకపోవడం** → map.size తప్పు అవుతుంది.
- **formed update లో >= వాడటం** – **==** ఉండాలి (లేకపోతే over-count).

Interview దృష్టి

- **"Sliding window ఎందుకు $O(n)$?"** → ప్రతి element గరిష్టంగా 2 సార్లు touch (right ఒకసారి, left ఒకసారి) → amortized $O(n)$.
- **"Exactly K ఎలా?"** → **atMost(K) - atMost(K-1)** – ఇది తెలిస్తే senior signal.
- **"Negatives ఉంటే?"** → sliding window invalid → prefix sum + hash map.

4. Prefix Sum & Difference Array

🔍 Recognition signals

- ✓ "subarray sum equals K" (ముఖ్యంగా ****negative numbers**** ఉంటే)
- ✓ అనేక **range queries** ("sum from i to j") – పదే పదే
- ✓ "count of subarrays with property X"
- ✓ "range update" (difference **array**)
- ✓ 2D **grid** లో **submatrix sum**

వివరణ

Prefix sum = $\text{prefix}[i]$ = మొదటి **i** elements యొక్క sum. దీనితో **ఏ range sum అయినా $O(1)$ లో**:

```
sum(i..j) = prefix[j+1] - prefix[i]
```

Real-life Scenario

Prefix sum = milestone markers on a highway. ప్రతి milestone దగ్గర "Chennai నుండి ఇక్కడి వరకు 240 km" అని రాసి ఉంటుంది. **రెండు నగరాల మధ్య దూరం కావాలంటే** – మొత్తం మళ్ళీ కొలవరు, **రెండు markers ని తీసేస్తారు:** $380 - 240 = 140$ km . Preprocessing ఒకసారి, queries అనంతం.

Template 1 — Basic prefix sum

```
function buildPrefix(nums) {
  const prefix = new Array(nums.length + 1).fill(0);    // 🗝️ size n+1 (offset తో edge cases సులభం)
  for (let i = 0; i < nums.length; i++) prefix[i + 1] = prefix[i] + nums[i];
  return prefix;
}
const rangeSum = (prefix, i, j) => prefix[j + 1] - prefix[i];    // O(1) ✅
```

Template 2 — Subarray sum equals K (prefix + hash map) ★

```
function subarraySum(nums, k) {
  const seen = new Map([[0, 1]]);    // 🗝️ prefix 0 ఒకసారి "చూశాం" (subarray మొదటి నుండి)
  let sum = 0, count = 0;

  for (const num of nums) {
    sum += num;
    // sum - k అనే prefix ఎప్పుడైనా చూశామా? → అప్పటి నుండి ఇక్కడి వరకు sum = k ✅
    count += seen.get(sum - k) ?? 0;
    seen.set(sum, (seen.get(sum) ?? 0) + 1);
  }
  return count;
}
// O(n) time, O(n) space – negatives ఉన్నా పని చేస్తుంది ✅ (sliding window చేయదు!)
```

ఎందుకు పని చేస్తుంది:

$prefix[j] - prefix[i] = k \iff prefix[i] = prefix[j] - k$
అంటే: ప్రస్తుత **prefix** నుండి k తీసేస్తే వచ్చే విలువ ఇంతకుముందు ఎన్నిసార్లు వచ్చిందో, అన్ని **subarrays** ఇక్కడ ముగుస్తాయి.

Variant — Divisible by K / same remainder

```
function subarraysDivByK(nums, k) {
  const count = new Map([[0, 1]]);
  let sum = 0, res = 0;
  for (const num of nums) {
    sum += num;
    const mod = ((sum % k) + k) % k;    // ⚠️ JS లో negative modulo fix
    res += count.get(mod) ?? 0;
    count.set(mod, (count.get(mod) ?? 0) + 1);
  }
}
```

```

    return res;
}
// 🔑 "ఒకే remainder ఉన్న రెండు prefixes మధ్య subarray k చేత భాగించబడుతుంది"

```

Variant — Binary array లో equal 0s and 1s

```

function findMaxLength(nums) {
    const first = new Map([[0, -1]]); // prefix → మొదటిసారి కనిపించిన index
    let sum = 0, max = 0;
    for (let i = 0; i < nums.length; i++) {
        sum += nums[i] === 0 ? -1 : 1; // 🔑 0 ని -1 గా treat → sum 0 = equal count
        if (first.has(sum)) max = Math.max(max, i - first.get(sum));
        else first.set(sum, i); // ⚠ మొదటి occurrence మాత్రమే (longest కోసం)
    }
    return max;
}

```

📁 Template 3 — Difference Array (range updates $O(1)$)

```

// సమస్య: [start, end] range కి val add చేయడం, అనేకసార్లు → చివర్లో final array
function rangeUpdates(n, updates) {
    const diff = new Array(n + 1).fill(0);

    for (const [start, end, val] of updates) {
        diff[start] += val; // ✅  $O(1)$  per update!
        diff[end + 1] -= val;
    }

    const result = [];
    let running = 0;
    for (let i = 0; i < n; i++) { running += diff[i]; result.push(running); }
    return result;
}
//  $O(n + m)$  – naive approach  $O(n \times m)$  కంటే భారీగా మెరుగు
// ఉపయోగాలు: Corporate Flight Bookings, Car Pooling, Range Addition

```

📁 Template 4 — 2D Prefix Sum (submatrix sums)

```

function build2DPrefix(matrix) {
    const m = matrix.length, n = matrix[0].length;
    const p = Array.from({ length: m + 1 }, () => new Array(n + 1).fill(0));

    for (let i = 0; i < m; i++)
        for (let j = 0; j < n; j++)
            p[i + 1][j + 1] = matrix[i][j] + p[i][j + 1] + p[i + 1][j] - p[i][j]; // 🔑 inclusion-exclusion
    return p;
}

const submatrixSum = (p, r1, c1, r2, c2) =>
    p[r2 + 1][c2 + 1] - p[r1][c2 + 1] - p[r2 + 1][c1] + p[r1][c1];

```

Sliding Window vs Prefix Sum — ఎప్పుడు ఏది

	Sliding Window	Prefix Sum
Negatives	✗	✓
Space	O(1)	O(n)
"longest/shortest"	✓	కష్టం
"count of subarrays"	పరిమితం	✓
Range queries (అనేకసార్లు)	✗	✓

ఈ pattern కి చెందిన problems

Easy: Running Sum · Range Sum Query (immutable) · Find Pivot Index
Medium: Subarray Sum Equals K ★ · Contiguous Array · Subarray Sums Divisible by K
Product of Array Except Self (prefix+suffix) · Range Sum Query 2D
Corporate Flight Bookings (difference array) · Car Pooling
Hard: Maximum Size Subarray Sum Equals k · Count of Range Sum

Gotchas

- map.set(0, 1) initialization మర్చిపోవడం → array మొదటి నుండి మొదలయ్యే subarrays miss.
- JS negative modulo — $-7 \% 3 = -1$ (Python లా -2 కాదు) → $((x \% k) + k) \% k$.
- "longest" కి first occurrence, "count" కి frequency — Map లో ఏం store చేయాలో గందరగోళం.
- Prefix array size n వాడటం — n+1 వాడితే edge cases సులభం.
- 2D లో inclusion-exclusion తప్పు — +p[i][j] మర్చిపోవడం.

Interview దృష్టి

- "Subarray sum = K ని sliding window తో చేయొచ్చా?" → negatives లేకపోతే అవును; ఉంటే prefix+hashmap తప్పనిసరి — ఈ తేడా చెప్తే బలమైన signal.
- "Range updates అనేకసార్లు?" → difference array O(1) per update.

5. Kadane's Algorithm & Running Optimum

🔍 Recognition signals

- ✓ "maximum/minimum sum subarray" (contiguous)
- ✓ "maximum product subarray"
- ✓ "best time to buy and sell stock"
- ✓ ప్రతి index దగ్గర "ఇక్కడ ముగిసే ఉత్తమమైనది" అనే ఆలోచన

వివరణ

Kadane = ఒక DP, కానీ $O(1)$ space. ప్రతి index దగ్గర ఒకే నిర్ణయం: "ఇక్కడి నుండి కొత్తగా మొదలుపెట్టాలా, లేక ముందుది కొనసాగించాలా?"

Real-life Scenario

Kadane = ప్రయాణంలో సహచరుడిని ఎంచుకోవడం. మీరు ముందుకు వెళ్తున్నారు; ఇప్పటివరకు మీతో ఉన్న group నష్టం కలిగిస్తుంటే (negative sum), వాళ్ళను వదిలేసి ఇక్కడి నుండి కొత్తగా మొదలుపెట్టడం మేలు. లాభం ఇస్తుంటే — కొనసాగించండి. ప్రతి అడుగులో ఈ ఒక్క నిర్ణయమే.

Template — Maximum subarray sum

```
function maxSubArray(nums) {  
  let maxEndingHere = nums[0];           // ఈ index తో ముగిసే గరిష్ట sum  
  let maxSoFar = nums[0];                 // మొత్తంలో గరిష్టం  
  
  for (let i = 1; i < nums.length; i++) {  
    // 🔑 కీలక నిర్ణయం: కొత్తగా మొదలాలా, కొనసాగించాలా?  
    maxEndingHere = Math.max(nums[i], maxEndingHere + nums[i]);  
    maxSoFar = Math.max(maxSoFar, maxEndingHere);  
  }  
  return maxSoFar;  
}  
//  $O(n)$  time,  $O(1)$  space
```

Variant — Subarray indices కూడా కావాలంటే

```
function maxSubArrayWithIndices(nums) {  
  let maxEnd = nums[0], maxSoFar = nums[0];  
  let start = 0, end = 0, tempStart = 0;  
  
  for (let i = 1; i < nums.length; i++) {  
    if (nums[i] > maxEnd + nums[i]) { maxEnd = nums[i]; tempStart = i; }  
    else maxEnd += nums[i];  
  
    if (maxEnd > maxSoFar) { maxSoFar = maxEnd; start = tempStart; end = i; }  
  }  
  return { sum: maxSoFar, start, end };  
}
```

Variant — Maximum product subarray (min ని కూడా track చేయాలి!)

```
function maxProduct(nums) {  
  let maxSoFar = nums[0], curMax = nums[0], curMin = nums[0];  
  
  for (let i = 1; i < nums.length; i++) {  
    const n = nums[i];  
    const tempMax = curMax;  
    curMax = Math.max(n, tempMax * n, tempMin * n);  
    curMin = Math.min(n, tempMax * n, tempMin * n);  
  }  
  return curMax;  
}
```

```
// 🗝️ negative × negative = positive → చిన్నదే పెద్దది అవ్వచ్చు!
curMax = Math.max(n, tempMax * n, curMin * n);
curMin = Math.min(n, tempMax * n, curMin * n);
maxSoFar = Math.max(maxSoFar, curMax);
}
return maxSoFar;
}
```

Variant — Circular array maximum sum

```
function maxSubarraySumCircular(nums) {
  let total = 0, maxEnd = 0, maxSum = -Infinity, minEnd = 0, minSum = Infinity;

  for (const n of nums) {
    total += n;
    maxEnd = Math.max(n, maxEnd + n); maxSum = Math.max(maxSum, maxEnd);
    minEnd = Math.min(n, minEnd + n); minSum = Math.min(minSum, minEnd);
  }
  // 🗝️ circular max = total - (minimum subarray)
  // ⚠️ అన్నీ negative అయితే - total - minSum = 0 (ఖాళీ subarray) → invalid
  return maxSum > 0 ? Math.max(maxSum, total - minSum) : maxSum;
}
```

Variant — Best Time to Buy and Sell Stock (same idea)

```
function maxProfit(prices) {
  let minPrice = Infinity, maxProfit = 0;
  for (const p of prices) {
    minPrice = Math.min(minPrice, p); // ఇప్పటివరకు అత్యల్ప ధర
    maxProfit = Math.max(maxProfit, p - minPrice);
  }
  return maxProfit;
}
// 🗝️ ఇది Kadane యొక్క ఇంకో ముఖం: "running minimum + current" pattern
```

ఈ pattern కి చెందిన problems

Easy: Maximum Subarray · Best Time to Buy and Sell Stock
 Medium: Maximum Product Subarray · Maximum Sum Circular Subarray
 Best Time to Buy and Sell Stock II/with Cooldown/with Fee
 Maximum Absolute Sum of Any Subarray · K-Concatenation Maximum Sum

Gotchas

- `maxSoFar = 0` తో initialize చేయడం → అన్నీ negative అయితే తప్పు సమాధానం (0). `nums[0]` తో మొదలుపెట్టాలి.
- Product లో min ని track చేయకపోవడం → negatives తో fail.
- Circular లో అన్నీ negative case handle చేయకపోవడం.

- Kadane ని subsequence కి వాడటం — ఇది contiguous కి మాత్రమే.

Interview దృష్టి

- "Kadane ఒక DP ఆ?" → అవును — $dp[i] = \max(nums[i], dp[i-1] + nums[i])$, space optimized to $O(1)$.
- "అన్నీ negative అయితే?" → $nums[0]$ initialization; ఇది classic edge case.

6. Cyclic Sort (Index-as-Hash)

🔍 Recognition signals

- ✓ Array లో numbers **1 to n** (లేదా **0 to n-1**) range లో ఉన్నాయి
- ✓ "missing number" / "duplicate number" / "first missing positive"
- ✓ " $O(n)$ time and $O(1)$ space" అని ప్రత్యేకంగా చెప్పారు

వివరణ

Numbers **1..n** range లో ఉంటే — **ప్రతి number కి దాని సరైన స్థానం తెలుసు** (value $v \rightarrow$ index $v-1$). అందుకే **array నే hash map గా వాడొచ్చు** — extra space అవసరం లేదు.

Real-life Scenario

Cyclic sort = theatre లో seat numbers. ప్రతి టికెట్ మీద seat number ఉంది. అందరూ తప్పు seats లో కూర్చున్నారు. **పరిష్కారం:** ఒక్కొక్కరిని తీసుకొని "నీ seat ఇది" అని సరైన చోట కూర్చోబెట్టండి; అక్కడ ఇంకొకరు ఉంటే వాళ్ళని లేపి వాళ్ళ seat కి పంపండి. **చివరికి — ఖాళీగా ఉన్న seat = missing person; రెండు టికెట్లు ఒకే number = duplicate.**

📄 Template — Cyclic sort

```
function cyclicSort(nums) {           // numbers 1..n
  let i = 0;
  while (i < nums.length) {
    const correctIdx = nums[i] - 1;    // value v → index v-1
    if (nums[i] !== nums[correctIdx]) {
      [nums[i], nums[correctIdx]] = [nums[correctIdx], nums[i]]; // swap
      // ⚠️ i++ చేయొద్దు — కొత్తగా వచ్చిన element ని కూడా check చేయాలి
    } else {
      i++;
    }
  }
  return nums;
}
//  $O(n)$  — ప్రతి swap ఒక element ని శాశ్వతంగా సరైన చోట పెడుతుంది → గరిష్టంగా n swaps
```

⚠️ `nums[i] !== nums[correctIdx]` (indices compare కాదు, values compare) – duplicates ఉన్నప్పుడు infinite loop నివారణ.

Variant — Missing number(s)

```
function findDisappearedNumbers(nums) {
  let i = 0;
  while (i < nums.length) {
    const j = nums[i] - 1;
    if (nums[i] !== nums[j]) [nums[i], nums[j]] = [nums[j], nums[i]];
    else i++;
  }
  const missing = [];
  for (let k = 0; k < nums.length; k++) if (nums[k] !== k + 1) missing.push(k + 1);
  return missing;
}
```

Variant — First Missing Positive (Hard, కానీ pattern ఒకటే) ★

```
function firstMissingPositive(nums) {
  const n = nums.length;
  let i = 0;
  while (i < n) {
    const j = nums[i] - 1;
    // 🔑 1..n range లో ఉన్న valid numbers ని మాత్రమే place చేయడం
    if (nums[i] > 0 && nums[i] <= n && nums[i] !== nums[j]) {
      [nums[i], nums[j]] = [nums[j], nums[i]];
    } else i++;
  }
  for (let k = 0; k < n; k++) if (nums[k] !== k + 1) return k + 1;
  return n + 1;
}
// O(n) time, O(1) space ✅ – hash set వాడితే O(n) space
```

📁 Alternative — Negative marking (in-place hash)

```
// Array modify చేయొచ్చు, values 1..n → sign ని "visited" flag గా వాడటం
function findDuplicates(nums) {
  const res = [];
  for (const num of nums) {
    const idx = Math.abs(num) - 1;
    if (nums[idx] < 0) res.push(Math.abs(num)); // ఇప్పటికే mark అయింది → duplicate
    else nums[idx] = -nums[idx]; // mark as seen
  }
  return res;
}
// O(n) time, O(1) space
```

ఈ pattern కి చెందిన problems

Easy: Missing Number · Find All Numbers Disappeared in an Array
Medium: Find the Duplicate Number (cyclic sort లేదా fast-slow ★)
Find All Duplicates in an Array · Set Mismatch
Hard: First Missing Positive

Gotchas

- `i++` ని swap తర్వాత చేయడం → కొత్త element check కాదు.
- Indices compare చేయడం (`i !== correctIdx`) → duplicates తో infinite loop; values compare చేయాలి.
- Range బయటి values (negatives, > n) handle చేయకపోవడం (First Missing Positive లో కీలకం).
- Input ని modify చేయకూడదని constraint ఉంటే → ఈ pattern వాడలేం (fast-slow pointers లేదా binary search).

Interview దృష్టి

- "O(1) space ఎలా సాధించావు?" → array నే hash map గా (index-as-key) వాడటం.
- "Duplicate కనుగొనడానికి ఇంకో మార్గం?" → Floyd cycle detection (`i → nums[i]`), లేదా binary search on value range.

7. Merge Intervals & Sweep Line

🔍 Recognition signals

- ✓ "intervals" / "[start, end]" pairs
- ✓ "overlapping" / "merge" / "insert" / "non-overlapping"
- ✓ "meeting rooms" / "schedule" / "booking"
- ✓ "minimum number of X to cover/remove"

వివరణ

దాదాపు అన్ని interval problems **sort చేయడంతో మొదలవుతాయి** (సాధారణంగా `start` ప్రకారం, కొన్నిసార్లు `end` ప్రకారం). Sort చేశాక – ప్రతి interval ని ముందుదానితో పోల్చడమే.

Real-life Scenario

Merge intervals = calendar meetings కలపడం. మీ calendar లో 10-11, 10:30-12, 2-3 అని ఉంది. మొదటి రెండూ **overlap అవుతున్నాయి** → వాటిని ఒకటిగా (10-12) చూడాలి. **సూత్రం:** meetings ని **మొదలయ్యే సమయం ప్రకారం** వరుసలో పెట్టండి; తర్వాతిది ముందుదాని లోపల మొదలైతే – కలపండి.

📁 Template 1 — Merge overlapping intervals

```
function merge(intervals) {  
  if (intervals.length <= 1) return intervals;  
  intervals.sort((a, b) => a[0] - b[0]); // 🔑 start ప్రకారం sort  
  
  const result = [intervals[0]];  
  for (let i = 1; i < intervals.length; i++) {  
    const last = result[result.length - 1];  
    const cur = intervals[i];  
  
    if (cur[0] <= last[1]) { // overlap ✅  
      last[1] = Math.max(last[1], cur[1]); // ⚠️ max (cur[1] కాదు - nested కావచ్చు)  
    } else {  
      result.push(cur);  
    }  
  }  
  return result;  
}  
// O(n log n) - sorting dominates
```

📁 Template 2 — Insert interval (sorted list లోకి)

```
function insert(intervals, newInterval) {  
  const res = [];  
  let i = 0, n = intervals.length;  
  
  // 1. newInterval కి ముందు ఉన్నవి  
  while (i < n && intervals[i][1] < newInterval[0]) res.push(intervals[i++]);  
  
  // 2. overlap అయ్యేవన్నీ merge  
  while (i < n && intervals[i][0] <= newInterval[1]) {  
    newInterval[0] = Math.min(newInterval[0], intervals[i][0]);  
    newInterval[1] = Math.max(newInterval[1], intervals[i][1]);  
    i++;  
  }  
  res.push(newInterval);  
  
  // 3. మిగతావి  
  while (i < n) res.push(intervals[i++]);  
  return res;  
}  
// O(n) - ఇప్పటికే sorted
```

📁 Template 3 — Non-overlapping (greedy: end ప్రకారం sort) ★

```
function eraseOverlapIntervals(intervals) {  
  if (!intervals.length) return 0;  
  intervals.sort((a, b) => a[1] - b[1]); // 🔑 END ప్రకారం! (start కాదు)  
  
  let count = 0, prevEnd = intervals[0][1];  
  for (let i = 1; i < intervals.length; i++) {  
    if (intervals[i][0] < prevEnd) count++; // overlap → ఈ interval తొలగించు  
    prevEnd = intervals[i][1];  
  }  
  return count;  
}
```

```

    else prevEnd = intervals[i][1];
  }
  return count;
}
// 🗝 ఎందుకు end ప్రకారం: ముందుగా ముగిసేది ఎంచుకుంటే – తర్వాతి వాటికి ఎక్కువ స్థలం మిగులుతుంది
// (ఇది classic greedy "activity selection")

```

📅 Template 4 — Sweep Line (meeting rooms II) ★

```

function minMeetingRooms(intervals) {
  const events = [];
  for (const [start, end] of intervals) {
    events.push([start, 1]); // meeting మొదలు → +1 room
    events.push([end, -1]); // meeting ముగింపు → -1 room
  }
  // ⚠ ఒకే సమయంలో end మరియు start ఉంటే – end ముందు (room reuse అవుతుంది)
  events.sort((a, b) => a[0] - b[0] || a[1] - b[1]);

  let rooms = 0, maxRooms = 0;
  for (const [, delta] of events) {
    rooms += delta;
    maxRooms = Math.max(maxRooms, rooms);
  }
  return maxRooms;
}
// O(n log n)
// 🗝 Sweep line = "timeline మీద ఎడమ నుండి కుడికి నడుస్తూ events process చేయడం"

```

ప్రత్యామ్నాయం – min-heap తో:

```

function minMeetingRoomsHeap(intervals) {
  intervals.sort((a, b) => a[0] - b[0]);
  const heap = new MinHeap(); // ముగిసే సమయాలు
  for (const [start, end] of intervals) {
    if (heap.size && heap.peak() <= start) heap.pop(); // ఒక room ఖాళీ అయింది
    heap.push(end);
  }
  return heap.size; // ఏకకాలంలో ఎన్ని rooms
}

```

Overlap check — గుర్తుపెట్టుకోవలసిన సూత్రం

```

// రెండు intervals [a1,a2], [b1,b2] overlap అవుతాయా?
const overlaps = a1 <= b2 && b1 <= a2;
// Overlap లేదు అంటే: a2 < b1 || b2 < a1 (ఒకటి పూర్తిగా ముందు)
// Intersection: [Math.max(a1,b1), Math.min(a2,b2)]

```

ఈ pattern కి చెందిన problems

Easy: Meeting Rooms (I) · Summary Ranges
Medium: Merge Intervals ★ · Insert Interval · Non-overlapping Intervals
Meeting Rooms II ★ · Minimum Arrows to Burst Balloons · Interval List Intersections
Car Pooling (difference array) · My Calendar I
Hard: Employee Free Time · Data Stream as Disjoint Intervals · My Calendar III

Gotchas

- **Sort చేయకపోవడం** – దాదాపు అన్ని interval problems కి sort తప్పనిసరి.
- **start vs end ప్రకారం sort** – merge కి start, non-overlapping/greedy కి end.
- **last[1] = cur[1]** రాయడం – Math.max కావాలి (nested intervals: [1,10], [2,3]).
- **Boundary touching** – [1,2] మరియు [2,3] overlap అ? Problem చదవాలి (< vs <=).
- **Sweep line లో tie-breaking** – ఒకే సమయంలో start/end → end ముందు (లేకపోతే rooms ఎక్కువ లెక్కిస్తారు).

Interview దృష్టి

- "ఎందుకు end ప్రకారం sort?" → greedy activity selection proof – ముందుగా ముగిసేది ఎంచుకుంటే గరిష్ట intervals fit అవుతాయి.
- "Meeting Rooms II ని రెండు విధాలుగా చేయి" → sweep line (events) మరియు min-heap.

8. In-place Array Manipulation

🔍 Recognition signals

- ✓ "in-place" / "O(1) extra space"
- ✓ "rotate array" / "reverse"
- ✓ "remove/move elements" (duplicates, zeros)
- ✓ "sort colors" / partition
- ✓ Matrix "rotate 90°" / "spiral" / "set zeroes"

📅 Pattern A — Reversal trick (rotation)

```
function rotate(nums, k) {  
  k %= nums.length;           // ⚠️ k > n కావచ్చు  
  reverse(nums, 0, nums.length - 1); // మొత్తం reverse  
  reverse(nums, 0, k - 1);      // మొదటి k  
  reverse(nums, k, nums.length - 1); // మిగతావి  
}  
function reverse(arr, l, r) {  
  while (l < r) { [arr[l], arr[r]] = [arr[r], arr[l]]; l++; r--; }  
}  
// [1,2,3,4,5,6,7], k=3  
// reverse all: [7,6,5,4,3,2,1]
```

```
// reverse 0..2: [5,6,7,4,3,2,1]
// reverse 3..6: [5,6,7,1,2,3,4] ✓
// O(n) time, O(1) space
```

Pattern B — Read/Write pointers (removal)

```
function removeDuplicates(nums) { // sorted array
    let write = 1;
    for (let read = 1; read < nums.length; read++) {
        if (nums[read] !== nums[read - 1]) nums[write++] = nums[read];
    }
    return write; // కొత్త length
}

function moveZeroes(nums) {
    let write = 0;
    for (let read = 0; read < nums.length; read++)
        if (nums[read] !== 0) nums[write++] = nums[read];
    while (write < nums.length) nums[write++] = 0;
}
// 🔑 "read pointer scans, write pointer places" – removal problems అన్నిటినీ ఇదే
```

Pattern C — Dutch National Flag (3-way partition)

```
function sortColors(nums) { // 0s, 1s, 2s
    let low = 0, mid = 0, high = nums.length - 1;

    while (mid <= high) { // ⚠️ <= (high వరకు)
        if (nums[mid] === 0) { [nums[low], nums[mid]] = [nums[mid], nums[low]]; low++; mid++; }
        else if (nums[mid] === 1) mid++;
        else { [nums[mid], nums[high]] = [nums[high], nums[mid]]; high--; }
        // ⚠️ ఇక్కడ mid++ చేయొద్దు – high నుండి వచ్చినది ఇంకా check కాలేదు
    }
}
// O(n) single pass, O(1) space
// Invariant: [0..low-1]=0s, [low..mid-1]=1s, [high+1..n-1]=2s, [mid..high]=unknown
```

Pattern D — Matrix rotation (transpose + reverse)

```
function rotate90(matrix) {
    const n = matrix.length;
    // 1. Transpose (rows ↔ columns)
    for (let i = 0; i < n; i++)
        for (let j = i + 1; j < n; j++) // ⚠️ j = i+1 (రెండుసార్లు swap కాకుండా)
            [matrix[i][j], matrix[j][i]] = [matrix[j][i], matrix[i][j]];

    // 2. ప్రతి row ని reverse
    for (const row of matrix) row.reverse();
}
// Anticlockwise: transpose + columns reverse (లేదా rows reverse + transpose)
```

Pattern E — Marking with sign/first row-col

```
function setZeroes(matrix) { // O(1) space
  const m = matrix.length, n = matrix[0].length;
  let firstRowZero = false, firstColZero = false;

  for (let j = 0; j < n; j++) if (matrix[0][j] === 0) firstRowZero = true;
  for (let i = 0; i < m; i++) if (matrix[i][0] === 0) firstColZero = true;

  // 🗝 first row/col ನಿ markers ಗೆ ವಾಡಿಕು
  for (let i = 1; i < m; i++)
    for (let j = 1; j < n; j++)
      if (matrix[i][j] === 0) { matrix[i][0] = 0; matrix[0][j] = 0; }

  for (let i = 1; i < m; i++)
    for (let j = 1; j < n; j++)
      if (matrix[i][0] === 0 || matrix[0][j] === 0) matrix[i][j] = 0;

  if (firstRowZero) for (let j = 0; j < n; j++) matrix[0][j] = 0;
  if (firstColZero) for (let i = 0; i < m; i++) matrix[i][0] = 0;
}
```

Pattern F — Spiral traversal (boundary shrinking)

```
function spiralOrder(matrix) {
  const res = [];
  let top = 0, bottom = matrix.length - 1, left = 0, right = matrix[0].length - 1;

  while (top <= bottom && left <= right) {
    for (let j = left; j <= right; j++) res.push(matrix[top][j]); // →
    top++;
    for (let i = top; i <= bottom; i++) res.push(matrix[i][right]); // ↓
    right--;
    if (top <= bottom) { // ⚠ guard
      for (let j = right; j >= left; j--) res.push(matrix[bottom][j]); // ←
      bottom--;
    }
    if (left <= right) { // ⚠ guard
      for (let i = bottom; i >= top; i--) res.push(matrix[i][left]); // ↑
      left++;
    }
  }
  return res;
}
```

ಈ pattern ನ್ನಿ ಪಂದಿಸಿ problems

Easy: [Remove Duplicates](#) · [Remove Element](#) · [Move Zeroes](#) · [Reverse String](#)
Medium: [Rotate Array](#) · [Sort Colors](#) · [Rotate Image](#) · [Spiral Matrix](#) · [Set Matrix Zeroes](#)
[Game of Life](#) · [Next Permutation](#)

Gotchas

- Rotation లో $k \% = n$ మర్చిపోవడం.
- Dutch flag లో high swap తర్వాత $mid++$ చేయడం (తప్పు).
- Transpose లో $j = 0$ వాడటం $\rightarrow$ రెండుసార్లు swap $\rightarrow$ మారదు.
- Spiral లో middle row/column guards లేకపోవడం $\rightarrow$ duplicates.
- Return value – length ఆ, array ఆ? (in-place problems లో సాధారణంగా length).

Interview దృష్టి

- "Rotate array 3 విధాలుగా చేయి" $\rightarrow$ extra array $O(n)$, cyclic replacements, **reversal trick** .
 - "Sort colors single pass లో ఎలా?" $\rightarrow$ Dutch national flag + invariant వివరణ.
-

Part 2 — Hashing & Counting

9. Hash Map Patterns

🔍 Recognition signals

- ✓ "Have I seen this before?" (duplicates, complement)
- ✓ Frequency/counting అవసరం
- ✓ $O(n^2)$ nested loops ని $O(n)$ కి తగ్గించాలి
- ✓ Grouping (anagrams, categories)
- ✓ Index ని గుర్తుపెట్టుకోవాలి (sorting చేయకూడదు)

వివరణ

Hash map = $O(1)$ lookup. Nested loop యొక్క inner loop ని hash lookup తో భర్తీ చేయడమే — ఇది DSA లో అత్యంత సాధారణ optimization.

$O(n^2)$ nested loop → "నాకు ఏమి కావాలో ముందే తెలుసు" (complement) → $O(n)$ hash lookup

Real-life Scenario

Hash map = ఇంటి తాళాల hook board. ప్రతి తాళం ఒక నిర్దిష్ట hook మీద. కావలసిన తాళం కోసం అన్ని hooks వెతకరు — దాని hook కి నేరుగా వెళ్తారు ($O(1)$). Board లేకపోతే — ప్రతిసారి ఇల్లంతా వెతకాలి ($O(n)$).

📁 Pattern A — Complement lookup (Two Sum family)

```
function twoSum(nums, target) {  
  const seen = new Map(); // value → index  
  for (let i = 0; i < nums.length; i++) {  
    const need = target - nums[i]; // 🔑 నాకు ఏమి కావాలో ముందే తెలుసు  
    if (seen.has(need)) return [seen.get(need), i];  
    seen.set(nums[i], i); // ⚠️ check తర్వాత add (ఒకే element రెండుసార్లు  
    వద్దు)  
  }  
  return [];  
}  
//  $O(n)$  time,  $O(n)$  space — sorting కంటే మేలు (indices నిలుస్తాయి)
```

📁 Pattern B — Frequency map

```
function isAnagram(s, t) {  
  if (s.length !== t.length) return false;  
  const count = new Map();  
  for (const c of s) count.set(c, (count.get(c) ?? 0) + 1);  
}
```

```

for (const c of t) {
  if (!count.has(c)) return false;
  count.set(c, count.get(c) - 1);
  if (count.get(c) === 0) count.delete(c);
}
return count.size === 0;
}

// ⚡ lowercase letters మాత్రమే అయితే – array వేగం (hash overhead లేదు)
function isAnagramFast(s, t) {
  if (s.length !== t.length) return false;
  const count = new Array(26).fill(0);
  for (let i = 0; i < s.length; i++) {
    count[s.charCodeAt(i) - 97]++;
    count[t.charCodeAt(i) - 97]--;
  }
  return count.every((c) => c === 0);
}

```

Pattern C — Grouping (key design ముఖ్యం)

```

function groupAnagrams(strs) {
  const groups = new Map();
  for (const str of strs) {
    // 🔑 Key design: anagrams కి ఒకే key ఎలా ఇవ్వాలి?
    const key = [...str].sort().join(""); // O(k log k)
    // ⚡ ప్రత్యామ్నాయం (వేగం): character count signature – O(k)
    // const cnt = new Array(26).fill(0);
    // for (const c of str) cnt[c.charCodeAt(0) - 97]++;
    // const key = cnt.join("#");

    if (!groups.has(key)) groups.set(key, []);
    groups.get(key).push(str);
  }
  return [...groups.values()];
}

```

Grouping problems లో అసలు పని = "సరైన key ఏమిటి?" అని కనుక్కోవడం. Anagrams → sorted string / char count. Diagonals → $i - j$. Anti-diagonals → $i + j$.

Pattern D — Seen set (duplicates, cycles)

```

function containsNearbyDuplicate(nums, k) {
  const lastIndex = new Map();
  for (let i = 0; i < nums.length; i++) {
    if (lastIndex.has(nums[i]) && i - lastIndex.get(nums[i]) <= k) return true;
    lastIndex.set(nums[i], i);
  }
  return false;
}

```

📁 Pattern E — Set తో $O(n)$ sequence (తెలివైన ఉపయోగం) ★

```
function longestConsecutive(nums) {
  const set = new Set(nums);
  let longest = 0;

  for (const num of set) {
    if (set.has(num - 1)) continue; // 🔑 sequence మొదలు కాదు → skip
    // ఇక్కడి నుండి మాత్రమే లెక్కిస్తాం → ప్రతి sequence ఒక్కసారి traverse →  $O(n)$  మొత్తం
    let cur = num, len = 1;
    while (set.has(cur + 1)) { cur++; len++; }
    longest = Math.max(longest, len);
  }
  return longest;
}
//  $O(n)$  - inner while ఉన్నా, మొత్తం iterations  $O(n)$  (amortized)
```

📁 Pattern F — Bidirectional mapping (isomorphic)

```
function isIsomorphic(s, t) {
  const mapST = new Map(), mapTS = new Map();
  for (let i = 0; i < s.length; i++) {
    const a = s[i], b = t[i];
    if (mapST.has(a) && mapST.get(a) !== b) return false;
    if (mapTS.has(b) && mapTS.get(b) !== a) return false; // 🔑 రెండు దిశలూ (one-to-one)
    mapST.set(a, b); mapTS.set(b, a);
  }
  return true;
}
```

Map vs Object vs Set — JavaScript-specific

	Map	Object	Set
Keys	ఏదైనా (objects కూడా)	string/symbol మాత్రమే	values
Size	.size ✅	Object.keys().length	.size
Iteration order	insertion ✅	numeric keys ముందు! ⚠️	insertion
Performance	frequent add/delete కి మేలు	property access కొంచెం వేగం	membership
Prototype pollution	✅ సురక్షితం	⚠️ __proto__ ప్రమాదం	సురక్షితం

DSA లో Map / Set వాడండి — object యొక్క numeric-key ordering surprise మరియు prototype issues నివారించడానికి.

ఈ pattern కి చెందిన problems

Easy: Two Sum · Valid Anagram · Contains Duplicate · Ransom Note · Intersection of Arrays
First Unique Character · Majority Element
Medium: Group Anagrams · Longest Consecutive Sequence ★ · Top K Frequent Elements
Subarray Sum Equals K · Isomorphic Strings · 4Sum II · Insert Delete GetRandom O(1)
Copy List with Random Pointer · LRU Cache
Hard: Substring with Concatenation of All Words · First Missing Positive (hash ప్రత్యామ్నాయం)

Gotchas

- `seen.set()` ని check కి ముందు — element తనతో తానే jodi అవుతుంది (Two Sum).
- Object ని map గా వాడి numeric keys → iteration order ఆశ్చర్యం.
- `map.get()` undefined check — `?? 0` వాడకపోవడం → NaN.
- Space complexity మర్చిపోవడం — hash map $O(n)$ space (interviewer అడుగుతాడు).
- Longest Consecutive లో `if (set.has(num-1)) continue` లేకపోవడం → $O(n^2)$.
- Grouping key design తప్పు → collisions లేదా అనవసర groups.

Interview దృష్టి

- "Two Sum ని hash map తో ఎందుకు, sorting తో కాదు?" → indices నిలుస్తాయి + $O(n) < O(n \log n)$.
- "Longest Consecutive $O(n)$ ఎలా?" → sequence starts మాత్రమే process — amortized argument.
- "Space-time trade-off?" → hash map = $O(n)$ space ఇచ్చి $O(n)$ time పొందడం.

10. Monotonic Stack ★

🔍 Recognition signals

- ✓ "next greater element" / "next smaller element"
- ✓ "previous greater/smaller"
- ✓ "largest rectangle" / "histogram"
- ✓ "daily temperatures" (ఎన్ని రోజుల్లో వేడి పెరుగుతుంది)
- ✓ "span" / "stock span"
- ✓ ప్రతి element కి "దాని కంటే పెద్దది/చిన్నది ఎక్కడ?" అనే ప్రశ్న

వివరణ

Monotonic stack = ఒక క్రమంలో (increasing/decreasing) ఉండే elements ని మాత్రమే ఉంచే stack. కొత్త element వచ్చినప్పుడు — ఆ క్రమాన్ని ఉల్లంఘించే elements ని pop చేస్తాం, అప్పుడే వాటి సమాధానం దొరుకుతుంది.

ఎందుకు $O(n)$: ప్రతి element గరిష్టంగా ఒకసారి push, ఒకసారి pop → $2n$ operations.

Real-life Scenario

Monotonic stack = వరుసలో నిలబడిన వ్యక్తులు ముందు చూడటం. అందరూ ఒకే వరుసలో నిలబడి ఉన్నారు, ప్రతి ఒక్కరూ "నా కంటే పొడవైన వ్యక్తి ముందు ఎవరు?" అని తెలుసుకోవాలి.

మీరు వరుస **వెనుక నుండి** నడుస్తూ వెళ్తారు. మీ చేతిలో ఒక జాబితా ఉంది. కొత్త వ్యక్తి వచ్చినప్పుడు — **అతని కంటే పొట్టివాళ్ళను జాబితా నుండి తీసేస్తారు** (వాళ్ళు ఇక ఎవరికీ "పొడవైనవాడు" కాలేరు — ఈ కొత్తవాడు వాళ్ళను మరుగున పెట్టేశాడు). అలా జాబితాలో ఎప్పుడూ **పొడవు క్రమంలో** ఉన్నవాళ్ళే మిగులుతారు.

Template — Next Greater Element

```
function nextGreaterElements(nums) {  
  const res = new Array(nums.length).fill(-1);  
  const stack = []; // indices (values కాదు — దూరం లెక్కించడానికి)  
  
  for (let i = 0; i < nums.length; i++) {  
    // 🔑 ప్రస్తుత element stack top కంటే పెద్దది → top కి సమాధానం దొరికింది  
    while (stack.length && nums[i] > nums[stack[stack.length - 1]]) {  
      const idx = stack.pop();  
      res[idx] = nums[i]; // లేదా (i - idx) దూరం కావాలంటే  
    }  
    stack.push(i);  
  }  
  return res; // stack లో మిగిలినవాటికి next greater లేదు (-1)  
}  
// O(n) time, O(n) space
```

Variants — ఒకే template, చిన్న మార్పులు:

కావలసినది	Stack క్రమం	Comparison
Next greater	decreasing	nums[i] > stack.top
Next smaller	increasing	nums[i] < stack.top
Previous greater	decreasing (కుడి నుండి ఎడమకి, లేదా push ముందు peek)	
Previous smaller	increasing	

ఉదాహరణ 1 — Daily Temperatures

```
function dailyTemperatures(temps) {  
  const res = new Array(temps.length).fill(0);  
  const stack = [];  
  for (let i = 0; i < temps.length; i++) {  
    while (stack.length && temps[i] > temps[stack[stack.length - 1]]) {  
      const j = stack.pop();  
      res[j] = i - j; // 🔑 ఎన్ని రోజులు (దూరం)  
    }  
    stack.push(i);  
  }  
}
```

```

    return res;
}

```

ఉదాహరణ 2 — Largest Rectangle in Histogram (Hard, classic) ★

```

function largestRectangleArea(heights) {
    const stack = []; // increasing heights (indices)
    let maxArea = 0;
    const h = [...heights, 0]; // 🔑 sentinel 0 → చివర్లో stack ఖాళీ చేయిస్తుంది

    for (let i = 0; i < h.length; i++) {
        while (stack.length && h[i] < h[stack[stack.length - 1]]) {
            const height = h[stack.pop()];
            // 🔑 width = ఎడమ boundary నుండి కుడి boundary వరకు
            const width = stack.length ? i - stack[stack.length - 1] - 1 : i;
            maxArea = Math.max(maxArea, height * width);
        }
        stack.push(i);
    }
    return maxArea;
}
// O(n) – ప్రతి bar ఒకసారి push, ఒకసారి pop
// అంతర్దృష్టి: ప్రతి bar కి – "ఈ bar ఎత్తుతో ఎంత వెడల్పు rectangle సాధ్యం?"
// = ఎడమ వైపు మొదటి చిన్నది నుండి కుడి వైపు మొదటి చిన్నది వరకు

```

ఉదాహరణ 3 — Circular array (Next Greater II)

```

function nextGreaterElementsCircular(nums) {
    const n = nums.length;
    const res = new Array(n).fill(-1);
    const stack = [];
    // 🔑 రెండు rounds – circular ని simulate చేయడానికి (array ని రెట్టింపు చేయకుండా)
    for (let i = 0; i < 2 * n; i++) {
        const cur = nums[i % n];
        while (stack.length && cur > nums[stack[stack.length - 1]]) res[stack.pop()] = cur;
        if (i < n) stack.push(i); // ⚠ మొదటి round లో మాత్రమే push
    }
    return res;
}

```

📋 Monotonic Deque — Sliding Window Maximum

```

function maxSlidingWindow(nums, k) {
    const deque = []; // indices, decreasing values
    const res = [];

    for (let i = 0; i < nums.length; i++) {
        // 1. Window బయటికి వెళ్ళిన index ని తొలగించడం
        if (deque.length && deque[0] <= i - k) deque.shift();

        // 2. చిన్నవాటిని వెనుక నుండి తొలగించడం (అవి ఇక max కాల్వవు)
    }
}

```

```

while (deque.length && nums[deque[deque.length - 1]] <= nums[i]) deque.pop();

deque.push(i);
if (i >= k - 1) res.push(nums[deque[0]]); // front = window max ✓
}
return res;
}
// O(n) - ప్రతి element ఒకసారి in, ఒకసారి out

```

ఈ pattern కి చెందిన problems

Medium: [Next Greater Element I/II](#) · [Daily Temperatures](#) ★ · [Online Stock Span](#)
[Remove K Digits](#) · [Remove Duplicate Letters](#) · [132 Pattern](#)
[Sum of Subarray Minimums](#) · [Asteroid Collision](#) · [Car Fleet](#)
Hard: [Largest Rectangle in Histogram](#) ★ · [Maximal Rectangle](#) (histogram × n)
[Trapping Rain Water](#) (stack version) · [Sliding Window Maximum](#) (deque)
[Shortest Subarray with Sum at Least K](#)

Gotchas

- **Values ఒడులు indices push చేయడం మర్చిపోవడం** → దూరం/width లెక్కించలేం.
- **> vs >=** — duplicates handling మారుతుంది (problem ప్రకారం).
- **Histogram లో sentinel లేకపోవడం** → చివర్లో stack లో మిగిలినవి process కావు.
- **Width calculation తప్పు** — $i - \text{stack.top} - 1$ (stack ఖాళీ అయితే i).
- **Stack ఖాళీ అని check చేయకపోవడం** → `undefined` comparison.
- **$O(n^2)$ అని అనుకోవడం** — nested while ఉన్నా amortized $O(n)$ (ప్రతి element 2 ops).

Interview దృష్టి

- *"Monotonic stack ఎందుకు $O(n)$?"* → ప్రతి element ఒకసారి push + ఒకసారి pop → amortized.
- *"Largest rectangle intuition?"* → ప్రతి bar కి "ఎడమ/కుడి వైపు మొదటి చిన్నది" boundaries.
- *"Sliding window maximum ఎలా $O(n)$?"* → monotonic deque; heap $O(n \log k)$ కంటే మేలు.

📄 Template 2 — Lower bound / First occurrence ★ (అత్యంత ఉపయోగకరం)

```
// 🔑 ఈ ఒక్క template తో దాదాపు అన్ని binary search variants చేయొచ్చు
function lowerBound(nums, target) {
    let left = 0, right = nums.length; // ⚠️ right = n (n-1 కాదు)

    while (left < right) { // ⚠️ < (<= కాదు)
        const mid = left + Math.floor((right - left) / 2);
        if (nums[mid] < target) left = mid + 1; // mid సరిపోదు → కుడికి
        else right = mid; // mid సరిపోవచ్చు → దాన్ని ఉంచు
    }
    return left; // >= target ఉన్న మొదటి index (లేకపోతే n)
}

// Upper bound = > target ఉన్న మొదటి index
function upperBound(nums, target) {
    let left = 0, right = nums.length;
    while (left < right) {
        const mid = left + Math.floor((right - left) / 2);
        if (nums[mid] <= target) left = mid + 1; // 🔑 <= (< కాదు)
        else right = mid;
    }
    return left;
}

// ఉపయోగాలు
const first = lowerBound(nums, t); // మొదటి occurrence
const last = upperBound(nums, t) - 1; // చివరి occurrence
const count = upperBound(nums, t) - lowerBound(nums, t); // ఎన్నిసార్లు ఉంది
const insertPos = lowerBound(nums, t); // insert position
```

నా సలహా: `lowerBound` template ఒక్కటే బట్టి పెట్టండి. Off-by-one errors యొక్క ప్రధాన మూలం రకరకాల templates గుర్తుపెట్టుకోవడం.

📄 Template 3 — Rotated sorted array

```
function searchRotated(nums, target) {
    let left = 0, right = nums.length - 1;

    while (left <= right) {
        const mid = left + Math.floor((right - left) / 2);
        if (nums[mid] === target) return mid;

        // 🔑 కేలక అంతర్దృష్టి: ఏదో ఒక సగం ఎప్పుడూ sorted గా ఉంటుంది
        if (nums[left] <= nums[mid]) { // ఎడమ సగం sorted
            if (nums[left] <= target && target < nums[mid]) right = mid - 1;
            else left = mid + 1;
        } else { // కుడి సగం sorted
            if (nums[mid] < target && target <= nums[right]) left = mid + 1;
            else right = mid - 1;
        }
    }
}
```

```

    return -1;
}

```

```

// Rotation point (minimum element)
function findMin(nums) {
    let left = 0, right = nums.length - 1;
    while (left < right) {
        const mid = left + Math.floor((right - left) / 2);
        if (nums[mid] > nums[right]) left = mid + 1; // minimum కుడి వైపు
        else right = mid; // mid కావచ్చు
    }
    return nums[left];
}

```

Template 4 — 2D matrix binary search

```

function searchMatrix(matrix, target) {
    const m = matrix.length, n = matrix[0].length;
    let left = 0, right = m * n - 1;

    while (left <= right) {
        const mid = left + Math.floor((right - left) / 2);
        const val = matrix[Math.floor(mid / n)][mid % n]; // 🔑 1D index → 2D
        if (val === target) return true;
        else if (val < target) left = mid + 1;
        else right = mid - 1;
    }
    return false;
}

// Sorted rows + columns (rows మరియు columns లోని elements sorted) → staircase search O(m+n)
function searchMatrixII(matrix, target) {
    let row = 0, col = matrix[0].length - 1; // 🔑 కుడి-పై మూల నుండి
    while (row < matrix.length && col >= 0) {
        if (matrix[row][col] === target) return true;
        else if (matrix[row][col] > target) col--; // ఈ column లో అన్ని పెద్దవి
        else row++; // ఈ row లో అన్ని చిన్నవి
    }
    return false;
}

```

ఈ pattern కి చెందిన problems

Easy: [Binary Search](#) · [Search Insert Position](#) · [First Bad Version](#) · [Sqrt\(x\)](#)
[Valid Perfect Square](#) · [Guess Number Higher or Lower](#)

Medium: [Search in Rotated Sorted Array \(I/II\)](#) · [Find Minimum in Rotated Sorted Array](#)
[Find First and Last Position](#) · [Search a 2D Matrix \(I/II\)](#) · [Find Peak Element](#)
[Single Element in Sorted Array](#) · [Time Based Key-Value Store](#)

Hard: [Median of Two Sorted Arrays](#) 🌟 · [Find K-th Smallest Pair Distance](#)

Gotchas (binary search = off-by-one errors యొక్క రాజ్యం)

- $(left + right) / 2$ overflow – JS లో పెద్ద సమస్య కాదు కానీ $left + (right - left) / 2$ అలవాటు మంచిది.
- `Math.floor` మర్చిపోవడం → JS లో fractional index → `nums[2.5] = undefined`.
- `left <= right` vs `left < right` గందరగోళం – template ప్రకారం స్థిరంగా ఉండాలి.
- `right = mid` vs `right = mid - 1` – infinite loop కి ప్రధాన కారణం.
- Infinite loop – `left = mid` రాయడం (`mid` ముందుకు వెళ్ళదు) → `mid + 1` ఉండాలి.
- Rotated array లో duplicates – worst case $O(n)$ అవుతుంది (`nums[left] === nums[mid]` అయితే `left++`).

Interview దృష్టి

- "Binary search ఎప్పుడు వాడతావు?" → sorted **లేదా monotonic predicate** ఉన్నప్పుడు – ఇది చెప్తే తర్వాతి pattern కి దారి.
- "First/last occurrence ఎలా?" → lowerBound/upperBound.
- "Rotated array లో ఎలా?" → "ఒక సగం ఎప్పుడూ sorted" invariant.

12. Binary Search on Answer ★★

🔍 Recognition signals (SDE2 interviews లో అత్యంత విలువైన pattern)

- ✓ "minimize the maximum" / "maximize the minimum"
- ✓ "minimum capacity/speed/days/size to do X"
- ✓ "smallest divisor such that..."
- ✓ Answer ఒక `**range**` లో ఉంది + "ఒక విలువ పని చేస్తే, దాని కంటే పెద్దవి కూడా పని చేస్తాయి"
- ✓ Brute force = ప్రతి సాధ్యమైన సమాధానాన్ని `try` చేయడం

వివరణ – DSA లో అత్యంత "aha!" moment

Array లో వెతకడం లేదు – సమాధానంలో వెతుకుతున్నాం.

సాధారణ binary search: `array` indices మీద `[0, 1, 2, ..., n-1]`
Binary search on answer: సాధ్యమైన సమాధానాల మీద `[minPossible, ..., maxPossible]`

షరతు – monotonicity: "ఒక సమాధానం x పని చేస్తే, $x+1$, $x+2$... అన్నీ పని చేయాలి."

Answer space: `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`
Works?: `[F, F, F, F, T, T, T, T, T, T]`

↑
ఇదే మనకి కావలసినది (మొదటి `T`) → binary search!

Real-life Scenario

Binary search on answer = సరైన size చెప్పులు కొనడం. Shop లో 6 నుండి 12 sizes ఉన్నాయి. ప్రతి size try చేయరు (linear). **9 try చేస్తారు** – వదులుగా ఉంది → ఇక 10, 11, 12 అనవసరం (అన్నీ వదులే). 7 try చేస్తారు – బిగుతుగా ఉంది → 6 అనవసరం. **ప్రతి try సగం అవకాశాలను తొలగిస్తుంది.**

కీలకం: "ఈ size సరిపోతుందా?" అనేది **monotonic** – పెద్ద size సరిపోతే, ఇంకా పెద్దవి కూడా సరిపోతాయి (వదులుగా అయినా).

Universal Template

```
function binarySearchOnAnswer(input) {  
  // 1 Answer range నిర్ణయించడం  
  let left = /* కనిష్ట సాధ్యమైన సమాధానం */;  
  let right = /* గరిష్ట సాధ్యమైన సమాధానం */;  
  
  // 2 Feasibility function - "ఈ candidate పని చేస్తుందా?"  
  const isFeasible = (candidate) => {  
    // సాధారణంగా O(n) greedy simulation  
    return true / false;  
  };  
  
  // 3 Binary search (lowerBound pattern)  
  while (left < right) {  
    const mid = left + Math.floor((right - left) / 2);  
    if (isFeasible(mid)) right = mid;      // ✓ పని చేసింది → ఇంకా చిన్నది try  
    else left = mid + 1;                  // ✗ చేయలేదు → పెద్దది కావాలి  
  }  
  return left;                          // మొదటి feasible answer  
}  
// Time: O(n × log(range))
```

ఉదాహరణ 1 — Koko Eating Bananas (classic) ★

```
// K గంటల్లో అన్ని అరటిపళ్ళు తినాలంటే కనీస వేగం (per hour) ఎంత?  
function minEatingSpeed(piles, h) {  
  let left = 1, right = Math.max(...piles);      // 1 range: 1 నుండి అతిపెద్ద pile  
  
  const canFinish = (speed) => {                // 2 feasibility  
    let hours = 0;  
    for (const pile of piles) hours += Math.ceil(pile / speed);  
    return hours <= h;  
  };  
  
  while (left < right) {                          // 3 binary search  
    const mid = left + Math.floor((right - left) / 2);  
    if (canFinish(mid)) right = mid;              // వేగం సరిపోతుంది → తగ్గించి చూడు  
    else left = mid + 1;                          // సరిపోలేదు → పెంచాలి  
  }  
  return left;  
}
```

```
// O(n log(max)) - brute force O(max × n) కంటే భారీగా మెరుగు  
// Monotonicity: వేగం ఎక్కువైతే సమయం తక్కువ → "పని చేస్తుందా" monotonic 
```

ఉదాహరణ 2 — Ship Packages Within D Days

```
function shipWithinDays(weights, days) {  
  let left = Math.max(...weights); //  కనీసం అతిపెద్ద package పట్టాలి  
  let right = weights.reduce((a, b) => a + b); //  ఒకే రోజులో అన్నీ  
  
  const canShip = (capacity) => {  
    let daysNeeded = 1, current = 0;  
    for (const w of weights) {  
      if (current + w > capacity) { daysNeeded++; current = 0; }  
      current += w;  
    }  
    return daysNeeded <= days;  
  };  
  
  while (left < right) {  
    const mid = left + Math.floor((right - left) / 2);  
    if (canShip(mid)) right = mid;  
    else left = mid + 1;  
  }  
  return left;  
}
```

ఉదాహరణ 3 — Split Array Largest Sum (Hard, కానీ అదే template)

```
// Array ని k subarrays గా విభజించి, largest subarray sum ని minimize చేయాలి  
function splitArray(nums, k) {  
  let left = Math.max(...nums), right = nums.reduce((a, b) => a + b);  
  
  const canSplit = (maxSum) => {  
    let count = 1, current = 0;  
    for (const n of nums) {  
      if (current + n > maxSum) { count++; current = n; }  
      else current += n;  
    }  
    return count <= k;  
  };  
  
  while (left < right) {  
    const mid = left + Math.floor((right - left) / 2);  
    if (canSplit(mid)) right = mid;  
    else left = mid + 1;  
  }  
  return left;  
}  
//  ఇది "minimize the maximum" యొక్క ప్రామాణిక రూపం - DP తో కూడా చేయొచ్చు కానీ ఇది సులభం + వేగం
```

ఉదాహరణ 4 – Floating point (precision తో)

```
function findMedianSortedArrays(nums1, nums2) { /* binary search on partition */ }

// Floating point binary search
function sqrt(x, precision = 1e-6) {
  let left = 0, right = Math.max(1, x);
  while (right - left > precision) { // ⚠ precision loop (indices కాదు)
    const mid = (left + right) / 2;
    if (mid * mid < x) left = mid;
    else right = mid;
  }
  return left;
}
```

🔑 గుర్తించడం ఎలా – 3 ప్రశ్నలు

- సమాధానం ఒక సంఖ్య, మరియు దాని range నాకు తెలుసా? → అవును
- "ఈ సమాధానం పని చేస్తుందా?" అని $O(n)$ లో **check** చేయగలనా? → అవును
- పని చేస్తే – పెద్దది/చిన్నది కూడా పని చేస్తాయా (monotonic)? → అవును

↓
✅ Binary Search on Answer

ఈ pattern కి చెందిన problems

Medium: Koko Eating Bananas ★ · Capacity to Ship Packages Within D Days
Minimum Number of Days to Make m Bouquets · Find the Smallest Divisor
Magnetic Force Between Two Balls · Minimize Max Distance to Gas Station
Search in Rotated Array · Find Peak Element · Kth Smallest in Sorted Matrix

Hard: Split Array Largest Sum · Median of Two Sorted Arrays ★
Minimize Max Difference of Pairs · Maximum Number of Removable Characters

Gotchas

- Monotonicity లేకపోవడం** → binary search invalid (ముందు నిరూపించాలి).
- Range boundaries తప్పు** – **left** / **right** initial values జాగ్రత్తగా (ఉదా. ship లో **max(weights)**).
- isFeasible లో bug** → మొత్తం తప్పు (ఇదే asli logic).
- left = mid రాయడం** → infinite loop.
- Floating point లో left < right** → ఎప్పటికీ terminate కాదు; precision loop వాడాలి.
- "Maximize the minimum" లో దిశ తారుమారు** – feasible అయితే **left = mid** వైపు వెళ్ళాలి.

Interview దృష్టి

- "ఇది binary search అని ఎలా గుర్తించావు?" → answer range + monotonic feasibility – ఈ reasoning చెప్పడమే అసలు score.
- "Monotonicity నిరూపించు" → "capacity పెంచితే days తగ్గుతాయి, ఎప్పుడూ పెరగవు" వంటి argument.

- "Complexity?" → $O(n \times \log(\text{range}))$ – range logarithmic కాబట్టి పెద్ద ranges కి కూడా వేగం.

13. Top-K & Heap Patterns

🔍 Recognition signals

- ✓ "K largest / K smallest / Kth largest"
- ✓ "top K frequent"
- ✓ "merge K sorted lists/arrays"
- ✓ "median of a data stream" (two heaps)
- ✓ "closest K points"
- ✓ **Streaming data** – అన్నీ store చేయలేం

వివరణ

Heap = ఎప్పుడూ min/max ని $O(1)$ లో ఇచ్చే data structure (insert/delete $O(\log n)$).

కీలక అంతర్దృష్టి – K largest కి min-heap వాడతాం (max-heap కాదు!):

Size K యొక్క min-heap ఉంచుకుంటే –
heap లో ఎప్పుడూ "ఇప్పటివరకు చూసిన అతిపెద్ద K elements" ఉంటాయి
heap **top** = ఆ K లో అతిచిన్నది = Kth largest ✓
కొత్త element **top** కంటే పెద్దదైతే → **top** తీసికొత్తది పెట్టు

Real-life Scenario

Top-K = ఒక పోటీలో top 3 అవార్డులు. వేలమంది పాల్గొంటున్నారు, కానీ మే దగ్గర 3 కుర్చీలే ఉన్నాయి (min-heap size K). కొత్త పోటీదారుడు వచ్చినప్పుడు – **ప్రస్తుతం కూర్చున్న 3 లో అత్యల్పమైనది పోల్చుతారు.** కొత్తవాడు మెరుగైతే – అత్యల్పమైనది తీసివేస్తారు. **వేలమందిని sort చేయాల్సిన అవసరం లేదు** ($O(n \log n) \rightarrow O(n \log k)$).

📄 JavaScript Heap implementation (JS లో built-in లేదు – interview కి తెలియాలి)

```
class MinHeap {
  constructor(compare = (a, b) => a - b) { this.heap = []; this.compare = compare; }
  get size() { return this.heap.length; }
  peek() { return this.heap[0]; }

  push(val) {
    this.heap.push(val);
    this.#bubbleUp(this.heap.length - 1);
  }
  pop() {
    if (this.size === 0) return undefined;
    const top = this.heap[0];
    const last = this.heap.pop();
    if (this.size > 0) { this.heap[0] = last; this.#bubbleDown(0); }
  }
  #bubbleUp(i) {
    while (i > 0 && this.compare(this.heap[i], this.heap[Math.floor(i/2)]) < 0) {
      [this.heap[i], this.heap[Math.floor(i/2)]] = [this.heap[Math.floor(i/2)], this.heap[i]];
      i = Math.floor(i/2);
    }
  }
  #bubbleDown(i) {
    let j = 2*i + 1;
    while (j < this.size) {
      if (j + 1 < this.size && this.compare(this.heap[j], this.heap[j+1]) > 0) j = j+1;
      if (this.compare(this.heap[i], this.heap[j]) > 0) [this.heap[i], this.heap[j]] = [this.heap[j], this.heap[i]];
      i = j;
      j = 2*i + 1;
    }
  }
}
```

```

    return top;
}
#bubbleUp(i) {
    while (i > 0) {
        const parent = (i - 1) >> 1;
        if (this.compare(this.heap[i], this.heap[parent]) >= 0) break;
        [this.heap[i], this.heap[parent]] = [this.heap[parent], this.heap[i]];
        i = parent;
    }
}
#bubbleDown(i) {
    const n = this.size;
    while (true) {
        let smallest = i;
        const l = 2 * i + 1, r = 2 * i + 2;
        if (l < n && this.compare(this.heap[l], this.heap[smallest]) < 0) smallest = l;
        if (r < n && this.compare(this.heap[r], this.heap[smallest]) < 0) smallest = r;
        if (smallest === i) break;
        [this.heap[i], this.heap[smallest]] = [this.heap[smallest], this.heap[i]];
        i = smallest;
    }
}
}
// Max-heap: new MinHeap((a, b) => b - a)
// Objects: new MinHeap((a, b) => a.dist - b.dist)

```

Interview tip: heap implementation రాయమని అడగకపోతే – "I'll assume a MinHeap class with push/pop/peek" అని చెప్పి ముందుకు వెళ్ళండి. సమయం ఆదా.

Template 1 — Kth largest (min-heap of size K)

```

function findKthLargest(nums, k) {
    const heap = new MinHeap();
    for (const num of nums) {
        heap.push(num);
        if (heap.size > k) heap.pop(); // 🔑 size K నిలుపుకోవడం
    }
    return heap.peak(); // K largest లో అతిచిన్నది = Kth largest ✅
}
// O(n log k) time, O(k) space – sorting O(n log n) కంటే మేలు (k << n అయితే)

```

Template 2 — Top K frequent (heap + hash map)

```

function topKFrequent(nums, k) {
    const freq = new Map();
    for (const n of nums) freq.set(n, (freq.get(n) ?? 0) + 1);

    const heap = new MinHeap((a, b) => a[1] - b[1]); // [value, count] – count ప్రకారం
    for (const entry of freq) {
        heap.push(entry);
        if (heap.size > k) heap.pop();
    }
}

```

```

    return heap.heap.map(([val]) => val);
}
// O(n log k)

// ⚡ Bucket sort ప్రత్యామ్నాయం - O(n)!
function topKFrequentBucket(nums, k) {
    const freq = new Map();
    for (const n of nums) freq.set(n, (freq.get(n) ?? 0) + 1);

    const buckets = Array.from({ length: nums.length + 1 }, () => []);
    for (const [num, count] of freq) buckets[count].push(num); // 🔑 count = index

    const res = [];
    for (let i = buckets.length - 1; i >= 0 && res.length < k; i--) res.push(...buckets[i]);
    return res.slice(0, k);
}
// 🔑 "frequency కి upper bound n ఉంది" అనే fact ని exploit చేయడం

```

📁 Template 3 — Merge K sorted lists ★

```

function mergeKLists(lists) {
    const heap = new MinHeap((a, b) => a.val - b.val);
    for (const list of lists) if (list) heap.push(list); // ప్రతి list యొక్క head

    const dummy = { next: null };
    let tail = dummy;

    while (heap.size) {
        const node = heap.pop(); // అన్ని heads లో అతిచిన్నది
        tail.next = node;
        tail = node;
        if (node.next) heap.push(node.next); // 🔑 అదే list నుండి తర్వాతిది
    }
    return dummy.next;
}
// O(N log k) - N = మొత్తం nodes, k = lists సంఖ్య
// ప్రత్యామ్నాయం: divide & conquer merge - అదే complexity, heap అవసరం లేదు

```

📁 Template 4 — Two Heaps (median of stream) ★

```

class MedianFinder {
    constructor() {
        this.small = new MinHeap((a, b) => b - a); // max-heap (చిన్న సగం)
        this.large = new MinHeap((a, b) => a - b); // min-heap (పెద్ద సగం)
    }

    addNum(num) {
        // 1. ఎప్పుడూ small లోకి push, తర్వాత అతిపెద్దదాన్ని large కి తరలించడం
        this.small.push(num);
        this.large.push(this.small.pop());

        // 2. Size balance (small ≥ large, గరిష్ట తేడా 1)
        if (this.large.size > this.small.size) this.small.push(this.large.pop());
    }
}

```

```

}

findMedian() {
  return this.small.size > this.large.size
    ? this.small.peak()
    : (this.small.peak() + this.large.peak()) / 2;
}
}

// addNum O(log n), findMedian O(1)
// 🔑 అంతర్దృష్టి: median = "రెండు సగల మధ్య సరిహద్దు" → రెండు heaps తో ఆ సరిహద్దును O(1) లో పట్టుకోవడం

```

📁 Template 5 — K closest points

```

function kClosest(points, k) {
  const heap = new MinHeap((a, b) => b[0] - a[0]); // max-heap by distance
  for (const [x, y] of points) {
    const dist = x * x + y * y; // 🔑 sqrt అవసరం లేదు (comparison కి)
    heap.push([dist, [x, y]]); // అత్యధిక దూరం ఉన్నది తీసేయ్
    if (heap.size > k) heap.pop();
  }
  return heap.heap.map(([_, p]) => p);
}

```

Heap vs QuickSelect vs Sorting — ఎప్పుడు ఏది

	Sorting	Heap (size K)	QuickSelect
Time	$O(n \log n)$	$O(n \log k)$	$O(n)$ average, $O(n^2)$ worst
Space	$O(1)$ – $O(n)$	$O(k)$	$O(1)$
Streaming	✗	✓	✗
అన్ని K elements	✓ sorted	✓ unsorted	✓ unsorted
ఎప్పుడు	$K \approx n$	streaming / $k \ll n$ ✓	ఒకసారి Kth కావాలంటే

```

// QuickSelect – Kth largest O(n) average
function quickSelect(nums, k) {
  const target = nums.length - k; // Kth largest = (n-k)th smallest
  let left = 0, right = nums.length - 1;

  while (true) {
    const p = partition(nums, left, right);
    if (p === target) return nums[p];
    else if (p < target) left = p + 1;
    else right = p - 1;
  }
}

function partition(nums, left, right) {
  const pivot = nums[right];
  let i = left;
  for (let j = left; j < right; j++) if (nums[j] <= pivot) [nums[i], nums[j]] = [nums[j],

```

```

nums[i++]];
[nums[i], nums[right]] = [nums[right], nums[i]];
return i;
}

```

ఈ pattern కి చెందిన problems

Easy: Kth Largest Element in a Stream · Last Stone Weight
Medium: Kth Largest Element in an Array ★ · Top K Frequent Elements/Words
 K Closest Points to Origin · Task Scheduler · Reorganize String
 Sort Characters By Frequency · Find K Pairs with Smallest Sums
 Kth Smallest Element in a Sorted Matrix · Ugly Number II
Hard: Merge K Sorted Lists ★ · Find Median from Data Stream ★
 Sliding Window Median · IPO · Smallest Range Covering K Lists

Gotchas

- K largest కి max-heap వాడటం → $O(n \log n)$ (min-heap size $K = O(n \log k)$ ✓).
- JS లో built-in heap లేదు – interview లో ముందే చెప్పండి (implement చేయాలా అని అడగండి).
- **Array.sort()** ని heap అనుకోవడం – sort $O(n \log n)$, heap operations $O(\log n)$.
- Two heaps లో balance మర్చిపోవడం → median తప్పు.
- Distance లో **Math.sqrt** – అనవసరం (comparison కి), నెమ్మది + precision issues.
- Heap ని sorted అనుకోవడం – heap array sorted కాదు! (top మాత్రమే guaranteed).

Interview దృష్టి

- "Kth largest – ఎన్ని approaches?" → sort $O(n \log n)$, heap $O(n \log k)$, quickselect $O(n)$ avg – trade-offs తే.
- "Median of stream ఎలా?" → two heaps + balance invariant.
- "Streaming data లో top-K?" → heap (అన్నీ store చేయలేం కాబట్టి).

14. Sorting-based Patterns

🔍 Recognition signals

- ✓ "sort" అనే పదం లేకపోయినా – **order** ముఖ్యమైతే
- ✓ Custom comparison logic ("largest number", "sort by frequency")
- ✓ Greedy problems (దాదాపు అన్నిటికీ sort మొదటి అడుగు)
- ✓ Intervals, meetings, scheduling
- ✓ "K closest / K largest" (heap ప్రత్యామ్నాయం)

JavaScript sort – ముఖ్యమైన gotchas

```

// ✗ అతిపెద్ద JS DSA bug
[10, 9, 1, 2].sort();

// [1, 10, 2, 9] – string comparison! ✨

```

```
// ✅ ఎప్పుడూ comparator ఇవ్వాలి
[10, 9, 1, 2].sort((a, b) => a - b); // [1, 2, 9, 10] ✅ ascending
[10, 9, 1, 2].sort((a, b) => b - a); // descending

// Strings
words.sort(); // lexicographic (default OK)
words.sort((a, b) => a.localeCompare(b)); // locale-aware

// Objects – multi-level
people.sort((a, b) => a.age - b.age || a.name.localeCompare(b.name)); // 🔑 || chaining

// ⚠️ sort() array ని mutate చేస్తుంది
const sorted = [...arr].sort((a, b) => a - b); // copy ✅
const sorted2 = arr.toSorted((a, b) => a - b); // ES2023 ✅ non-mutating
```

JS sort() **stable** (ES2019+) – equal elements యొక్క సాపేక్ష క్రమం నిలుస్తుంది. **V8 లో TimSort** – $O(n \log n)$.

📁 Pattern A – Custom comparator (Largest Number) ★

```
function largestNumber(nums) {
  const strs = nums.map(String);
  // 🔑 తెలివైన comparator: "ఏ క్రమంలో పెడితే పెద్ద సంఖ్య వస్తుంది?"
  strs.sort((a, b) => (b + a).localeCompare(a + b)); // "9"+"30" vs "30"+"9"
  return strs[0] === "0" ? "0" : strs.join(""); // ⚠️ అన్నీ 0 అయితే
}
// [3, 30, 34, 5, 9] → "9534330"
```

📁 Pattern B – Sort + greedy (interval scheduling)

```
// అత్యధిక non-overlapping meetings – end ప్రకారం sort (Pattern 7)
// Minimum arrows – end ప్రకారం sort
// Task assignment – రెండు arrays sort + two pointers
function findContentChildren(greedy, cookies) {
  greedy.sort((a, b) => a - b);
  cookies.sort((a, b) => a - b);
  let child = 0, cookie = 0;
  while (child < greedy.length && cookie < cookies.length) {
    if (cookies[cookie] >= greedy[child]) child++; // ఈ cookie ఈ పిల్లవాడికి సరిపోతుంది
    cookie++;
  }
  return child;
}
```

📁 Pattern C – Counting sort (చిన్న value range)

```
function sortColors(nums) { // values 0,1,2 మాత్రమే
  const count = [0, 0, 0];
  for (const n of nums) count[n]++;
  let i = 0;
```

```

    for (let v = 0; v < 3; v++) while (count[v]-- > 0) nums[i++] = v;
}
// O(n) - comparison sort యొక్క O(n log n) lower bound ని దాటడం
// (ఎందుకంటే ఇది comparison sort కాదు)

```

Pattern D — Bucket sort (frequency-based)

```

// Top K frequent - Template 2 లో చూశాం
// Maximum Gap (Hard) - bucket sort తో O(n)

```

Pattern E — Merge sort adaptation (count inversions)

```

// "ప్రతి element కి - దాని కుడి వైపు ఎన్ని చిన్నవి ఉన్నాయి?" (Count of Smaller Numbers After Self)
function countSmaller(nums) {
    const n = nums.length;
    const counts = new Array(n).fill(0);
    const indexed = nums.map((val, idx) => ({ val, idx }));

    function mergeSort(arr) {
        if (arr.length <= 1) return arr;
        const mid = arr.length >> 1;
        const left = mergeSort(arr.slice(0, mid));
        const right = mergeSort(arr.slice(mid));

        const merged = [];
        let i = 0, j = 0;
        while (i < left.length || j < right.length) {
            if (j >= right.length || (i < left.length && left[i].val <= right[j].val)) {
                counts[left[i].idx] += j; // 🔑 j = ఇప్పటివరకు దాటిన కుడి elements (అన్నీ చిన్నవి)
                merged.push(left[i++]);
            } else {
                merged.push(right[j++]);
            }
        }
        return merged;
    }
    mergeSort(indexed);
    return counts;
}
// O(n log n) - merge sort ని counting కి adapt చేయడం (divide & conquer)

```

ఈ pattern కి చెందిన problems

Easy: [Sort Array By Parity](#) · [Merge Sorted Array](#) · [Assign Cookies](#) · [Height Checker](#)
Medium: [Sort Colors](#) · [Largest Number](#) ★ · [Top K Frequent](#) · [Kth Largest Meeting Rooms II](#) · [Non-overlapping Intervals](#) · [Custom Sort String](#)
[Wiggle Sort](#) · [Reorganize String](#) · [Car Fleet](#)
Hard: [Count of Smaller Numbers After Self](#) · [Maximum Gap](#) · [Reverse Pairs](#)

Gotchas

- **sort()** comparator లేకుండా numbers → lexicographic ✨ (JS లో #1 DSA bug).
- **sort()** mutation – original array మారుతుంది (toSorted /spread వాడాలి).
- **Sorting తర్వాత indices పోతాయి** → index అవసరమైతే {val, idx} pairs లేదా hash map.
- **Comparator లో a > b ? 1 : -1** – equal case (0) handle కాలేదు → stability పోతుంది.
- **O(n log n) ని అలవాటుగా అంగీకరించడం** – counting/bucket sort తో O(n) సాధ్యమేమో ఆలోచించాలి.

Interview దృష్టి

- "JS sort default ఎందుకు తప్పు?" → string conversion + lexicographic – తెలియకపోతే basics లేవని అర్థం.
 - "O(n log n) కంటే వేగంగా sort చేయగలమా?" → comparison sorts కి కాదు; counting/radix/bucket (value range పరిమితం అయితే) ✓.
-

Part 4 — Linked List Patterns

15. In-place Reversal

🔍 Recognition signals

- ✓ "reverse the linked list" (మొత్తం లేదా భాగం)
- ✓ "reverse in groups of k"
- ✓ "palindrome linked list"
- ✓ "reorder list"
- ✓ $O(1)$ space అవసరం (recursion $O(n)$ stack)

వివరణ

Three pointers: `prev` , `curr` , `next` . ప్రతి step లో ఒక link ని తిప్పడం.

Real-life Scenario

Linked list reversal = రైలు bogies తిప్పడం. ప్రతి bogie "నా తర్వాత ఎవరు" అని చూపిస్తుంది. Reverse చేయాలంటే – ప్రతి bogie ని "నా ముందు ఎవరో వాళ్ళ వైపు" చూపించమని మార్చాలి. కానీ మార్చేముందు – తర్వాతి bogie ని పట్టుకోవాలి (లేకపోతే మిగతా రైలు పోతుంది!). అదే `next` pointer యొక్క పని.

📄 Template — Iterative reversal

```
function reverseList(head) {
  let prev = null, curr = head;

  while (curr) {
    const next = curr.next;    // 1 తర్వాతిది భద్రపరచు (లేకపోతే link పోతుంది!)
    curr.next = prev;          // 2 link తిప్పు
    prev = curr;                // 3 prev ముందుకు
    curr = next;                // 4 curr ముందుకు
  }
  return prev;                  // 🔑 prev = కొత్త head (curr ఇప్పుడు null)
}
// O(n) time, O(1) space ✓
```

```
// Recursive version (O(n) stack space)
function reverseListRecursive(head) {
  if (!head || !head.next) return head;
  const newHead = reverseListRecursive(head.next);
  head.next.next = head;    // 🔑 తర్వాతివాడిని నా వైపు చూపించు
  head.next = null;
  return newHead;
}
```

Variant 1 — Reverse a sub-list (positions m..n)

```
function reverseBetween(head, left, right) {
  if (!head || left === right) return head;
  const dummy = { next: head }; // 🗝️ head reverse అయ్యే అవకాశం
  let prev = dummy;

  for (let i = 1; i < left; i++) prev = prev.next; // left కి ముందు node

  const start = prev.next; // reverse అయ్యే భాగం మొదలు
  let curr = start.next;

  // 🗝️ "Head insertion" - ప్రతి node ని prev తర్వాత ముందుకి తీసుకురావడం
  for (let i = 0; i < right - left; i++) {
    start.next = curr.next;
    curr.next = prev.next;
    prev.next = curr;
    curr = start.next;
  }
  return dummy.next;
}
```

Variant 2 — Reverse in K-groups (Hard) ★

```
function reverseKGroup(head, k) {
  // 1. k nodes ఉన్నాయా అని check
  let node = head;
  for (let i = 0; i < k; i++) {
    if (!node) return head; // k కంటే తక్కువ → అలాగే వదిలేయ్
    node = node.next;
  }

  // 2. మొదటి k nodes ని reverse
  let prev = null, curr = head;
  for (let i = 0; i < k; i++) {
    const next = curr.next;
    curr.next = prev;
    prev = curr;
    curr = next;
  }

  // 3. మిగతాదాన్ని recursion తో reverse చేసి, ఇప్పటి tail (head) కి attach
  head.next = reverseKGroup(curr, k);
  return prev; // ఈ group యొక్క కొత్త head
}
// O(n) time, O(n/k) recursion stack
```

Variant 3 — Palindrome linked list (reverse సగం)

```
function isPalindrome(head) {
  // 1. Middle కనుగొనడం (fast-slow)
  let slow = head, fast = head;
  while (fast && fast.next) { slow = slow.next; fast = fast.next.next; }
```

```
// 2. రెండో సగం reverse
let prev = null;
while (slow) { const next = slow.next; slow.next = prev; prev = slow; slow = next; }

// 3. రెండు సగాలను compare
let left = head, right = prev;
while (right) { // ⚠ right (చిన్న సగం) తో loop
  if (left.val !== right.val) return false;
  left = left.next; right = right.next;
}
return true;
}
// O(n) time, O(1) space ✅ (array కి copy చేస్తే O(n) space)
```

Variant 4 — Reorder list ($L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow \dots$)

```
function reorderList(head) {
  if (!head || !head.next) return;
  // 1. Middle
  let slow = head, fast = head;
  while (fast.next && fast.next.next) { slow = slow.next; fast = fast.next.next; }

  // 2. రెండో సగం reverse
  let second = slow.next;
  slow.next = null;
  let prev = null;
  while (second) { const next = second.next; second.next = prev; prev = second; second = next; }

  // 3. Merge alternately
  let first = head; second = prev;
  while (second) {
    const t1 = first.next, t2 = second.next;
    first.next = second; second.next = t1;
    first = t1; second = t2;
  }
}
// 🔑 ఇది మూడు patterns కలయిక: fast-slow + reversal + merge
```

ఈ pattern కి చెందిన problems

Easy: [Reverse Linked List](#) · [Palindrome Linked List](#) · [Middle of the Linked List](#)
 Medium: [Reverse Linked List II](#) · [Reorder List](#) · [Swap Nodes in Pairs](#)
[Rotate List](#) · [Odd Even Linked List](#) · [Partition List](#)
 Hard: [Reverse Nodes in k-Group](#) ★

Gotchas

- next** ని భద్రపరచకపోవడం → మిగతా list పోతుంది (అత్యంత సాధారణ తప్పు).
- head** ని return చేయడం — reversal తర్వాత prev కొత్త head.

- **Dummy node లేకపోవడం** → head modify అయ్యే cases లో edge case bugs.
- **Palindrome లో list ని restore చేయకపోవడం** – కొన్ని interviews లో అడుగుతారు.
- **K-group లో "k కంటే తక్కువ nodes" check మర్చిపోవడం.**
- **Cycle create చేయడం** – head.next = null మర్చిపోతే infinite loop.

Interview దృష్టి

- "Iterative vs recursive reversal?" → $O(1)$ vs $O(n)$ space; recursion elegant కానీ deep lists లో stack overflow.
- "Palindrome ని $O(1)$ space లో?" → middle + reverse half + compare.
- "Dummy node ఎందుకు?" → head modify/delete అయ్యే cases ని ఏకరూపంగా handle చేయడానికి.

16-17. Linked List — Fast-Slow, Dummy Node & Pointer Patterns

Fast-slow pointers Pattern 2 లో వివరంగా చూశాం. ఇక్కడ మిగతా linked list techniques.

Dummy Node Pattern (linked list యొక్క అత్యంత ఉపయోగకరమైన ట్రిక్)

```
// ❌ Dummy లేకుండా – head edge case ప్రతిసారి విడిగా handle చేయాలి
function removeElements(head, val) {
  while (head && head.val === val) head = head.next; // head handling విడిగా
  let curr = head;
  while (curr && curr.next) {
    if (curr.next.val === val) curr.next = curr.next.next;
    else curr = curr.next;
  }
  return head;
}

// ✅ Dummy తో – ఏకరూప logic
function removeElementsClean(head, val) {
  const dummy = { next: head }; // 🔑 head కి ముందు ఒక fake node
  let curr = dummy;
  while (curr.next) {
    if (curr.next.val === val) curr.next = curr.next.next;
    else curr = curr.next;
  }
  return dummy.next; // 🔑 dummy.next = నిజమైన head
}
```

నియమం: "Head మారే అవకాశం ఉన్న ప్రతి linked list problem లో dummy node వాడండి." ఇది edge cases సగం తగ్గిస్తుంది.

Merge two sorted lists

```
function mergeTwoLists(l1, l2) {
  const dummy = { next: null };
  let tail = dummy;

  while (l1 && l2) {
    if (l1.val <= l2.val) { tail.next = l1; l1 = l1.next; }
    else { tail.next = l2; l2 = l2.next; }
    tail = tail.next;
  }
  tail.next = l1 ?? l2; // 🗝️ మిగిలినది attach
  return dummy.next;
}
```

Add two numbers (carry handling)

```
function addTwoNumbers(l1, l2) {
  const dummy = { next: null };
  let tail = dummy, carry = 0;

  while (l1 || l2 || carry) { // 🗝️ carry కూడా condition లో!
    const sum = (l1?.val ?? 0) + (l2?.val ?? 0) + carry;
    carry = Math.floor(sum / 10);
    tail.next = { val: sum % 10, next: null };
    tail = tail.next;
    l1 = l1?.next; l2 = l2?.next;
  }
  return dummy.next;
}
```

Copy list with random pointer (interleaving ట్రిక్)

```
function copyRandomList(head) {
  if (!head) return null;

  // 1️⃣ ప్రతి node తర్వాత దాని copy ని interleave
  let curr = head;
  while (curr) {
    const copy = { val: curr.val, next: curr.next, random: null };
    curr.next = copy;
    curr = copy.next;
  }

  // 2️⃣ Random pointers set (copy.random = original.random.next)
  curr = head;
  while (curr) {
    if (curr.random) curr.next.random = curr.random.next;
    curr = curr.next.next;
  }

  // 3️⃣ రెండు lists ని విడదీయడం
  const newHead = head.next;
```

```
curr = head;
while (curr) {
  const copy = curr.next;
  curr.next = copy.next;
  copy.next = copy.next?.next ?? null;
  curr = curr.next;
}
return newHead;
}
// O(n) time, O(1) space ✅ (hash map approach O(n) space)
```

Key Points

- **Dummy node** = head-modifying problems కి universal ట్రిక్.
- Fast-slow = middle, cycle, nth-from-end.
- Merge/add patterns = while (l1 || l2 || carry) style.
- Interleaving = O(1) space deep copy ట్రిక్.

Interview దృష్టి

- "Linked list problems లో మీ మొదటి అడుగు?" → dummy node + edge cases (empty, single node, head change) – ఇది చెప్తే systematic thinking కనిపిస్తుంది.
-

Part 5 — Tree Patterns

18. Tree DFS — Top-Down vs Bottom-Up ★

🔍 Recognition signals

- ✓ Tree + "path" / "depth" / "sum" / "property check"
- ✓ ప్రతి **node** కి దాని **subtrees** గురించి సమాచారం కావాలి
- ✓ "root to leaf"
- ✓ Tree structure ని recursively process చేయాలి

వివరణ — ఇది అర్థమైతే 80% tree problems అయిపోయాయి

ప్రతి tree recursion problem ఈ రెండింటిలో ఒకటి:

- ▼ TOP-DOWN (preorder): తల్లి నుండి పిల్లలకి సమాచారం పంపడం
"నేను ఇప్పటివరకు తెలుసుకున్నది నీకు ఇస్తున్నాను"
→ parameters ద్వారా కిందికి పంపడం
ఉదా: root-to-leaf path sum, depth tracking, path collection
- ▲ BOTTOM-UP (postorder): పిల్లల నుండి తల్లికి సమాధానం రావడం
"నా పిల్లలు ఏం చెప్పారో దాని ఆధారంగా నేను నిర్ణయిస్తాను"
→ return value ద్వారా పైకి రావడం
ఉదా: height, balanced check, diameter, subtree sums

Real-life Scenario

Top-down = రాజు నుండి ఆజ్ఞ. రాజు మంత్రికి, మంత్రి సైన్యాధిపతికి, అతను సైనికులకి — సమాచారం పైనుండి కిందికి ప్రవహిస్తుంది. ప్రతి స్థాయిలో దానికి తన సమాచారం జోడించి పంపుతారు (`pathSum + node.val`).

Bottom-up = గ్రామాల నుండి జనాభా లెక్కలు. ప్రతి గ్రామం తన లెక్క మండలానికి, మండలాలు జిల్లాకి, జిల్లాలు రాష్ట్రానికి — సమాచారం కింద నుండి పైకి వస్తుంది. ప్రతి స్థాయిలో పిల్లల సమాధానాలను కలిపి తన సమాధానం తయారు చేస్తారు (`left + right + 1`).

📄 Template — Top-Down (preorder)

```
function topDown(node, /* state from parent */ pathSum, results) {  
  if (!node) return; // 1 base case  
  
  pathSum += node.val; // 2 ప్రస్తుత node ని state కి కలపడం  
  
  if (!node.left && !node.right) { // 3 leaf → సమాధానం record  
    results.push(pathSum);  
  }  
  
  topDown(node.left, pathSum, results); // 4 state ని పిల్లలకి పంపడం
```

```
topDown(node.right, pathSum, results);
}
```

ఉదాహరణ — Path Sum:

```
function hasPathSum(root, targetSum) {
  if (!root) return false;
  if (!root.left && !root.right) return targetSum === root.val; // leaf
  const remaining = targetSum - root.val; // కిందికి పంపడం
  return hasPathSum(root.left, remaining) || hasPathSum(root.right, remaining);
}
```

📁 Template — Bottom-Up (postorder) ★ అత్యంత ముఖ్యం

```
function bottomUp(node) {
  if (!node) return /* base value: 0, null, true, Infinity... */;

  const left = bottomUp(node.left); // 1 ఎడమ subtree సమాధానం
  const right = bottomUp(node.right); // 2 కుడి subtree సమాధానం

  return /* left, right, node.val కలిపి నా సమాధానం */; // 3 ఫైన్లీ పంపడం
}
```

ఉదాహరణ — Maximum Depth:

```
function maxDepth(root) {
  if (!root) return 0;
  return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

🔑 Bottom-up యొక్క శక్తి — global variable + return value (అత్యంత ముఖ్యమైన pattern)

కొన్ని problems లో — return చేసేది ఒకటి, track చేసేది ఇంకొకటి.

```
// Diameter — ఏ రెండు nodes మధ్య అయినా longest path
function diameterOfBinaryTree(root) {
  let diameter = 0; // 🔑 global answer

  function depth(node) {
    if (!node) return 0;
    const left = depth(node.left);
    const right = depth(node.right);

    diameter = Math.max(diameter, left + right); // 🔑 ఇక్కడ answer update
    // ↑ ఈ node గుండా వెళ్ళే path

    return 1 + Math.max(left, right); // 🔑 కానీ return చేసేది depth
    // ↑ parent కి ఉపయోగపడేది (path parent గుండా వెళ్ళాలంటే ఒక వైపు)
  }

  depth(root);
}
```

```

    return diameter;
}

```

ఈ "return one thing, track another" pattern – diameter, max path sum, longest univalue path, balanced check అన్నిటిలో ఉంది. దీన్ని గుర్తిస్తే tree problems చాలా సులభం అవుతాయి.

```

// Maximum Path Sum (Hard, కానీ అదే pattern)
function maxPathSum(root) {
    let maxSum = -Infinity;

    function gain(node) {
        if (!node) return 0;
        const left = Math.max(gain(node.left), 0); // 🗝️ negative అయితే ఆ subtree వద్దు
        const right = Math.max(gain(node.right), 0);

        maxSum = Math.max(maxSum, node.val + left + right); // ఈ node "peak" గా ఉన్న path

        return node.val + Math.max(left, right); // parent కి - ఒక వైపు మాత్రమే
    }
    gain(root);
    return maxSum;
}

```

```

// Balanced Binary Tree - O(n) (naive O(n^2) కంటే మేలు)
function isBalanced(root) {
    function check(node) {
        if (!node) return 0;
        const left = check(node.left);
        if (left === -1) return -1; // 🗝️ early exit signal
        const right = check(node.right);
        if (right === -1) return -1;
        if (Math.abs(left - right) > 1) return -1;
        return 1 + Math.max(left, right);
    }
    return check(root) !== -1;
}

```

DFS traversals — మూడు రూపాలు

```

// Preorder: root → left → right (tree copy, serialization, top-down)
function preorder(node, res = []) {
    if (!node) return res;
    res.push(node.val);
    preorder(node.left, res); preorder(node.right, res);
    return res;
}

```

```

// Inorder: left → root → right (BST → sorted order 🌟)
function inorder(node, res = []) {
    if (!node) return res;
    inorder(node.left, res); res.push(node.val); inorder(node.right, res);
}

```

```

    return res;
}

// Postorder: left → right → root (deletion, bottom-up computation)
function postorder(node, res = []) {
    if (!node) return res;
    postorder(node.left, res); postorder(node.right, res); res.push(node.val);
    return res;
}

```

```

// Iterative inorder (stack) – recursion అనుమతించకపోతే
function inorderIterative(root) {
    const res = [], stack = [];
    let curr = root;
    while (curr || stack.length) {
        while (curr) { stack.push(curr); curr = curr.left; } // ఎడమ చివరికి
        curr = stack.pop();
        res.push(curr.val);
        curr = curr.right;
    }
    return res;
}

```

ఈ pattern కి చెందిన problems

Easy: Maximum Depth · Same Tree · Symmetric Tree · Invert Binary Tree
 Path Sum · Balanced Binary Tree · Diameter · Merge Two Binary Trees
 Minimum Depth · Sum of Left Leaves
Medium: Path Sum II/III · Sum Root to Leaf Numbers · Flatten Binary Tree to Linked List
 Count Good Nodes · House Robber III · Longest Univalued Path
 All Nodes Distance K · Delete Nodes And Return Forest
Hard: Binary Tree Maximum Path Sum ★ · Serialize and Deserialize Binary Tree
 Binary Tree Cameras · Vertical Order Traversal

Gotchas

- **Base case మర్చిపోవడం** (`if (!node)`) → null pointer error.
- **Top-down/bottom-up గందరగోళం** – "నాకు పిల్లల సమాధానం కావాలా, లేక తల్లి సమాచారం కావాలా?"
- **Diameter లో 1 + కలపడం** return లో మర్చిపోవడం.
- **Global variable ని reset చేయకపోవడం** (multiple test cases).
- **Negative values** – max path sum లో `Math.max(gain, 0)` మర్చిపోవడం.
- **Leaf definition** – `!node.left && !node.right` (single-child node leaf కాదు!) – Minimum Depth లో classic bug.
- **Deep recursion** → stack overflow ($n = 10^5$ skewed tree) → iterative వాడాలి.

Interview దృష్టి

- "Top-down vs bottom-up ఎప్పుడు?" → parent state కావాలా (top-down), children answers కావాలా (bottom-up) – ఈ framework చెప్పడమే బలమైన signal.
- "Diameter ఎందుకు tricky?" → return చేసేది depth, track చేసేది diameter.
- "Recursion లేకుండా inorder?" → explicit stack.

19. Tree BFS — Level Order Family

🔍 Recognition signals

- ✓ "level by level" / "level order"
- ✓ "left view / right view / top view"
- ✓ "zigzag traversal"
- ✓ "minimum depth" (BFS వేగం – మొదటి leaf దగ్గరే ఆగుతుంది)
- ✓ "connect next right pointers"
- ✓ Level-wise aggregate (average, max per level)

📄 Universal Template — Level order

```
function levelOrder(root) {  
  if (!root) return [];  
  const res = [];  
  const queue = [root];  
  
  while (queue.length) {  
    const levelSize = queue.length;           // 🔑 ఈ level లో ఎన్ని nodes (snapshot!)  
    const level = [];  
  
    for (let i = 0; i < levelSize; i++) {     // 🔑 ఈ level ని మాత్రమే process  
      const node = queue.shift();  
      level.push(node.val);  
      if (node.left) queue.push(node.left);  
      if (node.right) queue.push(node.right);  
    }  
    res.push(level);  
  }  
  return res;  
}  
// O(n) time, O(w) space (w = గరిష్ట width)  
// ⚠️ JS లో shift() O(n) – పెద్ద trees కి index pointer వాడాలి
```

```
// ⚡ Optimized queue (shift O(n) నివారణ)  
function levelOrderFast(root) {  
  if (!root) return [];  
  const res = [];  
  let queue = [root];  
  while (queue.length) {
```

```

    res.push(queue.map((n) => n.val));
    const next = [];
    for (const node of queue) {
        if (node.left) next.push(node.left);
        if (node.right) next.push(node.right);
    }
    queue = next;
    // 🗝 level swap – shift అవసరం లేదు
}
return res;
}

```

Real-life Scenario

BFS = అపార్ట్‌మెంట్ లో floor-by-floor announcement. మీరు ground floor నుండి మొదలుపెట్టి – **ఒక floor లోని అందరికీ చెప్పాకే** పై floor కి వెళ్ళారు. levelSize = "ఈ floor లో ఎన్ని flats" – దాన్ని ముందే గుర్తుపెట్టుకోవాలి, ఎందుకంటే చెప్తున్నప్పుడే కొత్తవాళ్ళు (పై floor) queue లోకి వస్తారు.

Variants – అన్నీ ఒకే template మీద

```

// 1. Right side view – ప్రతి level లో చివరి node
function rightSideView(root) {
    if (!root) return [];
    const res = [], queue = [root];
    while (queue.length) {
        const size = queue.length;
        for (let i = 0; i < size; i++) {
            const node = queue.shift();
            if (i === size - 1) res.push(node.val);
            if (node.left) queue.push(node.left);
            if (node.right) queue.push(node.right);
        }
    }
    return res;
}

// 2. Zigzag – alternate levels reverse
function zigzagLevelOrder(root) {
    if (!root) return [];
    const res = [], queue = [root];
    let leftToRight = true;
    while (queue.length) {
        const size = queue.length, level = [];
        for (let i = 0; i < size; i++) {
            const node = queue.shift();
            if (leftToRight) level.push(node.val);
            else level.unshift(node.val);
            if (node.left) queue.push(node.left);
            if (node.right) queue.push(node.right);
        }
        res.push(level);
        leftToRight = !leftToRight;
    }
}

```

// 🗝 చివరిది

// 🗝 ముందు చేర్చడం

```

    return res;
}

// 3. Minimum depth – BFS లో మొదటి leaf దగ్గరే అగుతుంది ⚡ (DFS మొత్తం traverse చేస్తుంది)
function minDepth(root) {
    if (!root) return 0;
    const queue = [root];
    let depth = 1;
    while (queue.length) {
        const size = queue.length;
        for (let i = 0; i < size; i++) {
            const node = queue.shift();
            if (!node.left && !node.right) return depth; // ✅ మొదటి leaf
            if (node.left) queue.push(node.left);
            if (node.right) queue.push(node.right);
        }
        depth++;
    }
    return depth;
}

// 4. Connect next right pointers
function connect(root) {
    if (!root) return root;
    let leftmost = root;
    while (leftmost.left) { // perfect binary tree
        let head = leftmost;
        while (head) {
            head.left.next = head.right; // same parent
            if (head.next) head.right.next = head.next.left; // 🔑 across parents
            head = head.next;
        }
        leftmost = leftmost.left;
    }
    return root;
}

// O(1) space – next pointers నే queue గా వాడటం ⭐

```

BFS vs DFS – ఎప్పుడు ఏది

	BFS	DFS
Space	$O(w)$ – width	$O(h)$ – height
Shortest path (unweighted)	✅	❌
Level info	✅	కష్టం (depth pass చేయాలి)
Deep narrow tree	✅ (width చిన్నది)	⚠️ stack overflow
Wide shallow tree	⚠️ queue పెద్దది	✅
Path/backtracking	కష్టం	✅
Minimum depth	✅ వేగం	మొత్తం traverse

ఈ pattern కి చెందిన problems

Easy: Level Order Traversal · Average of Levels · Minimum Depth · Symmetric Tree
Medium: Level Order II · Zigzag Level Order · Right Side View · Populating Next Right Pointers
Find Largest Value in Each Row · Add One Row to Tree · Deepest Leaves Sum
All Nodes Distance K in Binary Tree
Hard: Vertical Order Traversal · Binary Tree Maximum Width

Gotchas

- `levelSize` ని ముందే snapshot చేయకపోవడం → loop లో queue పెరుగుతుంది → levels కలిసిపోతాయి.
- JS `shift()` $O(n)$ → పెద్ద trees లో $O(n^2)$; level-swap లేదా index pointer వాడాలి.
- Null children push చేయడం → queue లో nulls.
- Minimum depth కి DFS వాడటం – BFS వేగం (early exit).
- Zigzag లో `unshift()` $O(n)$ – చిన్న levels కి OK, పెద్దవాటికి push + reverse.

Interview దృష్టి

- "BFS vs DFS ఎప్పుడు?" → space (width vs height), level info, shortest path, early exit.
- "`levelSize` ఎందుకు అవసరం?" → level boundary గుర్తించడానికీ.

20. BST Patterns

🔍 Recognition signals

- ✓ "binary search tree" అని చెప్పారు
- ✓ "kth smallest/largest"
- ✓ "validate BST"
- ✓ "search / insert / delete in BST"
- ✓ "sorted order" అవసరం

🔑 BST యొక్క ఒకే ఒక్క సూత్రం

ప్రతి node కి: left subtree లో అన్నీ $<$ `node.val` $<$ right subtree లో అన్నీ

- ★ దీని పర్యవసానం: INORDER TRAVERSAL = SORTED ORDER
→ దాదాపు అన్ని BST problems కి ఇదే కీలకం

Real-life Scenario

BST = సరిగ్గా అమర్చిన నిఘంటువు. ప్రతి పేజీ దగ్గర – "ఈ పదం ముందా వెనుకా?" అని తెలుసు కాబట్టి సగం పుస్తకం వదిలేయొచ్చు. **Inorder traversal = మొదటి పేజీ నుండి చివరి పేజీ వరకు వరుసగా చదవడం** → అక్షర క్రమంలో వస్తుంది.

Template 1 – Search / Insert ($O(h)$)

```
function searchBST(root, val) {
  while (root) {
    if (root.val === val) return root;
    root = val < root.val ? root.left : root.right;    // 🔑 సగం వదిలేయడం
  }
  return null;
}

function insertIntoBST(root, val) {
  if (!root) return { val, left: null, right: null };
  if (val < root.val) root.left = insertIntoBST(root.left, val);
  else root.right = insertIntoBST(root.right, val);
  return root;
}
//  $O(h)$  – balanced అయితే  $O(\log n)$ , skewed అయితే  $O(n)$ 
```

Template 2 – Validate BST (range passing) ★

```
// ❌ సాధారణ తప్పు – parent తో మాత్రమే compare చేయడం
function isValidBSTWrong(root) {
  if (!root) return true;
  if (root.left && root.left.val >= root.val) return false;
  if (root.right && root.right.val <= root.val) return false;
  return isValidBSTWrong(root.left) && isValidBSTWrong(root.right);
}
//      5
//     / \
//    1   6
//     / \
//    3   7    ← 3 అనేది 5 కంటే తక్కువ కానీ కుడి subtree లో ఉంది → INVALID
//                పై code దీన్ని పట్టుకోదు! ❌

// ✅ సరైనది – valid range ని కిందికి పంపడం (top-down)
function isValidBST(root, min = -Infinity, max = Infinity) {
  if (!root) return true;
  if (root.val <= min || root.val >= max) return false;
  return isValidBST(root.left, min, root.val) &&    // 🔑 కుడి boundary = node.val
    isValidBST(root.right, root.val, max);          // 🔑 ఎడమ boundary = node.val
}

// ✅ ప్రత్యామ్నాయం – inorder sorted ఆ అని check
function isValidBSTInorder(root) {
  let prev = -Infinity, valid = true;
  function inorder(node) {
```

```

    if (!node || !valid) return;
    inorder(node.left);
    if (node.val <= prev) { valid = false; return; }
    prev = node.val;
    inorder(node.right);
  }
  inorder(root);
  return valid;
}

```

Template 3 — Kth smallest (inorder + counter)

```

function kthSmallest(root, k) {
  const stack = [];
  let curr = root, count = 0;

  while (curr || stack.length) {
    while (curr) { stack.push(curr); curr = curr.left; }
    curr = stack.pop();
    if (++count === k) return curr.val; // 🗝️ early exit - O(h + k)
    curr = curr.right;
  }
  return -1;
}
// ⚡ Iterative వాడటం వల్ల k దగ్గరే ఆగివ్వు (recursion లో మొత్తం traverse అవుతుంది)
// Follow-up: తరచూ kth అడుగుతుంటే → ప్రతి node లో subtree size store చేయాలి → O(h)

```

Template 4 — Delete node (3 cases)

```

function deleteNode(root, key) {
  if (!root) return null;

  if (key < root.val) root.left = deleteNode(root.left, key);
  else if (key > root.val) root.right = deleteNode(root.right, key);
  else {
    // కనుగొన్నాం - 3 cases
    if (!root.left) return root.right; // 1 ఎడమ లేదు
    if (!root.right) return root.left; // 2 కుడి లేదు
    // 3 రెండూ ఉన్నాయి → inorder successor (కుడి subtree లో అతిచిన్నది)
    let successor = root.right;
    while (successor.left) successor = successor.left;
    root.val = successor.val;
    root.right = deleteNode(root.right, successor.val);
  }
  return root;
}

```

Template 5 — LCA in BST (O(h), tree కంటే సులభం)

```

function lowestCommonAncestor(root, p, q) {
  while (root) {

```

```

    if (p.val < root.val && q.val < root.val) root = root.left;      // రెండూ ఎడమ
    else if (p.val > root.val && q.val > root.val) root = root.right; // రెండూ కుడి
    else return root;      // ✨ split అయ్యే చోట LCA
  }
  return null;
}

```

📄 Template 6 — Sorted array → balanced BST

```

function sortedArrayToBST(nums, left = 0, right = nums.length - 1) {
  if (left > right) return null;
  const mid = left + Math.floor((right - left) / 2); // ✨ middle = root → balanced
  return {
    val: nums[mid],
    left: sortedArrayToBST(nums, left, mid - 1),
    right: sortedArrayToBST(nums, mid + 1, right),
  };
}

```

ఈ pattern కి చెందిన problems

Easy: Search in a **BST** · Minimum Absolute **Difference** in **BST** · Range Sum of **BST**
 Convert Sorted Array to **BST** · Two Sum IV (**BST**)

Medium: Validate **BST** ✨ · Kth Smallest Element in a **BST** · **Insert/Delete** in **BST**
 LCA of **BST** · **BST** Iterator · Recover **BST** · Inorder Successor
 Convert **BST** to Greater Tree · Trim a **BST**

Hard: Serialize and Deserialize **BST** · **Count** of Smaller Numbers After Self

Gotchas

- **Validate BST లో parent-only comparison** → అత్యంత సాధారణ తప్పు (ఫైనల demo).
- **Duplicates** — $\leq$ vs $<$ (problem definition చూడాలి).
- **Infinity boundaries** — `Number.MIN_SAFE_INTEGER` వాడితే edge case fail కావచ్చు.
- **Kth smallest recursion తో** → early exit లేదు (iterative మేలు).
- **Skewed BST** → $O(n)$ operations (balanced కాదని గుర్తుంచుకోవాలి).
- **Delete లో successor తర్వాత recursive delete** మర్చిపోవడం.

Interview దృష్టి

- "BST validate ఎలా?" → range passing లేదా inorder — **naive parent comparison ఎందుకు తప్పు** చెప్పడం ముఖ్యం.
- "Kth smallest ని తరచూ అడిగితే?" → augmented BST (subtree sizes) → $O(h)$.
- "BST vs hash map?" → ordered operations (range, successor, kth) vs $O(1)$ lookup.

21-22. LCA, Path Problems, Construction & Serialization

LCA in general binary tree (BST కాదు)

```
function lowestCommonAncestor(root, p, q) {  
  if (!root || root === p || root === q) return root;    // 🔑 base  
  
  const left = lowestCommonAncestor(root.left, p, q);  
  const right = lowestCommonAncestor(root.right, p, q);  
  
  if (left && right) return root;          // 🔑 రెండు వైపులా దొరికాయి → ఇదే LCA  
  return left ?? right;                   // ఒక వైపు మాత్రమే → అదే పైకి వంపు  
}  
// O(n) – bottom-up pattern
```

Path Sum III (prefix sum on tree ★)

```
// ఏ node నుండి ఏ node వరకైనా (downward) sum = target ఉన్న paths ఎన్ని?  
function pathSum(root, targetSum) {  
  const prefixCount = new Map([[0, 1]]);    // 🔑 Pattern 4 (prefix sum) ని tree కి apply!  
  let count = 0;  
  
  function dfs(node, curSum) {  
    if (!node) return;  
    curSum += node.val;  
    count += prefixCount.get(curSum - targetSum) ?? 0;  
  
    prefixCount.set(curSum, (prefixCount.get(curSum) ?? 0) + 1);  
    dfs(node.left, curSum);  
    dfs(node.right, curSum);  
    prefixCount.set(curSum, prefixCount.get(curSum) - 1); // 🔑 BACKTRACK – ఈ path నుండి  
    బయటికి  
  }  
  dfs(root, 0);  
  return count;  
}  
// O(n) – naive O(n2) కంటే మేలు  
// 🔑 Array pattern ని tree కి apply చేయడం + backtracking – ఇది impressive answer
```

Tree construction (preorder + inorder)

```
function buildTree(preorder, inorder) {  
  const idxMap = new Map(inorder.map((v, i) => [v, i])); // 🔑 O(1) lookup  
  let preIdx = 0;  
  
  function build(left, right) {  
    if (left > right) return null;  
    const rootVal = preorder[preIdx++]; // 🔑 preorder = root మొదట  
    const root = { val: rootVal, left: null, right: null };  
    const mid = idxMap.get(rootVal); // 🔑 inorder లో root స్థానం  
    root.left = build(left, mid - 1);  
    root.right = build(mid, right);  
    return root;  
  }  
  return build(0, inorder.length - 1);  
}
```

```

    root.left = build(left, mid - 1); // ⚠️ క్రమం ముఖ్యం (preIdx)
    root.right = build(mid + 1, right);
    return root;
}
return build(0, inorder.length - 1);
}
// O(n) - hash map లేకపోతే O(n²)

```

📁 Serialize & Deserialize

```

function serialize(root) {
    const res = [];
    function dfs(node) {
        if (!node) { res.push("#"); return; } // 🔑 null marker తప్పనిసరి
        res.push(String(node.val));
        dfs(node.left); dfs(node.right);
    }
    dfs(root);
    return res.join(",");
}

function deserialize(data) {
    const vals = data.split(",");
    let i = 0;
    function build() {
        if (vals[i] === "#") { i++; return null; }
        const node = { val: Number(vals[i++]), left: null, right: null };
        node.left = build();
        node.right = build();
        return node;
    }
    return build();
}
// 🔑 Null markers లేకపోతే - preorder ఒక్కటే tree ని uniquely define చేయదు

```

Key Points

- **LCA (general tree)** = bottom-up; రెండు వైపులా దిరిగితే ఆ node.
- **Path Sum III** = prefix sum + backtracking on tree (array pattern transfer ★).
- **Construction** = preorder/postorder (root) + inorder (split) + hash map.
- **Serialization** = null markers తప్పనిసరి.

Interview దృష్టి

- "LCA ఎలా?" → BST లో $O(h)$ (values compare), general tree లో $O(n)$ (bottom-up).
- "Path Sum III $O(n)$ ఎలా?" → prefix sum map + backtrack — **array pattern ని tree కి transfer చేయడం senior signal.**

Part 6 — Graph Patterns

23. Graph Representation & Traversal

🔍 Recognition signals

- ✓ "connected" / "path exists" / "network"
- ✓ "prerequisites" / "dependencies"
- ✓ Nodes మధ్య relationships
- ✓ Grid (ఇది కూడా graph! – Pattern 24)
- ✓ "friends" / "provinces" / "islands"

📄 Representations

```
// 1. Adjacency List (99% cases – ఇదే వాడాలి) ✓
const graph = new Map();
function addEdge(u, v, directed = false) {
  if (!graph.has(u)) graph.set(u, []);
  if (!graph.has(v)) graph.set(v, []);
  graph.get(u).push(v);
  if (!directed) graph.get(v).push(u);
}
// Space:  $O(V + E)$  · Neighbors:  $O(\text{degree})$ 

// 2. Edge list → adjacency list (interview input సాధారణంగా ఇలా వస్తుంది)
function buildGraph(n, edges, directed = false) {
  const adj = Array.from({ length: n }, () => []);
  for (const [u, v] of edges) {
    adj[u].push(v);
    if (!directed) adj[v].push(u);
  }
  return adj;
}

// 3. Adjacency Matrix (dense graphs,  $O(1)$  edge lookup)
const matrix = Array.from({ length: n }, () => new Array(n).fill(0));
// Space:  $O(V^2)$  – sparse graphs కి వృథా
```

📄 Template — DFS

```
function dfs(graph, start, visited = new Set()) {
  visited.add(start);
  process(start);

  for (const neighbor of graph.get(start) ?? []) {
    if (!visited.has(neighbor)) dfs(graph, neighbor, visited);
  }
}
```

```
// Iterative (deep graphs కి – stack overflow నివారణ)
function dfsIterative(graph, start) {
  const visited = new Set([start]);
  const stack = [start];
  while (stack.length) {
    const node = stack.pop();
    process(node);
    for (const nb of graph.get(node) ?? []) {
      if (!visited.has(nb)) { visited.add(nb); stack.push(nb); }
      // ⚠️ push సమయంలో visited mark చేయడం ముఖ్యం (duplicates నివారణ)
    }
  }
}
```

📁 Template — BFS (shortest path in unweighted graph) ★

```
function bfs(graph, start) {
  const visited = new Set([start]);
  const queue = [start];
  const dist = new Map([[start, 0]]);

  while (queue.length) {
    const node = queue.shift();
    for (const nb of graph.get(node) ?? []) {
      if (!visited.has(nb)) {
        visited.add(nb);
        dist.set(nb, dist.get(node) + 1);
        queue.push(nb);
      }
    }
  }
  return dist;
}
// 🔑 Unweighted graph లో BFS = shortest path (ప్రతి edge weight 1)
```

Real-life Scenario

DFS = ఒక దారిలో చివరివరకు వెళ్ళి, dead end వస్తే వెనక్కి వచ్చి వేరే దారి. అడవిలో దారి వెతకడం లాంటిది – ఒక దారి పట్టుకొని చివరివరకు.

BFS = అలలు (ripples) విస్తరించడం. నీటిలో రాయి వేస్తే – వృత్తాలు దూరం ప్రకారం విస్తరిస్తాయి. అందుకే మొదట కలిసేదే shortest path.

Cycle detection

```
// Undirected graph – parent tracking
function hasCycleUndirected(graph, n) {
  const visited = new Set();
  function dfs(node, parent) {
    visited.add(node);
```

```

    for (const nb of graph.get(node) ?? []) {
      if (!visited.has(nb)) { if (dfs(nb, node)) return true; }
      else if (nb !== parent) return true; // 🔑 visited + parent కాదు → cycle
    }
    return false;
  }
  for (let i = 0; i < n; i++) if (!visited.has(i) && dfs(i, -1)) return true;
  return false;
}

// Directed graph – 3 colors (white/gray/black)
function hasCycleDirected(graph, n) {
  const state = new Array(n).fill(0); // 0=unvisited, 1=in-progress, 2=done
  function dfs(node) {
    if (state[node] === 1) return true; // 🔑 back edge → cycle
    if (state[node] === 2) return false; // ఇప్పటికే process అయింది
    state[node] = 1;
    for (const nb of graph[node] ?? []) if (dfs(nb)) return true;
    state[node] = 2;
    return false;
  }
  for (let i = 0; i < n; i++) if (dfs(i)) return true;
  return false;
}

// ⚠️ Undirected vs directed cycle detection పూర్తిగా వేరు – గందరగోళపడిద్దు

```

Connected components

```

function countComponents(n, edges) {
  const adj = buildGraph(n, edges);
  const visited = new Set();
  let count = 0;

  for (let i = 0; i < n; i++) {
    if (!visited.has(i)) {
      count++;
      const stack = [i];
      visited.add(i);
      while (stack.length) {
        const node = stack.pop();
        for (const nb of adj[node]) if (!visited.has(nb)) { visited.add(nb); stack.push(nb); }
      }
    }
  }
  return count;
}

// 🔑 Union-Find తో కూడా చేయొచ్చు (Pattern 26)

```

Gotchas

- **Visited set మర్చిపోవడం** → infinite loop (cycles ఉంటే).
- **BFS లో dequeue సమయంలో visited mark చేయడం** → duplicates queue లోకి → $O(V^2)$.
- **Disconnected components** – అన్ని nodes నుండి traversal మొదలుపెట్టాలి.

- **Directed vs undirected cycle detection** గందరగోళం.
- **Self-loops, parallel edges** handle చేయకపోవడం.
- **Deep recursion** → iterative DFS వాడాలి.

Interview దృష్టి

- "BFS vs DFS graph లో?" → shortest path (BFS), path existence/cycle (DFS), memory (width vs depth).
- "Cycle detection directed vs undirected?" → 3-color vs parent tracking.

24. Grid/Matrix as Graph ★

🔍 Recognition signals

- ✓ 2D grid + "islands" / "regions" / "connected"
- ✓ "flood fill" / "paint"
- ✓ "shortest path in maze/grid"
- ✓ "rotting oranges" / "spread" (multi-source BFS)
- ✓ "surrounded regions" / "border"

వివరణ

Grid = implicit graph. ప్రతి cell ఒక node; పక్క cells (4 లేదా 8 దిశలు) edges. **Adjacency list build చేయనవసరం లేదు** — directions array చాలు.

📄 Universal Template — Grid DFS (islands)

```
const DIRS = [[0, 1], [1, 0], [0, -1], [-1, 0]]; // right, down, left, up
// 8 directions: [[-1,-1],[-1,0],[-1,1],[0,-1],[0,1],[1,-1],[1,0],[1,1]]

function numIslands(grid) {
  if (!grid.length) return 0;
  const m = grid.length, n = grid[0].length;
  let count = 0;

  function dfs(r, c) {
    // 🔑 Boundary + validity check ఒకే చోట (recursion సులభం)
    if (r < 0 || r >= m || c < 0 || c >= n || grid[r][c] !== "1") return;
    grid[r][c] = "0"; // 🔑 mark visited (in-place)
    for (const [dr, dc] of DIRS) dfs(r + dr, c + dc);
  }

  for (let r = 0; r < m; r++)
    for (let c = 0; c < n; c++)
      if (grid[r][c] === "1") { count++; dfs(r, c); } // కొత్త island

  return count;
}
```

```
// O(mxn) time, O(mxn) worst-case recursion space
// ⚠️ Input modify చేయకూడదంటే → visited 2D array
```

Real-life Scenario

Grid DFS = ఇంక చుక్క కాగితం మీద పడటం. ఒక cell లో ఇంక పడితే – అది అన్ని పక్క cells కి వ్యాపిస్తుంది (అదే రంగు ఉన్నంతవరకు). ఒక చుక్క = ఒక island. **Visited mark = ఆ ప్రాంతాన్ని రంగు మార్చడం** – మళ్ళీ లెక్కించకుండా.

Multi-source BFS = అనేక చోట్ల ఒకేసారి ఇంక పడటం – అన్నీ ఒకే వేగంతో వ్యాపిస్తాయి.

📄 Template – Grid BFS (shortest path in maze)

```
function shortestPathBinaryMatrix(grid) {
  const n = grid.length;
  if (grid[0][0] === 1 || grid[n-1][n-1] === 1) return -1;

  const DIRS8 = [[-1,-1],[-1,0],[-1,1],[0,-1],[0,1],[1,-1],[1,0],[1,1]];
  const queue = [[0, 0, 1]]; // [row, col, distance]
  grid[0][0] = 1; // mark visited

  while (queue.length) {
    const [r, c, dist] = queue.shift();
    if (r === n - 1 && c === n - 1) return dist; // ✅ మొదటిసారి చేరిందే shortest

    for (const [dr, dc] of DIRS8) {
      const nr = r + dr, nc = c + dc;
      if (nr >= 0 && nr < n && nc >= 0 && nc < n && grid[nr][nc] === 0) {
        grid[nr][nc] = 1; // 🔑 enqueue సమయంలో mark
        queue.push([nr, nc, dist + 1]);
      }
    }
  }
  return -1;
}
```

📄 Multi-source BFS ★ (అత్యంత ఉపయోగకరమైన variant)

```
// Rotting Oranges – అన్ని rotten oranges ఒకేసారి వ్యాపిస్తాయి
function orangesRotting(grid) {
  const m = grid.length, n = grid[0].length;
  const queue = [];
  let fresh = 0;

  // 🔑 అన్ని sources ని ఒకేసారి queue లో పెట్టడం
  for (let r = 0; r < m; r++)
    for (let c = 0; c < n; c++) {
      if (grid[r][c] === 2) queue.push([r, c]);
      else if (grid[r][c] === 1) fresh++;
    }

  let minutes = 0;
```

```

while (queue.length && fresh > 0) {
  const size = queue.length;
  for (let i = 0; i < size; i++) {
    const [r, c] = queue.shift();
    for (const [dr, dc] of DIRS) {
      const nr = r + dr, nc = c + dc;
      if (nr >= 0 && nr < m && nc >= 0 && nc < n && grid[nr][nc] === 1) {
        grid[nr][nc] = 2;
        fresh--;
        queue.push([nr, nc]);
      }
    }
  }
  minutes++;
}
return fresh === 0 ? minutes : -1;
}
// 🗝 Multi-source BFS = "అనేక చోట్ల నుండి ఒకసారి వ్యాపించడం" -
// Walls and Gates, 01 Matrix, Rotting Oranges, Shortest Bridge అన్నిటికీ ఇదే

```

📁 Border-first pattern (Surrounded Regions)

```

function solve(board) {
  const m = board.length, n = board[0].length;

  // 🗝 అంతర్దృష్టి: border ని touch చేసేవి "safe" - వాటిని ముందు mark చేయడం
  function dfs(r, c) {
    if (r < 0 || r >= m || c < 0 || c >= n || board[r][c] !== "0") return;
    board[r][c] = "S";
    for (const [dr, dc] of DIRS) dfs(r + dr, c + dc);
  }

  for (let r = 0; r < m; r++) { dfs(r, 0); dfs(r, n - 1); }
  for (let c = 0; c < n; c++) { dfs(0, c); dfs(m - 1, c); }

  for (let r = 0; r < m; r++)
    for (let c = 0; c < n; c++)
      board[r][c] = board[r][c] === "S" ? "0" : "X";

  // 🗝 "లోపలివి కనుగొనడం కష్టం → బయటివి కనుగొని మిగతావి తీసుకోవడం" - reverse thinking
}

```

ఈ pattern కి చెందిన problems

Easy: Flood Fill · Island Perimeter · Max Area of Island
Medium: Number of Islands 🌟 · Rotting Oranges 🌟 · Surrounded Regions
 Pacific Atlantic Water Flow · 01 Matrix · Walls and Gates
 Number of Closed Islands · Shortest Path in Binary Matrix
 Word Search (backtracking + grid)
Hard: Shortest Bridge · Word Search II (trie + grid) · Trapping Rain Water II (heap + BFS)
 Making A Large Island · Cut Off Trees for Golf Event

Gotchas

- **Boundary checks మర్చిపోవడం** → `undefined` access.
- **Visited mark చేయకపోవడం** → infinite recursion.
- **BFS లో dequeue సమయంలో mark చేయడం** → duplicates → TLE.
- **4 vs 8 directions** — problem చదవాలి.
- **Input modify చేయొచ్చా?** — చేయకూడదంటే visited array.
- **Multi-source BFS ని single-source loop గా చేయడం** → $O(\text{sources} \times mn)$ (multi-source $O(mn)$).
- **Deep recursion** (10^6 cells) → iterative DFS/BFS.

Interview దృష్టి

- "Islands ని DFS/BFS/Union-Find మూడింటితో చేయి" → trade-offs (recursion depth, streaming input).
- "Multi-source BFS ఎప్పుడు?" → అనేక starting points నుండి ఒకేసారి spread — "అన్నిటికీ shortest distance".
- "Grid = graph ఎలా?" → implicit adjacency (directions array), $V = m \times n$, $E = 4 \times m \times n$.

25. Topological Sort

🔍 Recognition signals

- ✓ "prerequisites" / "dependencies" / "before/after"
- ✓ "course schedule" / "build order" / "task ordering"
- ✓ DAG (directed acyclic graph)
- ✓ "valid ordering exists?" (cycle detection కూడా)

వివరణ

Topological sort = dependencies ని గౌరవించే linear ordering. DAG లో మాత్రమే సాధ్యం (cycle ఉంటే అసాధ్యం).

Real-life Scenario

Topological sort = వంట క్రమం. బిర్యానీ చేయాలంటే — బియ్యం నానబెట్టాలి → ఉడకబెట్టాలి; మసాలా వేయించాలి → చికెన్ కలపాలి; చివరికి రెండూ కలిపి దమ్ము. **కొన్ని పనులు ఇతర పనుల తర్వాతే** చేయాలి. కానీ "బియ్యం నానబెట్టడం" మరియు "మసాలా వేయించడం" — వీటి మధ్య dependency లేదు, ఏ క్రమంలోనైనా చేయొచ్చు.

Cycle = "A కి B కావాలి, B కి A కావాలి" → ఎప్పటికీ మొదలుపెట్టలేం (deadlock).

📄 Template 1 — Kahn's Algorithm (BFS, indegree) ★

```
function topologicalSort(numCourses, prerequisites) {  
  const adj = Array.from({ length: numCourses }, () => []);  
  const indegree = new Array(numCourses).fill(0);
```

```

for (const [course, prereq] of prerequisites) {
  adj[prereq].push(course); // prereq → course
  indegree[course]++; // 🗝️ course కి ఒక dependency
}

// 1 Indegree 0 ఉన్నవి (ఏ dependency లేనివి) queue లోకి
const queue = [];
for (let i = 0; i < numCourses; i++) if (indegree[i] === 0) queue.push(i);

const order = [];
while (queue.length) {
  const node = queue.shift();
  order.push(node);

  for (const next of adj[node]) {
    indegree[next]--;
    if (indegree[next] === 0) queue.push(next); // 🗝️ ఒక dependency తీరింది
  } // అన్నీ తీరాయి → ready
}

// 2 అన్ని nodes process అయ్యాయా? → లేకపోతే cycle ✅
return order.length === numCourses ? order : [];
}

// O(V + E) time, O(V + E) space

```

Kahn's algorithm cycle detection ని ఉచితంగా ఇస్తుంది – `order.length < n` అంటే cycle ఉంది.

📄 Template 2 — DFS-based (postorder reverse)

```

function topologicalSortDFS(n, edges) {
  const adj = Array.from({ length: n }, () => []);
  for (const [u, v] of edges) adj[u].push(v);

  const state = new Array(n).fill(0); // 0=unvisited, 1=visiting, 2=done
  const order = [];
  let hasCycle = false;

  function dfs(node) {
    if (state[node] === 1) { hasCycle = true; return; } // 🗝️ back edge → cycle
    if (state[node] === 2) return;

    state[node] = 1;
    for (const nb of adj[node]) dfs(nb);
    state[node] = 2;
    order.push(node); // 🗝️ postorder – పిల్లలందరూ అయ్యాక
  }

  for (let i = 0; i < n; i++) if (state[i] === 0) dfs(i);
  return hasCycle ? [] : order.reverse(); // 🗝️ reverse postorder = topological order
}

```

Variants

```
// 1. Course Schedule – order అవసరం లేదు, సాధ్యమో అని మాత్రమే
function canFinish(numCourses, prerequisites) {
    return topologicalSort(numCourses, prerequisites).length === numCourses;
}

// 2. Alien Dictionary (Hard) – words నుండి character ordering deduce చేయడం
function alienOrder(words) {
    const adj = new Map(), indegree = new Map();
    for (const w of words) for (const c of w) { adj.set(c, new Set()); indegree.set(c, 0); }

    for (let i = 0; i < words.length - 1; i++) {
        const [w1, w2] = [words[i], words[i + 1]];
        if (w1.length > w2.length && w1.startsWith(w2)) return ""; // 🗝️ invalid: "abc" before "ab"
        for (let j = 0; j < Math.min(w1.length, w2.length); j++) {
            if (w1[j] !== w2[j]) {
                if (!adj.get(w1[j]).has(w2[j])) {
                    adj.get(w1[j]).add(w2[j]);
                    indegree.set(w2[j], indegree.get(w2[j]) + 1);
                }
                break; // 🗝️ మొదటి తేడా మాత్రమే
            }
        }
    }
    // Kahn's...
}

// 3. Minimum Height Trees – leaves నుండి లోపలికి "peeling"
// 4. Parallel Courses – levels = semesters (BFS level count)
```

ఈ pattern కి చెందిన problems

Medium: [Course Schedule I/II](#) ★ · [Minimum Height Trees](#) · [Sequence Reconstruction](#)
[All Ancestors of a Node in a DAG](#) · [Find Eventual Safe States](#)
[Parallel Courses](#) · [Sort Items by Groups Respecting Dependencies](#)
Hard: [Alien Dictionary](#) ★ · [Course Schedule III](#) · [Longest Increasing Path in Matrix](#)
(DFS+memo)

Gotchas

- **Edge direction తారుమారు** – `[a, b]` అంటే "a కి b కావాలి" ఆ, "a తర్వాత b" ఆ? **Problem జాగ్రత్తగా చదవాలి.**
- **Cycle check మర్చిపోవడం** → invalid ordering return.
- **Disconnected components** – అన్ని indegree-0 nodes ని queue లో పెట్టాలి.
- **Alien dictionary లో prefix case** ("abc" ముందు "ab") – invalid input.
- **Self-loop** → ఎప్పుడూ cycle.

Interview దృష్టి

- "Cycle ఉందని ఎలా తెలుసుకుంటావు?" → Kahn's లో `order.length < n` ; DFS లో gray node (back edge).
- "BFS vs DFS topological sort?" → Kahn's సహజం + cycle detection built-in; DFS reverse postorder.
- "Multiple valid orderings?" → అవును (dependencies లేని nodes మధ్య ఏ క్రమమైనా).

26. Union-Find (Disjoint Set Union)

🔍 Recognition signals

- ✓ "connected components" (dynamic – edges add అవుతూ ఉంటాయి)
- ✓ "are these two connected?"
- ✓ "redundant connection" / "cycle in undirected graph"
- ✓ "number of provinces/friend circles"
- ✓ "minimum spanning tree" (Kruskal)
- ✓ "accounts merge" / grouping by connectivity

వివరణ

Union-Find = గుంపుల నిర్వహణ. రెండు operations: `find(x)` – x ఏ గుంపులో ఉంది? `union(x, y)` – రెండు గుంపులను కలపడం.

Real-life Scenario

Union-Find = గ్రామాల మధ్య వంతెనలు. ప్రతి గ్రామానికి ఒక "పెద్ద" (representative) ఉంటాడు. "ఈ రెండు గ్రామాలు కలిసి ఉన్నాయా?" అంటే – ఇద్దరి పెద్దలు ఒక్కరేనా అని చూస్తారు. కొత్త వంతెన కడితే (union) – ఒక పెద్ద ఇంకో పెద్ద కింద చేరతాడు.

Path compression = "నేను ఎవరి కిందున్నానో నేరుగా గుర్తుపెట్టుకోవడం" – ప్రతిసారి గొలుసు మొత్తం అడగనవసరం లేదు.

📁 Universal Template (optimizations తో)

```
class UnionFind {
  constructor(n) {
    this.parent = Array.from({ length: n }, (_, i) => i); // ప్రతివాడూ తనకి తానే పెద్ద
    this.rank = new Array(n).fill(0); // tree ఎత్తు (balancing కి)
    this.count = n; // components సంఖ్య
  }

  find(x) {
    if (this.parent[x] !== x) {
      this.parent[x] = this.find(this.parent[x]); // 🔑 PATH COMPRESSION
    }
    return this.parent[x];
  }
}
```

```

union(x, y) {
  const rootX = this.find(x), rootY = this.find(y);
  if (rootX === rootY) return false; // ఇప్పటికే కలిసి ఉన్నారు (cycle!)

  // 🔑 UNION BY RANK - చిన్న tree ని పెద్దదాని కింద (ఎత్తు తక్కువగా ఉంచడం)
  if (this.rank[rootX] < this.rank[rootY]) this.parent[rootX] = rootY;
  else if (this.rank[rootX] > this.rank[rootY]) this.parent[rootY] = rootX;
  else { this.parent[rootY] = rootX; this.rank[rootX]++; }

  this.count--;
  return true;
}

connected(x, y) { return this.find(x) === this.find(y); }
}
// రెండు optimizations తో: దాదాపు O(1) per operation - ఖచ్చితంగా O(α(n)) (inverse Ackermann, ≤ 4)

```

రెండు optimizations లేకపోతే – worst case $O(n)$ per operation (linked list లా degenerate అవుతుంది).
Interview లో రెండూ రాయాలి.

ఉదాహరణలు

```

// 1. Number of Provinces (connected components)
function findCircleNum(isConnected) {
  const n = isConnected.length;
  const uf = new UnionFind(n);
  for (let i = 0; i < n; i++)
    for (let j = i + 1; j < n; j++)
      if (isConnected[i][j]) uf.union(i, j);
  return uf.count;
}

// 2. Redundant Connection - cycle create చేసే edge
function findRedundantConnection(edges) {
  const uf = new UnionFind(edges.length + 1);
  for (const [u, v] of edges) {
    if (!uf.union(u, v)) return [u, v]; // 🔑 union false → ఇప్పటికే connected → cycle
  }
  return [];
}

// 3. Accounts Merge (emails ద్వారా people ని merge)
function accountsMerge(accounts) {
  const uf = new UnionFind(accounts.length);
  const emailToId = new Map();

  for (let i = 0; i < accounts.length; i++) {
    for (let j = 1; j < accounts[i].length; j++) {
      const email = accounts[i][j];
      if (emailToId.has(email)) uf.union(i, emailToId.get(email)); // 🔑 ఒకే email → ఒకే వ్యక్తి
      else emailToId.set(email, i);
    }
  }
}

```

```

}

const groups = new Map();
for (const [email, id] of emailToId) {
    const root = uf.find(id);
    if (!groups.has(root)) groups.set(root, []);
    groups.get(root).push(email);
}
return [...groups].map(([id, emails]) => [accounts[id][0], ...emails.sort()]);
}

// 4. Kruskal's MST (Minimum Spanning Tree)
function minimumCost(n, connections) {
    connections.sort((a, b) => a[2] - b[2]); // 🗝 cost ప్రకారం sort (greedy)
    const uf = new UnionFind(n + 1);
    let cost = 0, edges = 0;
    for (const [u, v, w] of connections) {
        if (uf.union(u, v)) { cost += w; edges++; } // cycle create చేయనిదైతే తీసుకో
    }
    return edges === n - 1 ? cost : -1; // అన్నీ connect అయ్యాయా?
}

```

Union-Find vs DFS/BFS – ఎప్పుడు ఏది

	Union-Find	DFS/BFS
Dynamic edges (streaming)	✅	❌ (ప్రతిసారి re-run)
"ఈ రెండు connected?" queries	✅ $O(\alpha)$	$O(V+E)$ ప్రతిసారి
Path కావాలి	❌	✅
Directed graphs	❌ (undirected కి)	✅
MST (Kruskal)	✅	❌
Cycle detection (undirected)	✅ సాగపెనది	✅
Implementation	~20 lines	~10 lines

ఈ pattern కి చెందిన problems

Medium: Number of Provinces · Redundant Connection · Accounts Merge ★
 Number of Islands (UF version) · Graph Valid Tree · Satisfiability of Equality Equations
 Most Stones Removed · Number of Operations to Make Network Connected
Hard: Redundant Connection II · Number of Islands II (dynamic!) ★
 Swim in Rising Water · Making A Large Island

Gotchas

- Path compression లేకపోవడం → $O(n)$ per find → TLE.

- **Union by rank/size లేకపోవడం** → skewed trees.
- **find బదులు parent[x] నేరుగా compare** → తప్పు (root కావాలి).
- **1-indexed vs 0-indexed** — `new UnionFind(n+1)` అవసరమా?
- **Union return value వాడకపోవడం** — cycle detection కి అదే కీలకం.
- **Directed graphs కి Union-Find** — పని చేయదు.

Interview దృష్టి

- *"Union-Find complexity?"* → path compression + union by rank → $O(\alpha(n)) \approx O(1)$ amortized.
- *"DFS బదులు Union-Find ఎప్పుడు?"* → dynamic connectivity, streaming edges, MST.
- *"Path compression ఎలా పని చేస్తుంది?"* → find సమయంలో అన్ని nodes ని నేరుగా root కి attach.

27. Shortest Path Algorithms

🔍 ఏది ఎప్పుడు — decision table ★

పరిస్థితి	Algorithm	Complexity
Unweighted graph	BFS	$O(V + E)$
Weighted, non-negative	Dijkstra	$O((V+E) \log V)$
Weighted, negative edges	Bellman-Ford	$O(V \times E)$
అన్ని jodi shortest paths	Floyd-Warshall	$O(V^3)$
Weighted + heuristic (games/maps)	A*	varies
DAG	Topological sort + relax	$O(V + E)$
Exactly k stops/edges	Bellman-Ford (k iterations)	$O(k \times E)$

📋 Dijkstra (min-heap తో)

```
function dijkstra(n, edges, start) {
  const adj = Array.from({ length: n }, () => []);
  for (const [u, v, w] of edges) { adj[u].push([v, w]); adj[v].push([u, w]); }

  const dist = new Array(n).fill(Infinity);
  dist[start] = 0;
  const heap = new MinHeap((a, b) => a[0] - b[0]); // [distance, node]
  heap.push([0, start]);

  while (heap.size) {
    const [d, node] = heap.pop();
    if (d > dist[node]) continue; // 🗝️ stale entry - skip

    for (const [nb, weight] of adj[node]) {
```

```

    const newDist = d + weight;
    if (newDist < dist[nb]) {
        dist[nb] = newDist;
        heap.push([newDist, nb]);
    }
}
return dist;
}
// O((V + E) log V)
// ⚠️ Negative weights తో పని చేయదు (greedy assumption విఫలం)

```

Dijkstra intuition: "ఇప్పటివరకు తెలిసిన అతి దగ్గరి node ని finalize చేయి, దాని పొరుగువాళ్ళ దూరాలను update చేయి." Negative edges ఉంటే – finalize చేసిన తర్వాత కూడా మెరుగైన path రావొచ్చు → greedy విఫలం.

📁 Bellman-Ford (negative weights + k-stops)

```

// Cheapest Flights Within K Stops
function findCheapestPrice(n, flights, src, dst, k) {
    let dist = new Array(n).fill(Infinity);
    dist[src] = 0;

    for (let i = 0; i <= k; i++) {
        const temp = [...dist];
        for (const [from, to, price] of flights) {
            if (dist[from] !== Infinity && dist[from] + price < temp[to]) {
                temp[to] = dist[from] + price;
            }
        }
        dist = temp;
    }
    return dist[dst] === Infinity ? -1 : dist[dst];
}
// ⚡ temp array ఎందుకు: ఒకే iteration లో ఒకటి కంటే ఎక్కువ edges వాడకుండా ఆపడానికి

```

📁 Floyd-Warshall (all pairs)

```

function floydWarshall(n, edges) {
    const dist = Array.from({ length: n }, (_, i) =>
        Array.from({ length: n }, (_, j) => (i === j ? 0 : Infinity))
    );
    for (const [u, v, w] of edges) dist[u][v] = Math.min(dist[u][v], w);

    for (let k = 0; k < n; k++)
        for (let i = 0; i < n; i++)
            for (let j = 0; j < n; j++)
                dist[i][j] = Math.min(dist[i][j], dist[i][k] + dist[k][j]);

    return dist;
}

```

```
//  $O(V^3) - n \leq 100-500$  అయితే మాత్రమే ఆచరణీయం  
// 🗝️ "k గుండా వెళ్తే i నుండి j కి దూరం తగ్గుతుందా?"
```

📁 0-1 BFS (weights 0 లేదా 1 మాత్రమే – deque)

```
// Dijkstra అవసరం లేదు – deque చాలు,  $O(V+E)$   
function zeroOneBFS(grid) {  
  const deque = [[0, 0, 0]]; // [dist, r, c]  
  while (deque.length) {  
    const [d, r, c] = deque.shift();  
    for (const [nr, nc, cost] of neighbors(r, c)) {  
      if (cost === 0) deque.unshift([d, nr, nc]); // 🗝️ ముందుకి (priority)  
      else deque.push([d + 1, nr, nc]); // వెనుకకి  
    }  
  }  
}
```

ఈ pattern కి చెందిన problems

Medium: Network Delay Time (Dijkstra) · Path with Maximum Probability
Cheapest Flights Within K Stops (Bellman-Ford) ★
Path With Minimum Effort (Dijkstra/binary search)
Number of Ways to Arrive at Destination

Hard: Swim in Rising Water · Minimum Cost to Make at Least One Valid Path (0-1 BFS)
Find the City With the Smallest Number of Neighbors (Floyd-Warshall)
Reachable Nodes In Subdivided Graph

Gotchas

- Unweighted graph కి Dijkstra – BFS సరిపోతుంది (సులభం + వేగం).
- Negative weights తో Dijkstra → తప్పు సమాధానం (silent!).
- Stale heap entries skip చేయకపోవడం → నెమ్మది (కానీ correctness OK).
- Bellman-Ford లో temp array లేకపోవడం (k-stops problems లో) → ఎక్కువ edges వాడేస్తుంది.
- Floyd-Warshall loop order – k బయటి loop (i, j కాదు!) – తప్పు order = తప్పు సమాధానం.
- JS లో heap లేదు – implement చేయాలి.

Interview దృష్టి

- "Unweighted graph లో shortest path?" → BFS (Dijkstra అనవసరం).
- "Negative weights ఉంటే?" → Bellman-Ford; negative cycle detect కూడా చేయొచ్చు (n-th iteration లో ఇంకా relax అయితే).
- "Dijkstra ఎందుకు negative weights తో fail?" → finalized node ని మళ్ళీ improve చేయలేం (greedy invariant).

Part 7 — Recursion, Backtracking & Divide-Conquer

28. Backtracking Universal Template ★

🔍 Recognition signals

- ✓ "all possible" / "all combinations" / "all permutations" / "all subsets"
- ✓ "generate all valid ..." (parentheses, IP addresses, words)
- ✓ "find a path/solution" (N-Queens, Sudoku, maze)
- ✓ n చాలా చిన్నది ($n \leq 20$ – exponential OK అని సూచన)
- ✓ నిర్ణయాల చెట్టు (decision tree) – ప్రతి step లో ఎంపికలు

వివరణ

Backtracking = "అన్ని అవకాశాలను క్రమపద్ధతిలో try చేయడం, పని చేయనివి వేసక్కి తీసుకోవడం."

మూడు దశలు (ప్రతి backtracking problem లో ఇవే):

- 1 CHOOSE – ఒక ఎంపిక తీసుకో
- 2 EXPLORE – ఆ ఎంపికతో ముందుకు వెళ్ళు (recursion)
- 3 UNCHOOSE – ఆ ఎంపికను వేసక్కి తీసుకో (backtrack) ★

Real-life Scenario

Backtracking = చిక్కుముడి (maze) లో దారి వెతకడం, చేతిలో సుద్దముక్కతో. ఒక దారి పట్టుకొని ముందుకు వెళ్ళారు, గోడ మీద గుర్తు పెడుతూ (choose). Dead end వస్తే – వేసక్కి వస్తూ ఆ గుర్తులను చెరిపేస్తారు (unchoose), వేరే దారి try చేస్తారు.

గుర్తులు చెరపడం (unchoose) ఎందుకు కీలకం: లేకపోతే – తర్వాతి దారిలో వెళ్ళినప్పుడు, పాత దారి గుర్తులు మీ నిర్ణయాలను పాడు చేస్తాయి.

📄 Universal Template

```
function backtrack(state, choices, result) {
  // 1 Base case – పూర్తి solution దొరికింది
  if (isComplete(state)) {
    result.push([...state]); // 🔑 COPY! (reference పంపితే తర్వాత మారుతుంది)
    return;
  }

  // 2 ప్రతి ఎంపికను try చేయడం
  for (const choice of choices) {
    if (!isValid(choice, state)) continue; // ✂ PRUNING (invalid → skip)

    state.push(choice); // 1 CHOOSE
    backtrack(state, updatedChoices, result); // 2 EXPLORE
    state.pop(); // 3 UNCHOOSE ★
  }
}
```

```
}  
}
```

Pattern A — Subsets (ప్రతి element: తీసుకో/వద్దు)

```
function subsets(nums) {  
  const res = [];  
  function backtrack(start, current) {  
    res.push([...current]); // 🔑 ప్రతి state ఒక valid subset  
  
    for (let i = start; i < nums.length; i++) {  
      current.push(nums[i]); // choose  
      backtrack(i + 1, current); // 🔑 i+1 (ప్రతిదే ఒక్కసారే)  
      current.pop(); // unchoose  
    }  
  }  
  backtrack(0, []);  
  return res;  
}  
//  $O(2^n \times n)$  -  $2^n$  subsets, ప్రతిదాన్ని copy చేయడానికి  $O(n)$   
  
// Duplicates ఉంటే - Subsets II  
function subsetsWithDup(nums) {  
  nums.sort((a, b) => a - b); // 🔑 duplicates పక్కపక్కన  
  const res = [];  
  function backtrack(start, current) {  
    res.push([...current]);  
    for (let i = start; i < nums.length; i++) {  
      if (i > start && nums[i] === nums[i - 1]) continue; // 🔑 ఒకే level లో duplicate skip  
      current.push(nums[i]);  
      backtrack(i + 1, current);  
      current.pop();  
    }  
  }  
  backtrack(0, []);  
  return res;  
}
```

Pattern B — Permutations (క్రమం ముఖ్యం)

```
function permute(nums) {  
  const res = [], used = new Array(nums.length).fill(false);  
  
  function backtrack(current) {  
    if (current.length === nums.length) { res.push([...current]); return; }  
  
    for (let i = 0; i < nums.length; i++) { // 🔑 ప్రతిసారి 0 నుండి (subsets లో start నుండి)  
      if (used[i]) continue;  
  
      used[i] = true; current.push(nums[i]);  
      backtrack(current);  
      current.pop(); used[i] = false; // 🔑 రెండూ undo  
    }  
  }  
}
```

```

    }
    backtrack([]);
    return res;
}
// O(n! × n)

// Duplicates ఉంటే
function permuteUnique(nums) {
    nums.sort((a, b) => a - b);
    const res = [], used = new Array(nums.length).fill(false);
    function backtrack(current) {
        if (current.length === nums.length) { res.push([...current]); return; }
        for (let i = 0; i < nums.length; i++) {
            if (used[i]) continue;
            // 🔑 ముందు duplicate ఇంకా వాడలేదు → ఇది skip (క్రమం fix చేయడం)
            if (i > 0 && nums[i] === nums[i - 1] && !used[i - 1]) continue;
            used[i] = true; current.push(nums[i]);
            backtrack(current);
            current.pop(); used[i] = false;
        }
    }
    backtrack([]);
    return res;
}

```

Pattern C — Combinations (n నుండి k ఎంచుకోవడం)

```

function combine(n, k) {
    const res = [];
    function backtrack(start, current) {
        if (current.length === k) { res.push([...current]); return; }

        // ✂ PRUNING - మిగిలినవి సరిపోవు అయితే ఆగిపోవడం
        for (let i = start; i <= n - (k - current.length) + 1; i++) {
            current.push(i);
            backtrack(i + 1, current);
            current.pop();
        }
    }
    backtrack(1, []);
    return res;
}

```

Pattern D — Combination Sum (reuse అనుమతి)

```

function combinationSum(candidates, target) {
    const res = [];
    candidates.sort((a, b) => a - b); // pruning కి

    function backtrack(start, current, remaining) {
        if (remaining === 0) { res.push([...current]); return; }

        for (let i = start; i < candidates.length; i++) {

```

```

    if (candidates[i] > remaining) break; // ✂ sorted → ఇక పెద్దవే
    current.push(candidates[i]);
    backtrack(i, current, remaining - candidates[i]); // 🔑 i (i+1 కాదు) → reuse ✅
    current.pop();
  }
}
backtrack(0, [], target);
return res;
}
// 🔑 i vs i+1 - ఒకే తేడా: reuse అనుమతా లేదా

```

📁 Pattern E — Grid backtracking (Word Search)

```

function exist(board, word) {
  const m = board.length, n = board[0].length;

  function backtrack(r, c, idx) {
    if (idx === word.length) return true; // ✅ మొత్తం దొరికింది
    if (r < 0 || r >= m || c < 0 || c >= n || board[r][c] !== word[idx]) return false;

    const temp = board[r][c];
    board[r][c] = "#"; // 🔑 choose (mark visited)

    const found = backtrack(r+1, c, idx+1) || backtrack(r-1, c, idx+1) ||
                  backtrack(r, c+1, idx+1) || backtrack(r, c-1, idx+1);

    board[r][c] = temp; // 🔑 unchoose (restore!) ☆
    return found;
  }

  for (let r = 0; r < m; r++)
    for (let c = 0; c < n; c++)
      if (backtrack(r, c, 0)) return true;
  return false;
}
// ⚠ restore చేయకపోతే - వేరే starting point నుండి ఆ cell వాడలేం → తప్పు సమాధానం

```

📁 Pattern F — Constraint satisfaction (N-Queens)

```

function solveNQueens(n) {
  const res = [];
  const cols = new Set(), diag1 = new Set(), diag2 = new Set();
  const board = Array.from({ length: n }, () => new Array(n).fill("."));

  function backtrack(row) {
    if (row === n) { res.push(board.map((r) => r.join("")); return; }

    for (let col = 0; col < n; col++) {
      // 🔑 O(1) conflict check - diagonals ని గణితంతో గుర్తించడం
      if (cols.has(col) || diag1.has(row - col) || diag2.has(row + col)) continue;

      cols.add(col); diag1.add(row - col); diag2.add(row + col);
      board[row][col] = "Q";
    }
  }
}

```

```

        backtrack(row + 1);

        cols.delete(col); diag1.delete(row - col); diag2.delete(row + col);
        board[row][col] = ".";
    }
}
backtrack(0);
return res;
}
// 🔑 diag1 (row - col) = "\ " దిశ; diag2 (row + col) = "⌵" దిశ - constant per diagonal

```

✂ Pruning — backtracking యొక్క అసలు నైపుణ్యం

Pruning లేకపోతే: $O(2^n)$ / $O(n!)$ — brute force

Pruning తో: ఆచరణలో చాలా వేగం

Pruning మ్యాప్లు:

1. Sort చేసి early break (combination sum: candidate > remaining → break)
2. Constraint **check** ముందే (N-Queens: conflict → skip)
3. Remaining count **check** (combinations: మిగిలినవి సరిపోవా?)
4. Memoization (overlapping states ఉంటే → DP!)
5. Duplicate skip (sorted + i > start check)

ఈ pattern కి చెందిన problems

Medium: Subsets I/II ★ · Permutations I/II · Combinations · Combination Sum I/II/III
 Letter Combinations of a Phone Number · Generate Parentheses ★
 Word Search · Palindrome Partitioning · Restore IP Addresses
 Target Sum · Beautiful Arrangement

Hard: N-Queens ★ · Sudoku Solver · Word Search II (trie + backtracking)
 Permutation Sequence · Remove Invalid Parentheses · Word Break II

Gotchas

- **result.push(current)** (copy లేకుండా) → అన్ని entries ఒకే array reference → చివర్లో అన్నీ ఖాళీ ✨ (#1 backtracking bug).
- **Unchoose మర్చిపోవడం** → state leak → తప్పు సమాధానాలు.
- **i vs i + 1 vs 0** — combinations (i+1), reuse (i), permutations (0 + used array).
- **Duplicates handling** — sort + i > start && nums[i] == nums[i-1] .
- **Grid లో restore చేయకపోవడం** (board[r][c] = temp).
- **Pruning లేకపోవడం** → TLE.
- **Base case లో return మర్చిపోవడం** → అనవసర recursion.

Interview దృష్టి

- "Backtracking template చెప్పు" → choose/explore/unchoose + base case + pruning.

- "Subsets vs permutations తేడా?" → order matters (permutations), **start** index vs used array.
- "Complexity?" → subsets $O(2^n \times n)$, permutations $O(n! \times n)$, N-Queens $O(n!)$.
- "Duplicates ఎలా handle?" → sort + same-level skip — ఎందుకు $i > \text{start}$ అని వివరించగలగాలి.

29. Divide & Conquer

🔍 Recognition signals

- ✓ Problem ని రెండు (లేదా ఎక్కువ) స్వతంత్ర ఉప-problems గా విడగొట్టొచ్చు
- ✓ "sorted array" + $O(\log n)$ / $O(n \log n)$
- ✓ Merge sort, quick sort variants
- ✓ "count pairs/inversions"
- ✓ Tree recursion (ఇది కూడా D&C)

📄 Template

```
function divideAndConquer(problem) {
  if (isBaseCase(problem)) return solveDirectly(problem); // 1 DIVIDE ఆగే చోటు

  const [left, right] = divide(problem); // 2 DIVIDE
  const leftResult = divideAndConquer(left); // 3 CONQUER
  const rightResult = divideAndConquer(right);

  return combine(leftResult, rightResult); // 4 COMBINE ★
}
```

Real-life Scenario

Divide & conquer = పెద్ద పని ని team కి పంచడం. 1000 పేజీల document proofread చేయాలి — ఇద్దరికీ 500 చొప్పున, వాళ్ళు మళ్ళీ నలుగురికి 250 చొప్పున... చివరికి **ప్రతి ఒక్కరూ 1 పేజీ** (base case). తర్వాత **ఫలితాలను కలిపి పైకి తీసుకురావడం** (combine) — ఇదే అసలు నైపుణ్యం.

ఉదాహరణ 1 — Merge Sort

```
function mergeSort(arr) {
  if (arr.length <= 1) return arr; // base
  const mid = arr.length >> 1;
  const left = mergeSort(arr.slice(0, mid)); // divide + conquer
  const right = mergeSort(arr.slice(mid));
  return merge(left, right); // combine
}

function merge(a, b) {
  const res = [];
  let i = 0, j = 0;
```

```

while (i < a.length && j < b.length) res.push(a[i] <= b[j] ? a[i++] : b[j++]);
return [...res, ...a.slice(i), ...b.slice(j)];
}
// O(n log n) time, O(n) space · STABLE ✅

```

ఉదాహరణ 2 — Quick Sort

```

function quickSort(arr, left = 0, right = arr.length - 1) {
  if (left >= right) return arr;
  const pivotIdx = partition(arr, left, right);
  quickSort(arr, left, pivotIdx - 1);
  quickSort(arr, pivotIdx + 1, right);
  return arr;
}
// O(n log n) average, O(n²) worst (sorted array + first/last pivot)
// 🔑 Random pivot లేదా median-of-three తో worst case నివారణ

```

ఉదాహరణ 3 — Binary Tree recursion (ఇది కూడా D&C)

```

function maxDepth(root) {
  if (!root) return 0; // base
  const left = maxDepth(root.left), right = maxDepth(root.right); // divide+conquer
  return 1 + Math.max(left, right); // combine
}

```

ఉదాహరణ 4 — Different Ways to Add Parentheses

```

function diffWaysToCompute(expression) {
  const res = [];
  for (let i = 0; i < expression.length; i++) {
    const ch = expression[i];
    if (ch === "+" || ch === "-" || ch === "*") {
      // 🔑 ప్రతి operator దగ్గర విభజించడం
      const left = diffWaysToCompute(expression.slice(0, i));
      const right = diffWaysToCompute(expression.slice(i + 1));

      for (const l of left)
        for (const r of right)
          res.push(ch === "+" ? l + r : ch === "-" ? l - r : l * r);
    }
  }
  return res.length ? res : [Number(expression)]; // base: pure number
}
// 🔑 Memoization జోడిస్తే భారీ improvement (overlapping subproblems)

```

Master Theorem (complexity analysis)

$$T(n) = a \cdot T(n/b) + O(n^d)$$

a = ఎన్ని subproblems · b = ఎంత విభజన · d = combine cost

```

if d > log_b(a) → O(n^d) [combine dominates]
if d = log_b(a) → O(n^d · log n) [balanced] ← merge sort: a=2, b=2, d=1 → O(n log n)
if d < log_b(a) → O(n^log_b(a)) [subproblems dominate]

```

Binary search: a=1, b=2, d=0 → O(log n)

Gotchas

- **Base case లేకపోవడం/తప్పు** → infinite recursion.
- **Combine step తప్పు** — divide సులభం, **combine లోనే అసలు logic**.
- **slice() వాడటం** → O(n) copy ప్రతి level లో → space పెరుగుదల (indices వాడితే in-place).
- **Overlapping subproblems ఉంటే D&C వాడటం** → అది **DP** కావాలి (memoization).
- **Quick sort worst case** — sorted input + naive pivot.

Interview దృష్టి

- "Merge sort vs quick sort?" → stability, space (O(n) vs O(log n)), worst case, cache locality, in-place.
- "D&C vs DP?" → subproblems **independent** (D&C) vs **overlapping** (DP → memoize).

30. Recursion → Memoization Transformation ★

🔍 ఎప్పుడు — DP కి ప్రవేశ ద్వారం

- ✓ Recursion రాశాను, కానీ TLE వస్తోంది
- ✓ Recursion tree లో ఒకే subproblem పదే పదే కనిపిస్తోంది
- ✓ "count ways" / "min/max" + choices

వివరణ — ప్రతి DP problem ఇలా మొదలవుతుంది

- 1 Brute force recursion రాయి (సరైనది, నెమ్మది అయినా)
- 2 Overlapping subproblems గుర్తించు (ఒకే arguments తో పదే పదే call)
- 3 Memo జోడించు (top-down DP) ✓ — ఇక్కడితో interview లో సరిపోతుంది
- 4 (ఐచ్ఛికం) Tabulation కి మార్పు (bottom-up)
- 5 (ఐచ్ఛికం) Space optimize చేయి

Interview మ్యాహం: recursion → memo వరకు చేస్తే చాలా సందర్భాల్లో సరిపోతుంది. Tabulation ని "I can convert this to bottom-up if needed" అని చెప్పి, సమయం ఉంటే చేయండి.

📄 Transformation ఉదాహరణ (step-by-step)

```

// 1 Brute force — O(2^n) 🐢
function fib(n) {

```

```

    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
// fib(5) recursion tree:
//      fib(5)
//      /      \
//    fib(4)    fib(3)    ← fib(3) రెండుసార్లు!
//   /  \    /  \
// fib(3) fib(2) fib(2) fib(1) ← fib(2) మూడుసార్లు!

// 2 Memoization (top-down DP) - O(n) ✓
function fibMemo(n, memo = new Map()) {
    if (n <= 1) return n;
    if (memo.has(n)) return memo.get(n); // 🔑 cache hit
    const result = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
    memo.set(n, result); // 🔑 cache store
    return result;
}

// 3 Tabulation (bottom-up) - O(n)
function fibTab(n) {
    if (n <= 1) return n;
    const dp = new Array(n + 1);
    dp[0] = 0; dp[1] = 1;
    for (let i = 2; i <= n; i++) dp[i] = dp[i - 1] + dp[i - 2];
    return dp[n];
}

// 4 Space optimized - O(1) ★
function fibOpt(n) {
    if (n <= 1) return n;
    let prev2 = 0, prev1 = 1;
    for (let i = 2; i <= n; i++) { const cur = prev1 + prev2; prev2 = prev1; prev1 = cur; }
    return prev1;
}

```

📁 Universal Memoization Template

```

function solve(input) {
    const memo = new Map();

    function dp(...state) {
        // 1 Base cases
        if (isBase(state)) return baseValue;

        // 2 Memo check (🔑 key = అన్ని state variables) // లేదా nested Map
        const key = state.join(",");
        if (memo.has(key)) return memo.get(key);

        // 3 Recurrence - అన్ని choices try చేయడం // -Infinity / Infinity / 0
        let result = worstValue;
        for (const choice of choices) {
            result = better(result, dp(...nextState(state, choice)));
        }
    }
}

```

```
// 4 Store & return
memo.set(key, result);
return result;
}
return dp(initialState);
}
```

వాస్తవిక ఉదాహరణ — Coin Change

```
// 1 Brute force
function coinChangeBrute(coins, amount) {
  if (amount === 0) return 0;
  if (amount < 0) return Infinity;
  let min = Infinity;
  for (const coin of coins) min = Math.min(min, 1 + coinChangeBrute(coins, amount - coin));
  return min;
}
// O(coins^amount) 🧠

// 2 Memoized ✅
function coinChange(coins, amount) {
  const memo = new Map();
  function dp(rem) {
    if (rem === 0) return 0;
    if (rem < 0) return Infinity;
    if (memo.has(rem)) return memo.get(rem);

    let min = Infinity;
    for (const coin of coins) min = Math.min(min, 1 + dp(rem - coin));

    memo.set(rem, min);
    return min;
  }
  const res = dp(amount);
  return res === Infinity ? -1 : res;
}
// O(amount × coins) ✅
```

🔑 Memo key design (అత్యంత సాధారణ bug source)

```
// ⚠️ Key లో **అన్ని** state variables ఉండాలి
function dp(i, remaining, canUse) { ... }
const key = `${i},${remaining},${canUse}`; // ✅ మూడూ

// ❌ ఒకటి మర్చిపోతే - తప్పు cache hits → తప్పు సమాధానం (silent bug!)
const key = `${i}`; // ❌ remaining మిస్

// 2D array (indices అయితే వేగం)
const memo = Array.from({ length: m }, () => new Array(n).fill(-1));
if (memo[i][j] !== -1) return memo[i][j];
```

Memoization vs Tabulation

	Top-down (memo)	Bottom-up (tabulation)
రాయడం	సులభం (recursion + 2 lines) ✓	order ఆలోచించాలి
Subproblems	అవసరమైనవి మాత్రమే ✓	అన్నీ
Stack overflow	⚠ ప్రమాదం	✓ లేదు
Space optimization	కష్టం	✓ సులభం (rolling array)
Interview లో	✓ మొదట ఇది	సమయం ఉంటే

Gotchas

- **Memo key అసంపూర్ణం** → తప్పు సమాధానం (debug కష్టం).
- **Mutable state ని key లో** – arrays/objects → serialize చేయాలి.
- `memo.has()` బదులు `memo.get() !== undefined` – value `0` / `undefined` అయితే bug.
- **Overlapping లేకుండా memoize చేయడం** → అనవసర memory (D&C సరిపోతుంది).
- **Deep recursion** → stack overflow ($n \geq 10^5$) → tabulation.
- **Memo ని function వెలుపల (global)** → multiple test cases మధ్య stale data.

Interview దృష్టి

- "DP problem ఎలా approach చేస్తావు?" → "brute force recursion రాస్తాను, overlapping subproblems చూపిస్తాను, memo జోడిస్తాను, తర్వాత tabulation/space optimize" – ఈ 4-step answer చెప్తే DP interview సగం గెలిచినట్టే.
 - "Memoization vs DP?" → memoization = top-down DP; రెండూ DP నే.
-

Part 8 — Dynamic Programming

31. DP Recognition & State Design Framework ★★

🔍 DP అని ఎలా గుర్తించాలి — 3 సంకేతాలు

1 OPTIMAL SUBSTRUCTURE

"పెద్ద problem యొక్క సమాధానం, చిన్న problems సమాధానాల నుండి కట్టవచ్చు"

2 OVERLAPPING SUBPROBLEMS

"ఒకే subproblem పదే పదే వస్తోంది" (లేకపోతే అది D&C)

3 కీలక పదాలు

"maximum/minimum ..." · "count the number of ways" · "is it possible to ..."

+ ప్రతి step లో **choices** (తీసుకోవాలా వద్దా, ఏది తీసుకోవాలి)

✗ DP కాదు అని ఎలా తెలుసుకోవాలి

- Greedy సరిపోతుంది (local optimal → global optimal నిరూపించగలిగితే)
- "all solutions" కావాలి → backtracking (DP counts/optimizes, enumerate చేయదు)
- Subproblems overlap కావు → D&C

🔑 State Design Framework (DP యొక్క అసలు నైపుణ్యం)

DP problem solve చేయడం = సరైన "state" నిర్వచించడం. Code రాయడం సులభం; state design కష్టం.

ప్రశ్న 1: "ఒక subproblem ని uniquely గుర్తించడానికి నాకు ఏ సమాచారం కావాలి?"
→ అవే మీ state variables

ప్రశ్న 2: "dp[state] అంటే ఖచ్చితంగా ఏమిటి?"
→ ఒక్క వాక్యంలో రాయండి (ఇది స్పష్టంగా లేకపోతే code తప్పు అవుతుంది)

ప్రశ్న 3: "ఈ state కి రావడానికి ముందు ఏం జరిగి ఉండాలి?"
→ అదే recurrence relation

ప్రశ్న 4: "అతి చిన్న subproblem ఏమిటి?"
→ base cases

📅 ఉదాహరణ — framework ని ప్రయోగించడం

Problem: House Robber — పక్కపక్క ఇళ్ళు దోచుకోలేం, గరిష్ట మొత్తం ఎంత?

1. State: "నేను ఏ ఇంటి దగ్గర ఉన్నాను?" → i
2. $dp[i]$ = "మొదటి i ఇళ్ళలో దోచుకోగల గరిష్ట మొత్తం"
3. Recurrence: ఈ ఇల్లు దోచుకోవాలా వద్దా?
 $dp[i] = \max(dp[i-1], \quad // \text{ వద్దు} \rightarrow \text{ముందుది}$

`dp[i-2] + nums[i]) // అవును → ఒకటి వదిలి + ఇది`

4. Base: `dp[0] = nums[0], dp[1] = max(nums[0], nums[1])`

DP రకాలు — state dimensions ప్రకారం

Dimensions	State	ఉదాహరణలు
1D	<code>dp[i]</code>	Fibonacci, House Robber, Climbing Stairs, LIS, Decode Ways
2D (grid)	<code>dp[i][j]</code>	Unique Paths, Minimum Path Sum, Maximal Square
2D (strings)	<code>dp[i][j]</code>	LCS, Edit Distance, Palindromes
2D (knapsack)	<code>dp[i][capacity]</code>	0/1 Knapsack, Subset Sum, Partition
1D + state	<code>dp[i][state]</code>	Stock problems (holding/not), Paint House
Interval	<code>dp[i][j]</code> (range)	Burst Balloons, Matrix Chain
Bitmask	<code>dp[mask]</code>	TSP, assignment ($n \leq 20$)
Tree	<code>dp[node][state]</code>	House Robber III, Tree diameter

DP problem solve చేసే క్రమం (interview లో ఇలా మాట్లాడాలి)

- "ఇది DP అనిపిస్తోంది ఎందుకంటే – ప్రతి step లో choices ఉన్నాయి, overlapping subproblems ఉన్నాయి"
- "State ని ఇలా నిర్వచిస్తాను: `dp[i] = ...`"
- "Recurrence: `dp[i] = ...` ఎందుకంటే ..."
- "Base cases: ..."
- "Brute force recursion రాస్తాను" → code
- "ఇక్కడ overlapping ఉంది → memo జోడిస్తాను" → code
- "Bottom-up కి మార్చొచ్చు, space $O(n) \rightarrow O(1)$ చేయొచ్చు"
- Complexity: $O(\text{states} \times \text{transitions})$

Complexity formula

DP Time = (states సంఖ్య) × (ప్రతి state కి transitions)

ఉదా: Coin Change → states = amount, transitions = coins.length → $O(\text{amount} \times \text{coins})$

Edit Distance → states = $m \times n$, transitions = 3 → $O(m \times n)$

LIS (naive) → states = n , transitions = n → $O(n^2)$

TSP → states = $2^n \times n$, transitions = n → $O(2^n \times n^2)$

Key Points

- DP = optimal substructure + overlapping subproblems.
- State design = అసలు పని; `dp[state]` అథాన్ని ఒక్క వాక్యంలో రాయగలగాలి.
- Recursion → memo → tabulation → space optimize (ఈ క్రమంలో).
- Time = states × transitions.

Interview దృష్టి

- "ఇది DP అని ఎలా తెలిసింది?" → 3 సంకేతాలు (choices, optimal substructure, overlapping).
- "State ఏమిటి?" → స్పష్టంగా నిర్వచించడం – ఇది సరిగ్గా చెప్పకపోతే మిగతాదంతా తప్పు.

32. 1D DP

🔍 Recognition signals

- ✓ ఒకే **sequence** (array/string) మీద
- ✓ ప్రతి **index** దగ్గర binary/limited choice
- ✓ `dp[i] = "మొదటి i elements వరకు సమాధానం"`

Family 1 — Fibonacci style (దారులు లెక్కించడం)

```
// Climbing Stairs – n మెట్లు, ఒకసారి 1 లేదా 2 మెట్లు
function climbStairs(n) {
  let prev2 = 1, prev1 = 1;
  for (let i = 2; i <= n; i++) { const cur = prev1 + prev2; prev2 = prev1; prev1 = cur; }
  return prev1;
}

// dp[i] = dp[i-1] + dp[i-2] – "చివరి అడుగు 1 మెట్లు, 2 మెట్లు?"

// Min Cost Climbing Stairs
function minCostClimbingStairs(cost) {
  let prev2 = 0, prev1 = 0;
  for (let i = 2; i <= cost.length; i++) {
    const cur = Math.min(prev1 + cost[i-1], prev2 + cost[i-2]);
    prev2 = prev1; prev1 = cur;
  }
  return prev1;
}
```

Family 2 — House Robber (adjacent constraint) ★

```
function rob(nums) {
  let prev2 = 0, prev1 = 0; // dp[i-2], dp[i-1]
  for (const num of nums) {
    const cur = Math.max(prev1, prev2 + num); // 🔑 skip vs take
    prev2 = prev1; prev1 = cur;
  }
  return prev1;
}

// Circular (House Robber II) – మొదటిది, చివరిది పక్కపక్కనే
function robCircular(nums) {
  if (nums.length === 1) return nums[0];
  // 🔑 రెండు cases: మొదటిది తీసుకో (చివరిది వద్దు) లేదా మొదటిది వద్దు
  return Math.max(rob(nums.slice(0, -1)), rob(nums.slice(1)));
}
```

```

}

// Tree (House Robber III) – bottom-up tree DP
function robTree(root) {
  function dfs(node) {
    if (!node) return [0, 0]; // [rob this, don't rob this]
    const [lRob, lSkip] = dfs(node.left);
    const [rRob, rSkip] = dfs(node.right);
    return [
      node.val + lSkip + rSkip, // ఇది దోచుకుంటే – పిల్లలు వద్దు
      Math.max(lRob, lSkip) + Math.max(rRob, rSkip), // వద్దంటే – పిల్లలు ఇష్టం
    ];
  }
  return Math.max(...dfs(root));
}

```

Family 3 – Decode Ways (string + validity)

```

function numDecodings(s) {
  if (s[0] === "0") return 0;
  let prev2 = 1, prev1 = 1;

  for (let i = 1; i < s.length; i++) {
    let cur = 0;
    if (s[i] !== "0") cur += prev1; // ఒక్క digit ("1"-"9")
    const two = Number(s.slice(i - 1, i + 1));
    if (two >= 10 && two <= 26) cur += prev2; // రెండు digits ("10"-"26")
    if (cur === 0) return 0; // 🚩 invalid string
    prev2 = prev1; prev1 = cur;
  }
  return prev1;
}

```

Family 4 – Jump Game (reachability)

```

function canJump(nums) {
  let maxReach = 0;
  for (let i = 0; i < nums.length; i++) {
    if (i > maxReach) return false; // ఇక్కడికి చేరలేం
    maxReach = Math.max(maxReach, i + nums[i]);
  }
  return true;
}

// 🗝️ ఇది greedy – DP అవసరం లేదు (Pattern 38)

function jump(nums) {
  let jumps = 0, curEnd = 0, farthest = 0; // కనీస jumps
  for (let i = 0; i < nums.length - 1; i++) {
    farthest = Math.max(farthest, i + nums[i]);
    if (i === curEnd) { jumps++; curEnd = farthest; } // 🗝️ level boundary (BFS లాంటిది)
  }
}

```

```
    return jumps;
}
```

Family 5 — Word Break (dictionary DP)

```
function wordBreak(s, wordDict) {
    const words = new Set(wordDict);
    const dp = new Array(s.length + 1).fill(false);
    dp[0] = true;                                     // ఖాళీ string valid

    for (let i = 1; i <= s.length; i++) {
        for (let j = 0; j < i; j++) {
            if (dp[j] && words.has(s.slice(j, i))) { dp[i] = true; break; }
        }
    }
    return dp[s.length];
}
// dp[i] = "మొదటి i characters ని words గా విడగొట్టవచ్చా?"
// O(n^2 * wordLength)
```

ఈ pattern కి చెందిన problems

Easy: [Climbing Stairs](#) · [Min Cost Climbing Stairs](#) · [Fibonacci](#) · [Best Time to Buy/Sell Stock](#)
Medium: [House Robber I/II/III](#) ★ · [Decode Ways](#) · [Word Break](#) · [Jump Game I/II](#)
[Longest Increasing Subsequence](#) · [Coin Change](#) · [Perfect Squares](#)
[Delete and Earn](#) · [Paint House](#) · [Maximum Product Subarray](#)
Hard: [Best Time to Buy and Sell Stock III/IV](#) · [Frog Jump](#) · [Russian Doll Envelopes](#)

Gotchas

- **Base cases తప్పు** — `dp[0]` , `dp[1]` (ఇక్కడే 90% bugs).
- **Space optimize చేసేటప్పుడు update order** — `prev2 = prev1` ముందా తర్వాతా.
- **Circular constraint** (House Robber II) — రెండు cases.
- **Decode Ways లో "0" handling** — "0" , "10" , "06" edge cases.
- **Off-by-one** — `dp` size `n` అ `n+1` అ (ఒకే convention పాటించాలి).

33. 2D DP — Grid Paths

🔍 Recognition signals

- ✓ 2D grid + "paths" / "minimum cost" / "maximum"
- ✓ కదలిక పరిమితం (కుడికి/కిందికి మాత్రమే)
- ✓ `dp[i][j] = "(i,j) కి చేరే సమాధానం"`

Template

```
function uniquePaths(m, n) {
  const dp = Array.from({ length: m }, () => new Array(n).fill(1)); // మొదటి row/col = 1

  for (let i = 1; i < m; i++)
    for (let j = 1; j < n; j++)
      dp[i][j] = dp[i-1][j] + dp[i][j-1]; // 🔑 పైననుండి + ఎడమనుండి

  return dp[m-1][n-1];
}
// O(m×n)

// ⚡ Space optimized - O(n)
function uniquePathsOpt(m, n) {
  const dp = new Array(n).fill(1);
  for (let i = 1; i < m; i++)
    for (let j = 1; j < n; j++)
      dp[j] += dp[j - 1]; // 🔑 dp[j] = పైన (పాతది), dp[j-1] = ఎడమ
  return dp[n - 1];
}
```

Variants

```
// 1. Obstacles 🏠
function uniquePathsWithObstacles(grid) {
  const m = grid.length, n = grid[0].length;
  if (grid[0][0] === 1) return 0;
  const dp = Array.from({ length: m }, () => new Array(n).fill(0));
  dp[0][0] = 1;

  for (let i = 0; i < m; i++)
    for (let j = 0; j < n; j++) {
      if (grid[i][j] === 1) { dp[i][j] = 0; continue; } // 🔑 obstacle
      if (i > 0) dp[i][j] += dp[i-1][j];
      if (j > 0) dp[i][j] += dp[i][j-1];
    }
  return dp[m-1][n-1];
}

// 2. Minimum path sum
function minPathSum(grid) {
  const m = grid.length, n = grid[0].length;
  const dp = Array.from({ length: m }, () => new Array(n).fill(0));
  dp[0][0] = grid[0][0];
  for (let i = 1; i < m; i++) dp[i][0] = dp[i-1][0] + grid[i][0];
  for (let j = 1; j < n; j++) dp[0][j] = dp[0][j-1] + grid[0][j];

  for (let i = 1; i < m; i++)
    for (let j = 1; j < n; j++)
      dp[i][j] = grid[i][j] + Math.min(dp[i-1][j], dp[i][j-1]);
  return dp[m-1][n-1];
}
```

```
// 3. Maximal Square (subproblem definition తెలివైనది ✨)
function maximalSquare(matrix) {
  const m = matrix.length, n = matrix[0].length;
  const dp = Array.from({ length: m + 1 }, () => new Array(n + 1).fill(0));
  let maxSide = 0;

  for (let i = 1; i <= m; i++)
    for (let j = 1; j <= n; j++) {
      if (matrix[i-1][j-1] === "1") {
        // ✨ dp[i][j] = "(i,j) కుడి-కింది మూలగా ఉన్న అతిపెద్ద square భుజం"
        dp[i][j] = 1 + Math.min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]);
        maxSide = Math.max(maxSide, dp[i][j]);
      }
    }
  return maxSide * maxSide;
}

// ✨ మూడు పొరుగువాళ్ళలో అతి చిన్నది + 1 - ఎందుకంటే square కి మూడు వైపులా support కావాలి

// 4. Triangle (bottom-up మేలు)
function minimumTotal(triangle) {
  const dp = [...triangle[triangle.length - 1]];
  for (let i = triangle.length - 2; i >= 0; i--)
    for (let j = 0; j <= i; j++)
      dp[j] = triangle[i][j] + Math.min(dp[j], dp[j + 1]);
  return dp[0];
}
```

Gotchas

- మొదటి row/column initialization మర్చిపోవడం.
- Obstacles ని dp లో 0 చేయకపోవడం.
- Space optimization లో update order — $dp[j] += dp[j-1]$ (ఎడమ నుండి కుడికి).
- Maximal Square యొక్క state definition — "ఇక్కడ ముగిసే square" అని స్పష్టంగా.

34. String DP (2D)

🔍 Recognition signals

- ✓ రెండు strings + "common" / "edit" / "transform" / "match"
- ✓ ఒక string + "palindrome" / "partition"
- ✓ $dp[i][j]$ = "s1 యొక్క మొదటి i, s2 యొక్క మొదటి j characters"

📄 Template 1 — LCS (Longest Common Subsequence) ✨ base pattern

```
function longestCommonSubsequence(text1, text2) {
  const m = text1.length, n = text2.length;
  const dp = Array.from({ length: m + 1 }, () => new Array(n + 1).fill(0));
```

```

for (let i = 1; i <= m; i++)
  for (let j = 1; j <= n; j++) {
    if (text1[i-1] === text2[j-1]) dp[i][j] = 1 + dp[i-1][j-1]; // 🔑 match → diagonal
    else dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]); // 🔑 ఒకటి skip
  }
return dp[m][n];
}
// O(m×n) – string DP అన్నిటికీ ఇదే ఆధారం

```

📄 Template 2 — Edit Distance (Levenshtein) ★

```

function minDistance(word1, word2) {
  const m = word1.length, n = word2.length;
  const dp = Array.from({ length: m + 1 }, () => new Array(n + 1).fill(0));

  for (let i = 0; i <= m; i++) dp[i][0] = i; // అన్నీ delete
  for (let j = 0; j <= n; j++) dp[0][j] = j; // అన్నీ insert

  for (let i = 1; i <= m; i++)
    for (let j = 1; j <= n; j++) {
      if (word1[i-1] === word2[j-1]) dp[i][j] = dp[i-1][j-1]; // ఖచ్చితం లేదు
      else dp[i][j] = 1 + Math.min(
        dp[i-1][j-1], // REPLACE
        dp[i-1][j], // DELETE (word1 నుండి)
        dp[i][j-1] // INSERT (word1 లోకి)
      );
    }
  return dp[m][n];
}
// 🔑 మూడు operations = మూడు దిశలు (diagonal/పైన/ఎడమ) – ఇది గుర్తుపెట్టుకుంటే code గుర్తుంటుంది

```

📄 Template 3 — Palindrome DP

```

// Longest Palindromic Substring – expand around center (DP కంటే సులభం + O(1) space)
function longestPalindrome(s) {
  let start = 0, maxLen = 1;
  function expand(l, r) {
    while (l >= 0 && r < s.length && s[l] === s[r]) { l--; r++; }
    const len = r - l - 1;
    if (len > maxLen) { maxLen = len; start = l + 1; }
  }
  for (let i = 0; i < s.length; i++) { expand(i, i); expand(i, i + 1); } // odd, even
  return s.slice(start, start + maxLen);
}
// O(n²) time, O(1) space ✅ (DP O(n²) space)

// Longest Palindromic Subsequence – LCS(s, reverse(s))
function longestPalindromeSubseq(s) {
  return longestCommonSubsequence(s, [...s].reverse().join(''));
}
// 🔑 తెలివైన reduction ★

```

```
// Palindromic Substrings (count)
function countSubstrings(s) {
  let count = 0;
  function expand(l, r) {
    while (l >= 0 && r < s.length && s[l] === s[r]) { count++; l--; r++; }
  }
  for (let i = 0; i < s.length; i++) { expand(i, i); expand(i, i + 1); }
  return count;
}
```

Template 4 — Regex / Wildcard Matching (Hard)

```
function isMatch(s, p) { // '.' = ఏ character, '*' = ముందుది 0+ సార్లు
  const m = s.length, n = p.length;
  const dp = Array.from({ length: m + 1 }, () => new Array(n + 1).fill(false));
  dp[0][0] = true;

  for (let j = 1; j <= n; j++) // ఖాళీ string కి pattern match అవుతుందా
    if (p[j-1] === "*") dp[0][j] = dp[0][j-2];

  for (let i = 1; i <= m; i++)
    for (let j = 1; j <= n; j++) {
      if (p[j-1] === "*") {
        dp[i][j] = dp[i][j-2]; // 0 సార్లు
        if (p[j-2] === "." || p[j-2] === s[i-1]) dp[i][j] |= dp[i-1][j]; // 1+ సార్లు
      } else if (p[j-1] === "." || p[j-1] === s[i-1]) {
        dp[i][j] = dp[i-1][j-1];
      }
    }
  return dp[m][n];
}
```

ఈ pattern కి చెందిన problems

Medium: [Longest Common Subsequence](#) ★ · [Longest Palindromic Substring/Subsequence](#)
[Palindromic Substrings](#) · [Delete Operation for Two Strings](#)
[Minimum ASCII Delete Sum](#) · [Interleaving String](#) · [Decode Ways](#)
[Longest Repeating Subsequence](#)

Hard: [Edit Distance](#) ★ · [Regular Expression Matching](#) · [Wildcard Matching](#)
[Distinct Subsequences](#) · [Shortest Common Supersequence](#) · [Palindrome Partitioning II](#)

Gotchas

- **Index off-by-one** — $dp[i][j]$ కి $s[i-1]$, $s[j-1]$ (dp 1-indexed, string 0-indexed).
- **Base row/column** — edit distance లో $dp[i][0] = i$ మర్చిపోవడం.
- **Substring vs subsequence** గందరగోళం — substring contiguous!
- **Space optimization** — 2 rows చాలు (కానీ ముందు correctness).

Interview దృష్టి

- "LCS recurrence వివరించు" → match → diagonal+1; mismatch → max(skip either).
- "Longest palindromic subsequence?" → LCS(s, reverse(s)) – ఈ reduction చెప్తే impressive.

35. Knapsack Family

🔍 Recognition signals

- ✅ "items with weights/values" + "capacity/budget/limit"
- ✅ "can we **select** items **to** reach exactly X?"
- ✅ "**partition into** equal subsets"
- ✅ "minimum coins / maximum **value**"

📁 0/1 Knapsack (ప్రతి item ఒక్కసారే)

```
function knapsack(weights, values, capacity) {
  const n = weights.length;
  const dp = Array.from({ length: n + 1 }, () => new Array(capacity + 1).fill(0));

  for (let i = 1; i <= n; i++)
    for (let w = 0; w <= capacity; w++) {
      dp[i][w] = dp[i-1][w]; // 🔑 ఈ item వద్దు
      if (weights[i-1] <= w) {
        dp[i][w] = Math.max(dp[i][w], values[i-1] + dp[i-1][w - weights[i-1]]); // 🔑 తీసుకో
      }
    }
  return dp[n][capacity];
}
// O(n × capacity)

// ⚡ Space optimized – 1D (⚠️ capacity ని REVERSE order లో!)
function knapsack1D(weights, values, capacity) {
  const dp = new Array(capacity + 1).fill(0);
  for (let i = 0; i < weights.length; i++)
    for (let w = capacity; w >= weights[i]; w--) { // 🔑 REVERSE! (ప్రతి item ఒక్కసారే)
      dp[w] = Math.max(dp[w], values[i] + dp[w - weights[i]]);
    }
  return dp[capacity];
}
```

★ **Reverse order ఎందుకు (0/1 knapsack లో):** forward order అయితే dp[w - weight] ఇప్పటికే ఈ item తో update అయి ఉంటుంది → ఒకే item రెండుసార్లు వాడేస్తాం (అది unbounded knapsack). ఇది interview లో favourite question.

📦 Unbounded Knapsack (items పరిమితం లేదు)

```
function unboundedKnapsack(weights, values, capacity) {
  const dp = new Array(capacity + 1).fill(0);
  for (let w = 1; w <= capacity; w++)
    for (let i = 0; i < weights.length; i++)
      if (weights[i] <= w) dp[w] = Math.max(dp[w], values[i] + dp[w - weights[i]]);
  return dp[capacity];
}
// 🔑 FORWARD order → reuse అనుమతిస్తుంది
```

Variants

```
// 1. Subset Sum – target sum సాధ్యమా?
function canPartitionSubset(nums, target) {
  const dp = new Array(target + 1).fill(false);
  dp[0] = true;
  for (const num of nums)
    for (let t = target; t >= num; t--) dp[t] ||= dp[t - num]; // reverse (0/1)
  return dp[target];
}

// 2. Partition Equal Subset Sum
function canPartition(nums) {
  const sum = nums.reduce((a, b) => a + b, 0);
  if (sum % 2 !== 0) return false; // ✂ odd → అసాధ్యం
  return canPartitionSubset(nums, sum / 2);
}

// 3. Coin Change – కనీస coins (unbounded)
function coinChange(coins, amount) {
  const dp = new Array(amount + 1).fill(Infinity);
  dp[0] = 0;
  for (let a = 1; a <= amount; a++)
    for (const coin of coins)
      if (coin <= a) dp[a] = Math.min(dp[a], 1 + dp[a - coin]);
  return dp[amount] === Infinity ? -1 : dp[amount];
}

// 4. Coin Change II – ఎన్ని ways (⚠ loop order కీలకం!)
function change(amount, coins) {
  const dp = new Array(amount + 1).fill(0);
  dp[0] = 1;
  for (const coin of coins) // 🔑 coins బయటి loop → combinations
    for (let a = coin; a <= amount; a++) dp[a] += dp[a - coin];
  return dp[amount];
}
// ⚠ Loops తారుమారు చేస్తే → permutations (క్రమం ముఖ్యం) లెక్కిస్తుంది – వేరే సమాధానం!

// 5. Target Sum (+/- signs) – subset sum గా మార్చడం
function findTargetSumWays(nums, target) {
  const sum = nums.reduce((a, b) => a + b, 0);
  if ((sum + target) % 2 !== 0 || Math.abs(target) > sum) return 0;
  const subsetSum = (sum + target) / 2; // 🔑 గణిత reduction
```

```
const dp = new Array(subsetSum + 1).fill(0);
dp[0] = 1;
for (const num of nums)
  for (let s = subsetSum; s >= num; s--) dp[s] += dp[s - num];
return dp[subsetSum];
}
```

Target Sum reduction: $P - N = \text{target}$, $P + N = \text{sum} \rightarrow P = (\text{sum} + \text{target}) / 2$. ఇలా "+/- signs" problem ని subset sum గా మార్చడం – **classic interview insight**.

★ Loop order cheat-sheet (అత్యంత గందరగోళం)

0/1 knapsack (ప్రతి item ఒక్కసారే):	capacity REVERSE
Unbounded (reuse అనుమతి):	capacity FORWARD
Combinations (క్రమం ముఖ్యం కాదు):	items బయట, capacity లోపల
Permutations (క్రమం ముఖ్యం):	capacity బయట, items లోపల

ఈ pattern కి చెందిన problems

Medium: Partition Equal Subset Sum ★ · Coin Change I/II · Target Sum
 Ones and Zeroes · Last Stone Weight II · Combination Sum IV (permutations!)
 Perfect Squares · Minimum Cost For Tickets
 Hard: Profitable Schemes · Number of Ways to Earn Points

Gotchas

- **Loop order తప్పు** → 0/1 vs unbounded మారిపోతుంది (**#1 knapsack bug**).
- **Coin Change II లో loops తారుమారు** → permutations లెక్కిస్తుంది.
- **dp[0] = 1** (counting) vs **dp[0] = 0** (minimizing) గందరగోళం.
- **Odd sum check** మర్చిపోవడం (partition).
- **Infinity handling** – 1 + Infinity overflow.

Interview దృష్టి

- "0/1 knapsack లో reverse loop ఎందుకు?" → item reuse నివారణ – ఈ ఒక్క ప్రశ్న DP అర్థమైందా అని పరీక్షిస్తుంది.
- "Combinations vs permutations?" → loop order.

36. LIS Family

🔍 Recognition signals

- ✓ "longest increasing subsequence" (subsequence – contiguous కాదు!)
- ✓ "maximum chain/envelopes/dolls"

✅ ఒక dimension sort చేసి రెండో దానిపై LIS

📄 Template 1 — $O(n^2)$ DP

```
function lengthOfLIS(nums) {  
  const dp = new Array(nums.length).fill(1); // dp[i] = "i తో ముగిసే LIS పొడవు"  
  let max = 1;  
  
  for (let i = 1; i < nums.length; i++)  
    for (let j = 0; j < i; j++)  
      if (nums[j] < nums[i]) {  
        dp[i] = Math.max(dp[i], dp[j] + 1);  
        max = Math.max(max, dp[i]);  
      }  
  return nums.length ? max : 0;  
}  
//  $O(n^2)$ 
```

📄 Template 2 — $O(n \log n)$ (patience sorting) ★

```
function lengthOfLISFast(nums) {  
  const tails = []; // tails[i] = "పొడవు i+1 ఉన్న LIS యొక్క  
  అతిచిన్న చివరి element"  
  
  for (const num of nums) {  
    let left = 0, right = tails.length;  
    while (left < right) { // 🔑 lowerBound binary search  
      const mid = (left + right) >> 1;  
      if (tails[mid] < num) left = mid + 1;  
      else right = mid;  
    }  
    tails[left] = num; // replace లేదా append  
  }  
  return tails.length;  
}  
//  $O(n \log n)$   
// ⚠️ `tails` నిజమైన LIS కాదు - దాని *పొడవు* మాత్రమే సరైనది
```

Intuition: "ఒకే పొడవు ఉన్న subsequences లో — చివరి element ఎంత చిన్నదైతే, భవిష్యత్తులో అంత ఎక్కువ extend చేయొచ్చు." అందుకే ప్రతి పొడవుకీ అతిచిన్న tail ఉంచుతాం.

Variants

```
// 1. Number of LIS  
function findNumberOfLIS(nums) {  
  const n = nums.length;  
  const len = new Array(n).fill(1), count = new Array(n).fill(1);  
  let maxLen = 1;  
  
  for (let i = 1; i < n; i++) {  
    for (let j = 0; j < i; j++) {
```

```

    if (nums[j] < nums[i]) {
        if (len[j] + 1 > len[i]) { len[i] = len[j] + 1; count[i] = count[j]; }
        else if (len[j] + 1 === len[i]) count[i] += count[j]; // 🔑 ఇంకో మార్గం
    }
}
maxLen = Math.max(maxLen, len[i]);
}
return len.reduce((sum, l, i) => (l === maxLen ? sum + count[i] : sum), 0);
}

// 2. Russian Doll Envelopes – 2D → sort + LIS ★
function maxEnvelopes(envelopes) {
    // 🔑 width ascending, width equal అయితే height DESCENDING
    // (ఎందుకు: ఒకే width ఉన్నవి ఒకదానిలో ఒకటి పట్టవు → LIS లో ఒకటే తీసుకోవాలి)
    envelopes.sort((a, b) => a[0] - b[0] || b[1] - a[1]);
    return lengthOfLISFast(envelopes.map((e) => e[1])); // heights మీద LIS
}

// 3. Longest String Chain
function longestStrChain(words) {
    words.sort((a, b) => a.length - b.length);
    const dp = new Map();
    let max = 1;
    for (const word of words) {
        dp.set(word, 1);
        for (let i = 0; i < word.length; i++) {
            const pred = word.slice(0, i) + word.slice(i + 1);
            if (dp.has(pred)) dp.set(word, Math.max(dp.get(word), dp.get(pred) + 1));
        }
        max = Math.max(max, dp.get(word));
    }
    return max;
}

// 4. Maximum Length of Pair Chain – greedy (end ప్రకారం sort)  $O(n \log n)$ 

```

Gotchas

- **Subsequence vs subarray** – LIS contiguous కాదు!
- **Russian Dolls లో height descending sort** మర్చిపోవడం → తప్పు సమాధానం.
- **tails array ని LIS అనుకోవడం** – పొడవు మాత్రమే సరైనది.
- **Strictly vs non-strictly increasing** – $<$ vs $<=$ (binary search లో lowerBound vs upperBound).

Interview దృష్టి

- "LIS $O(n \log n)$ ఎలా?" → patience sorting + binary search; tails array అర్థం.
- "Russian Dolls ఎందుకు height descending?" → ఒకే width ఉన్నవి chain లో రాకుండా.

37. Advanced DP — Tree, Interval & Bitmask

Tree DP

```
// Binary Tree Maximum Path Sum (Pattern 18 లో చూశాం)
// House Robber III (Pattern 32)
// Diameter (Pattern 18)
// 🗝 సూత్రం: postorder + ప్రతి node కి "states" (ఈ node వాడామా వాడలేదా)
```

Interval DP

```
// Burst Balloons (Hard) – "చివరిగా ఏది పగలగొట్టాలి?" అని ఆలోచించడం ✨
function maxCoins(nums) {
  const arr = [1, ...nums, 1]; // 🗝 virtual boundaries
  const n = arr.length;
  const dp = Array.from({ length: n }, () => new Array(n).fill(0));

  for (let len = 2; len < n; len++) // 🗝 interval పొడవు పెంచుతూ
    for (let left = 0; left + len < n; left++) {
      const right = left + len;
      for (let k = left + 1; k < right; k++) {
        // 🗝 k = ఈ interval లో *చివరిగా* పగలగొట్టేది
        // → అప్పుడు left, right ఇంకా ఉంటాయి → subproblems స్వతంత్రం ✅
        dp[left][right] = Math.max(
          dp[left][right],
          dp[left][k] + arr[left] * arr[k] * arr[right] + dp[k][right]
        );
      }
    }
  return dp[0][n - 1];
}
// O(n³)
// 🗝 అంతర్దృష్టి: "మొదట ఏది పగలగొట్టాలి" అని ఆలోచిస్తే subproblems dependent అవుతాయి;
// "చివరిగా ఏది" అని ఆలోచిస్తే independent → DP సాధ్యం ✨

// Matrix Chain Multiplication, Minimum Cost to Cut a Stick – అదే pattern
```

Bitmask DP ($n \leq 20$)

```
// Travelling Salesman Problem
function tsp(dist) {
  const n = dist.length;
  const FULL = (1 << n) - 1;
  const memo = new Map();

  function dp(mask, pos) {
    if (mask === FULL) return dist[pos][0]; // అన్నీ visit → తిరిగి start కి
    const key = `${mask},${pos}`;
    if (memo.has(key)) return memo.get(key);

    let best = Infinity;
```

```

for (let city = 0; city < n; city++) {
  if (mask & (1 << city)) continue; // 🔑 ఇప్పటికే visit అయింది
  best = Math.min(best, dist[pos][city] + dp(mask | (1 << city), city));
}
memo.set(key, best);
return best;
}
return dp(1, 0); // city 0 నుండి
}
//  $O(2^n \times n^2) - n \leq 20$  వరకు అచరణీయం
// 🔑 mask = "ఏ cities visit అయ్యాయి" (bit set = visited)

```

📁 DP + State machine (Stock problems)

```

// Best Time to Buy and Sell Stock with Cooldown
function maxProfitCooldown(prices) {
  let hold = -Infinity; // stock చేతిలో ఉంది
  let sold = 0; // ఇప్పుడే అమ్మాను (cooldown లో)
  let rest = 0; // ఖాళీగా ఉన్నాను (కొనొచ్చు)

  for (const price of prices) {
    const prevSold = sold;
    sold = hold + price; // అమ్మడం
    hold = Math.max(hold, rest - price); // కొనడం లేదా ఉంచుకోవడం
    rest = Math.max(rest, prevSold); // విశ్రాంతి
  }
  return Math.max(sold, rest);
}
// 🔑 State machine DP - ప్రతి రోజు ఏ "స్థితి" లో ఉన్నానో track చేయడం
// ఈ pattern అన్ని stock problems కి (k transactions, fee, cooldown) పని చేస్తుంది

```

Key Points

- **Tree DP** = postorder + per-node states.
- **Interval DP** = `dp[left][right]` , పొడవు పెంచుతూ; "చివరిగా ఏది" ఆలోచన.
- **Bitmask DP** = `dp[mask][pos]` , $n \leq 20$.
- **State machine DP** = stock/game problems (hold/sold/rest).

Interview దృష్టి

- "Burst Balloons intuition?" → "చివరిగా ఏది" reframing – ఇది చెప్పగలగడం hard DP అర్థమైందని రుజువు.
- " $n \leq 20$ అంటే?" → bitmask DP సూచన.

Part 9 — Greedy, Bit & Math

38. Greedy — ఎప్పుడు పని చేస్తుంది

🔍 Recognition signals

- ✓ "minimum number of X" / "maximum number of Y"
- ✓ ప్రతి **step** లో "ఉత్తమమైనది" ఎంచుకుంటే సరిపోతుందని అనిపిస్తుంది
- ✓ Sorting తర్వాత ఒకే pass సరిపోతుంది
- ✓ Intervals, scheduling, resource **allocation**
- ⚠ కానీ — ఎప్పుడూ నిరూపించాలి! (greedy తప్పు అయ్యే ఉచ్చు చాలా సాధారణం)

వివరణ

Greedy = ప్రతి step లో local optimum ఎంచుకోవడం, global optimum వస్తుందని ఆశించడం.

Greedy పని చేయాలంటే రెండు లక్షణాలు అవసరం:

- 1 Greedy choice **property** — local optimal choice **global** optimal solution లో భాగం
- 2 Optimal substructure — subproblem యొక్క optimal solution కలిపితే **global** optimal

Real-life Scenario

Greedy = చిల్లర ఇవ్వడం. ₹87 ఇవ్వాలి — ₹50 ఒకటి, ₹20 ఒకటి, ₹10 ఒకటి, ₹5 ఒకటి, ₹2 ఒకటి = 5 notes. **ప్రతినారి అతిపెద్ద note తీసుకోవడం (greedy) → optimal** ✓.

కానీ ఒక ఊహాత్మక దేశంలో notes {1, 3, 4} ఉంటే — ₹6 ఇవ్వాలంటే greedy: 4+1+1 = **3 notes**. Optimal: 3+3 = **2 notes** ✗. అదే greedy యొక్క ప్రమాదం — coin system మీద ఆధారపడుతుంది!

✓ Greedy పని చేసే classic patterns

```
// 1. Interval scheduling — END ప్రకారం sort (Pattern 7)
// 2. Jump Game — reachability
function canJump(nums) {
  let maxReach = 0;
  for (let i = 0; i < nums.length; i++) {
    if (i > maxReach) return false;
    maxReach = Math.max(maxReach, i + nums[i]);
  }
  return true;
}

// 3. Gas Station — total sufficiency + reset
function canCompleteCircuit(gas, cost) {
  let total = 0, tank = 0, start = 0;
  for (let i = 0; i < gas.length; i++) {
```

```

    const diff = gas[i] - cost[i];
    total += diff;
    tank += diff;
    if (tank < 0) { start = i + 1; tank = 0; } // 🔑 ఇక్కడి వరకు ఏ start పని చేయదు
  }
  return total >= 0 ? start : -1;
}
// 🔑 Proof: total >= 0 అయితే ఒక valid start ఉంటుంది; tank negative అయిన ప్రతిసారి
//   ఆ range లోని ఏ point నుండైనా మొదలుపెట్టలేం → తర్వాతిది try చేయాలి

// 4. Task Scheduler – అత్యధిక frequency ఉన్నదాన్ని ముందు
function leastInterval(tasks, n) {
  const freq = new Array(26).fill(0);
  for (const t of tasks) freq[t.charCodeAt(0) - 65]++;
  freq.sort((a, b) => b - a);

  const maxFreq = freq[0];
  let idleSlots = (maxFreq - 1) * n;
  for (let i = 1; i < 26 && idleSlots > 0; i++) idleSlots -= Math.min(freq[i], maxFreq - 1);

  return tasks.length + Math.max(0, idleSlots);
}

// 5. Partition Labels – last occurrence tracking
function partitionLabels(s) {
  const last = new Map();
  for (let i = 0; i < s.length; i++) last.set(s[i], i);

  const res = [];
  let start = 0, end = 0;
  for (let i = 0; i < s.length; i++) {
    end = Math.max(end, last.get(s[i])); // 🔑 ఈ partition ఎక్కడ ముగియాలి
    if (i === end) { res.push(end - start + 1); start = i + 1; }
  }
  return res;
}

```

❌ Greedy విఫలమయ్యే classic ఉదాహరణలు (ఇవి తెలియాలి)

1. Coin Change {1, 3, 4}, amount 6
Greedy: 4 + 1 + 1 = 3 coins ❌ | Optimal (DP): 3 + 3 = 2 coins ✅
2. 0/1 Knapsack – value/weight ratio ప్రకారం greedy ❌
(Fractional knapsack లో greedy ✅ – items విభజించవచ్చు కాబట్టి)
3. Longest path in graph – greedy ❌
4. Word Break – greedy longest match ❌
"aaaa", dict = ["aaa", "aa"] → greedy "aaa" తీసుకుంటే "a" మిగులుతుంది ❌

ఇదే DP vs greedy తేడా: greedy లో ఒకసారి తీసుకున్న నిర్ణయాన్ని పునఃపరిశీలించం. DP లో అన్ని అవకాశాలను పరిశీలిస్తాం.

Greedy ని నిరూపించడం (interview లో ఇది అడుగుతారు)

1 EXCHANGE ARGUMENT (అత్యంత సాధారణం):

"ఒక optimal solution ని తీసుకో. అందులో నా greedy choice లేకపోతే – దాన్ని నా greedy choice తో మార్చినా (exchange) solution ఇంకా optimal గానే ఉంటుంది. కాబట్టి greedy choice ఉన్న optimal solution ఒకటి ఉంటుంది." ✓

ఉదా (interval scheduling): "మొదట ముగిసేదాన్ని ఎంచుకుంటే – ఏ optimal solution లోనైనా మొదటి interval ని దీనితో మార్చిచ్చు, ఎందుకంటే ఇది ముందుగా ముగుస్తుంది కాబట్టి మిగతా intervals కి ఎక్కువ స్థలం ఇస్తుంది."

2 COUNTEREXAMPLE వెతకడం:

నిరూపించలేకపోతే – చిన్న counterexample వెతకండి. దొరికితే → DP వైపు.

Greedy vs DP – decision guide

Greedy try చేయి, కానీ:

- └ చిన్న counterexample దొరికిందా? → DP కి వెళ్ళు
- └ "ఇది సరిపోతుందా?" అని సందేహం ఉందా? → DP సురక్షితం
- └ Exchange argument నిరూపించగలవా? → greedy ✓
- └ Interview లో సమయం తక్కువా? → "greedy works here because..." చెప్పి ముందుకు

ఈ pattern కి చెందిన problems

Easy: Assign Cookies · Lemonade Change · Best Time to Buy and Sell Stock II
Medium: Jump Game I/II · Gas Station ★ · Task Scheduler · Partition Labels
Non-overlapping Intervals · Minimum Arrows · Queue Reconstruction by Height
Boats to Save People · Hand of Straights
Hard: Candy ★ · Create Maximum Number · IPO · Minimum Number of Refueling Stops

```
// Candy (Hard) – రెండు passes greedy ★
function candy(ratings) {
  const n = ratings.length;
  const candies = new Array(n).fill(1);

  for (let i = 1; i < n; i++) // ఎడమ → కుడి
    if (ratings[i] > ratings[i-1]) candies[i] = candies[i-1] + 1;

  for (let i = n - 2; i >= 0; i--) // కుడి → ఎడమ
    if (ratings[i] > ratings[i+1]) candies[i] = Math.max(candies[i], candies[i+1] + 1);

  return candies.reduce((a, b) => a + b, 0);
}
// 🗝️ "ఒక పాస్ లో రెండు constraints satisfy చేయలేను → రెండు passes" – ఉపయోగకరమైన technique
```

Gotchas

- నిరూపించకుండా greedy → తప్పు సమాధానం (అత్యంత సాధారణ interview తప్పు).
- Sort key తప్పు – start vs end vs ratio.

- "Greedy సులభం కాబట్టి సరైనది" అనుకోవడం – DP అవసరమా అని ఆలోచించాలి.
- Two-pass greedy ని ఒకే pass లో చేయడానికి ప్రయత్నించడం (Candy).

Interview దృష్టి

- "Greedy పని చేస్తుందని ఎలా తెలుసు?" → **exchange argument** లేదా counterexample వెతకడం – ఈ reasoning చెప్పకపోతే greedy solution అంగీకరించరు.
- "Coin change లో greedy ఎందుకు fail?" → {1,3,4} counterexample – classic.

39. Bit Manipulation Patterns

🔍 Recognition signals

- ✓ "single number" / "appears once/twice/thrice"
- ✓ "power of two/four"
- ✓ "count bits" / "hamming distance"
- ✓ "subsets" ($n \leq 20 \rightarrow$ bitmask)
- ✓ "XOR" అనే పదం
- ✓ $O(1)$ space constraint + counting

📁 ప్రాథమిక operations

```
a & b    // AND – రెండూ 1 అయితే 1
a | b    // OR – ఏదో ఒకటి 1 అయితే 1
a ^ b    // XOR – వేర్వేరు అయితే 1 ★
~a       // NOT
a << n   // left shift =  $a \times 2^n$ 
a >> n   // right shift =  $a \div 2^n$  (sign preserve)
a >>> n  // unsigned right shift (JS-specific)
```

🔑 XOR యొక్క మాయా లక్షణాలు (అత్యంత ముఖ్యం)

```
a ^ a = 0          ← ఒకే సంఖ్య రెండుసార్లు → రద్దు ★
a ^ 0 = a
a ^ b ^ a = b      ← commutative + associative
```

```
// Single Number – అన్నీ రెండుసార్లు, ఒకటి ఒకసారి
function singleNumber(nums) {
  return nums.reduce((acc, num) => acc ^ num, 0);
}
// 🔑 జతలు అన్నీ రద్దు అవుతాయి → ఒంటరిది మిగులుతుంది ✨  $O(n)$  time,  $O(1)$  space

// Missing Number
function missingNumber(nums) {
  let xor = nums.length;
  for (let i = 0; i < nums.length; i++) xor ^= i ^ nums[i];
}
```

```

    return xor;
}

// Two numbers appear once, మిగతావి రెండుసార్లు (Single Number III)
function singleNumberIII(nums) {
    const xorAll = nums.reduce((a, b) => a ^ b, 0); // = a ^ b
    const lowestBit = xorAll & -xorAll; // 🔑 అతి కుడి set bit
    let a = 0, b = 0;
    for (const num of nums) { // 🔑 ఆ bit ఆధారంగా రెండు గుంపులు
        if (num & lowestBit) a ^= num;
        else b ^= num;
    }
    return [a, b];
}

```

📁 సాధారణ bit tricks (గుర్తుపెట్టుకోవాలి)

```

n & 1 // odd అ? (1 = odd)
n >> 1 // ÷2
n & (n - 1) // ★ అతి కుడి set bit ని తొలగించడం
n & (n - 1) === 0 // ★ power of 2 అ? (n > 0)
n & -n // ★ అతి కుడి set bit ని extract చేయడం
n | (1 << i) // i-వ bit ని set చేయడం
n & ~(1 << i) // i-వ bit ని clear చేయడం
n ^ (1 << i) // i-వ bit ని toggle
(n >> i) & 1 // i-వ bit ని చదవడం
~n + 1 // negation (two's complement)

```

```

// Count set bits (Brian Kernighan) ★
function hammingWeight(n) {
    let count = 0;
    while (n) { n &= n - 1; count++; } // 🔑 ప్రతిసారి ఒక set bit తొలగింపు
    return count;
}

// 0(set bits) - naive O(32) కంటే వేగం

// Counting Bits (0..n అన్నిటికీ) - DP + bits
function countBits(n) {
    const dp = new Array(n + 1).fill(0);
    for (let i = 1; i <= n; i++) dp[i] = dp[i >> 1] + (i & 1);
    // ↑ i/2 యొక్క bits + చివరి bit
    return dp;
}

// Power of two
const isPowerOfTwo = (n) => n > 0 && (n & (n - 1)) === 0;

// Reverse bits
function reverseBits(n) {
    let result = 0;
    for (let i = 0; i < 32; i++) {
        result = (result << 1) | (n & 1);
        n >>= 1;
    }
}

```

```
return result >>> 0; // ⚠️ unsigned
}
```

📁 Bitmask — subsets enumeration

```
// అన్ని subsets ని bitmask తో generate చేయడం (backtracking ప్రత్యామ్నాయం)
function subsets(nums) {
  const n = nums.length, res = [];
  for (let mask = 0; mask < (1 << n); mask++) { // 🔑 0 నుండి 2^n-1 వరకు
    const subset = [];
    for (let i = 0; i < n; i++)
      if (mask & (1 << i)) subset.push(nums[i]); // i-వ bit set → include
    res.push(subset);
  }
  return res;
}
// O(2^n × n) – n ≤ 20 వరకు
```

JavaScript-specific gotchas ⚠️

```
// 1. Bitwise operators 32-bit signed integers మీద పని చేస్తాయి
2 ** 31 | 0 // -2147483648 (overflow!)
Number.MAX_SAFE_INTEGER & 1 // ⚠️ తప్పు ఫలితం

// 2. Negative numbers – two's complement
-5 >> 1 // -3 (sign preserve)
-5 >>> 1 // 2147483645 (unsigned)

// 3. పెద్ద సంఖ్యలకి BigInt
BigInt(2) ** BigInt(64)

// 4. XOR with 0 initialization
[].reduce((a, b) => a ^ b, 0) // ✅ 0 (initial value తప్పనిసరి)
```

ఈ pattern కి చెందిన problems

Easy: Single Number ★ · Number of 1 Bits · Power of Two/Three/Four
 Hamming Distance · Reverse Bits · Complement of Base 10 Integer

Medium: Single Number II/III · Counting Bits · Sum of Two Integers (bit addition)
 Subsets (bitmask) · Bitwise AND of Numbers Range · Maximum XOR of Two Numbers (trie)
 Gray Code · Total Hamming Distance

Hard: Maximum XOR With an Element From Array · Minimum Number of K Consecutive Bit Flips

Gotchas

- JS 32-bit limitation — పెద్ద సంఖ్యలతో bitwise → తప్పు.
- Operator precedence — `a & 1 === 0` ❌ (=== ముందు) → `(a & 1) === 0` ✅.
- `>>` vs `>>>` — negative numbers తో తేడా.

- XOR reduce లో initial value మర్చిపోవడం.
- Bit tricks ని ఎప్పుడూ వాడాలని అనుకోవడం – readability ముఖ్యం; interview లో వివరించగలగాలి.

Interview దృష్టి

- "Single Number ని $O(1)$ space లో?" → XOR properties – classic.
- " n & $(n-1)$ ఏం చేస్తుంది?" → అతి కుడి set bit తొలగింపు → power of 2 check, bit counting.

40. Math & Number Theory Patterns

🔍 సాధారణ patterns

```
// 1. GCD / LCM (Euclidean algorithm)
const gcd = (a, b) => (b === 0 ? a : gcd(b, a % b));
const lcm = (a, b) => (a / gcd(a, b)) * b; // 🔑 overflow నివారణకి ముందు divide

// 2. Prime – Sieve of Eratosthenes  $O(n \log \log n)$ 
function sieve(n) {
  const isPrime = new Array(n + 1).fill(true);
  isPrime[0] = isPrime[1] = false;
  for (let i = 2; i * i <= n; i++) // 🔑  $\sqrt{n}$  వరకు చాలు
    if (isPrime[i])
      for (let j = i * i; j <= n; j += i) isPrime[j] = false; // 🔑  $i^2$  నుండి
  return isPrime;
}

const isPrimeSingle = (n) => {
  if (n < 2) return false;
  for (let i = 2; i * i <= n; i++) if (n % i === 0) return false;
  return true;
};

// 3. Fast exponentiation  $O(\log n)$ 
function power(base, exp, mod = Infinity) {
  let result = 1;
  base %= mod;
  while (exp > 0) {
    if (exp & 1) result = (result * base) % mod; // odd → ఒకసారి multiply
    base = (base * base) % mod;
    exp >>= 1;
  }
  return result;
}

// 4. Digit operations
const digits = (n) => String(Math.abs(n)).split('').map(Number);
const digitSum = (n) => { let s = 0; while (n) { s += n % 10; n = Math.floor(n / 10); } return s; };
const reverseNum = (n) => { let r = 0; while (n) { r = r * 10 + n % 10; n = Math.floor(n / 10); } return r; };

```

```
// 5. Modular arithmetic (counting problems)
const MOD = 1_000_000_007n;
// ⚠️ JS లో పెద్ద సంఖ్యలు → BigInt లేదా ప్రతి step లో % MOD

// 6. Combinatorics
function nCr(n, r) {
  if (r > n - r) r = n - r;
  let result = 1;
  for (let i = 0; i < r; i++) result = (result * (n - i)) / (i + 1);
  return Math.round(result);
}
// Unique Paths = C(m+n-2, m-1) ⭐ (DP అవసరం లేదు!)
```

సాధారణ math insights

```
// Happy Number → cycle detection (Pattern 2)
// Pow(x, n) → fast exponentiation + negative n handling
// Sqrt(x) → binary search
// Excel Sheet Column Number → base-26
// Roman to Integer → greedy + lookup
// Ugly Numbers → multiple pointers / heap
// Count Primes → sieve
// Add Digits → digital root: 1 + (n-1) % 9 ⭐ (O(1)!)
```

Gotchas

- **Integer overflow** — JS `Number.MAX_SAFE_INTEGER` = $2^{53} - 1$ → BigInt.
- **Negative modulo** — $((x \% m) + m) \% m$.
- **Math.floor vs Math.trunc** — negative numbers తో వేరు.
- **Floating point precision** — `0.1 + 0.2 !== 0.3`.
- **Division** — `Math.floor(a / b)` (integer division JS లో లేదు).

Part 10 — Design & Interview Craft

41. Data Structure Design Patterns

🔍 Recognition signals

- ✓ "design a class with $O(1)$ operations"
- ✓ "LRU/LFU cache"
- ✓ "implement iterator"
- ✓ "insert/delete/getRandom in $O(1)$ "
- ✓ "min stack" / "max queue"

📁 LRU Cache (అత్యంత అడిగే design question) ★

```
// 🔑 అంతర్దృష్టి: HashMap ( $O(1)$  lookup) + Doubly Linked List ( $O(1)$  reorder)
class LRUCache {
  constructor(capacity) {
    this.capacity = capacity;
    this.map = new Map();
    // Sentinel nodes – edge cases తీలగిస్తాయి
    this.head = { key: 0, val: 0, prev: null, next: null }; // most recent
    this.tail = { key: 0, val: 0, prev: null, next: null }; // least recent
    this.head.next = this.tail;
    this.tail.prev = this.head;
  }

  #remove(node) {
    node.prev.next = node.next;
    node.next.prev = node.prev;
  }

  #insertFront(node) {
    node.next = this.head.next;
    node.prev = this.head;
    this.head.next.prev = node;
    this.head.next = node;
  }

  get(key) {
    if (!this.map.has(key)) return -1;
    const node = this.map.get(key);
    this.#remove(node);
    this.#insertFront(node); // 🔑 recently used ను mark
    return node.val;
  }

  put(key, value) {
    if (this.map.has(key)) this.#remove(this.map.get(key));

    const node = { key, val: value, prev: null, next: null };
    this.#insertFront(node);
  }
}
```

```

    this.map.set(key, node);

    if (this.map.size > this.capacity) {
        const lru = this.tail.prev; // 🗝 least recently used
        this.#remove(lru);
        this.map.delete(lru.key);
    }
}
}
// get/put టెండు O(1) ✅

// ⚡ JS ట్రిక్: Map insertion order నిలుపుతుంది → సులభమైన LRU
class LRUCacheSimple {
    constructor(capacity) { this.capacity = capacity; this.map = new Map(); }
    get(key) {
        if (!this.map.has(key)) return -1;
        const val = this.map.get(key);
        this.map.delete(key); this.map.set(key, val); // 🗝 చివరికి తరలింపు
        return val;
    }
    put(key, value) {
        if (this.map.has(key)) this.map.delete(key);
        this.map.set(key, value);
        if (this.map.size > this.capacity) this.map.delete(this.map.keys().next().value);
    }
}
// ⚠ Interview లో - DLL version అడగొచ్చు (Map trick JS-specific)

```

📁 Min Stack (O(1) getMin)

```

class MinStack {
    constructor() { this.stack = []; this.minStack = []; }

    push(val) {
        this.stack.push(val);
        // 🗝 ప్రతి element దగ్గర "ఇప్పటివరకు minimum" ని కూడా store
        this.minStack.push(this.minStack.length === 0 ? val : Math.min(val, this.getMin()));
    }
    pop() { this.minStack.pop(); return this.stack.pop(); }
    top() { return this.stack[this.stack.length - 1]; }
    getMin() { return this.minStack[this.minStack.length - 1]; }
}
// అన్నీ O(1) ✅

```

📁 Insert Delete GetRandom O(1)

```

class RandomizedSet {
    constructor() { this.map = new Map(); this.arr = []; } // 🗝 array = random access కి

    insert(val) {
        if (this.map.has(val)) return false;
        this.map.set(val, this.arr.length);
        this.arr.push(val);
    }
}

```

```

    return true;
}
remove(val) {
  if (!this.map.has(val)) return false;
  const idx = this.map.get(val);
  const last = this.arr[this.arr.length - 1];
  // 🗝️ చివరిదాన్ని ఇక్కడికి తరలించి pop → O(1) removal
  this.arr[idx] = last;
  this.map.set(last, idx);
  this.arr.pop();
  this.map.delete(val);
  return true;
}
getRandom() { return this.arr[Math.floor(Math.random() * this.arr.length)]; }
}

```

Trie (prefix tree)

```

class Trie {
  constructor() { this.root = { children: new Map(), isEnd: false }; }

  insert(word) {
    let node = this.root;
    for (const ch of word) {
      if (!node.children.has(ch)) node.children.set(ch, { children: new Map(), isEnd: false });
    };
    node = node.children.get(ch);
  }
  node.isEnd = true;
}

#traverse(prefix) {
  let node = this.root;
  for (const ch of prefix) {
    if (!node.children.has(ch)) return null;
    node = node.children.get(ch);
  }
  return node;
}

search(word) { return this.#traverse(word)?.isEnd ?? false; }
startsWith(prefix) { return this.#traverse(prefix) !== null; }
}

// Insert/search O(len) – hash set కంటే prefix queries లో మేలు
// ఉపయోగాలు: autocomplete, word search II, maximum XOR, IP routing

```

Iterator design

```

class BSTIterator {
  // O(h) space, O(1) amortized next()
  constructor(root) { this.stack = []; this.#pushLeft(root); }
  #pushLeft(node) { while (node) { this.stack.push(node); node = node.left; } }
  next() {
    const node = this.stack.pop();
    this.#pushLeft(node.right);
    return node.val;
  }
}

```

```

}
hasNext() { return this.stack.length > 0; }
}
// 🗝 Inorder traversal ని "pause/resume" చేయగలిగేలా stack లో state ఉంచడం

```

ఈ pattern కి చెందిన problems

Medium: LRU Cache 🌟 · Min Stack · Insert Delete GetRandom $O(1)$ · Implement Trie
 BST Iterator · Design Twitter · Design Underground System
 Time Based Key-Value Store · Design Hit Counter · Snapshot Array
Hard: LFU Cache · Design In-Memory File System · All $O(1)$ Data Structure
 Design Search Autocomplete System · Max Stack

Interview దృష్టి

- "LRU cache design చేయి" → HashMap + DLL, sentinel nodes, $O(1)$ — అత్యంత సాధారణ design question.
- " $O(1)$ getMin ఎలా?" → auxiliary min stack (space-time trade-off).
- "getRandom $O(1)$ ఎలా?" → array + map + swap-with-last removal.

42. Interview Communication Framework

📅 UMPIRE — ప్రతి problem కి ఇదే క్రమం

U — UNDERSTAND	(5 min)	clarify + examples + edge cases
M — MATCH	(2 min)	ఏ pattern? (Part 0 signals)
P — PLAN	(5 min)	approach + complexity (code కి ముందు!)
I — IMPLEMENT	(20 min)	code (మాట్లాడుతూ)
R — REVIEW	(5 min)	dry run with example
E — EVALUATE	(3 min)	complexity + optimizations + trade-offs

1 UNDERSTAND — అడగవలసిన ప్రశ్నలు

- **Input** పరిమాణం? (constraints → algorithm — Part 0.1)
- **Input** రకం? (sorted? duplicates? negatives? empty? null?)
- Output ఏమిటి? (index vs value, all vs any, count vs list)
- Edge cases? (empty, single element, all same, overflow)
- **Input** modify చేయొచ్చా?
- Memory constraint ఉందా?
- Multiple valid answers ఉంటే? (ఏదైనా OK అ?)

ఉదాహరణ: "Array sorted అని అనుకోవచ్చా? Duplicates ఉండొచ్చా? n ఎంత పెద్దది? In-place modify చేయొచ్చా?"
 — ఈ 4 ప్రశ్నలు అడిగితేనే interviewer మీరు ఆలోచిస్తున్నారని నమ్ముతాడు.

2 MATCH + 3 PLAN — ఇలా మాట్లాడాలి

"Brute force ఏమిటంటే — ప్రతి jodi ని check చేయడం, అది $O(n^2)$.
అక్కడ నేను ఒకే విషయాన్ని పదే పదే recompute చేస్తున్నాను.

ఈ problem లో 'sorted array' + 'pair' అనే signals ఉన్నాయి —
ఇది two pointers pattern అనిపిస్తోంది.

నా plan: left, right pointers పెట్టి, sum target కంటే తక్కువైతే left++,
ఎక్కువైతే right--. ఇది $O(n)$ time, $O(1)$ space.

ఇది సరైన approach అనిపిస్తోందా, లేక వేరే constraint ఉందా?" ←  confirm చేయడం

ముఖ్యం: interviewer approve చేసేవరకు code రాయొద్దు. తప్పు దారిలో 20 నిమిషాలు వృథా అయితే తిరిగి రాలేరు.

4 IMPLEMENT — code రాస్తూ మాట్లాడటం

✓ చేయాల్సినవి:

- అర్థవంతమైన variable names (l/r కంటే left/right; a/b కంటే current/previous)
- Helper functions (code శుభ్రంగా)
- Edge cases ముందుగా handle
- ప్రతి కీలక line వద్ద ఎందుకో చెప్పడం
- Stuck అయితే చెప్పడం ("ఇక్కడ ఆలోచిస్తున్నాను...")

✗ చేయకూడనివి:

- మౌనంగా 10 నిమిషాలు code రాయడం
- Perfect syntax కోసం ఆగిపోవడం (logic ముఖ్యం)
- Hints ని ignore చేయడం (interviewer hint ఇస్తే — అది సూచన!)

5 REVIEW — dry run (ఇది skip చేయొద్దు!)

"ఇప్పుడు example [2, 7, 11, 15], target = 9 తో నడిపి చూస్తాను:

left=0 (2), right=3 (15) → sum=17 > 9 → right--
left=0 (2), right=2 (11) → sum=13 > 9 → right--
left=0 (2), right=1 (7) → sum=9 = 9 → return [0, 1] ✓

Edge cases: ఖాళీ array → while loop run కాదు → [-1,-1] ✓
ఒకే element → left = right → loop skip ✓ "

Dry run లో bugs దొరకడం మంచిది — interviewer "ఇతను తన code ని పరీక్షిస్తాడు" అని గమనిస్తాడు.

6 EVALUATE

"Time complexity: $O(n)$ — ప్రతి element గరిష్టంగా ఒకసారి touch అవుతుంది.
Space: $O(1)$ — రెండు pointers మాత్రమే.

Follow-ups గురించి:

- Array sorted కాకపోతే - hash map $O(n)$ time, $O(n)$ space
- Triplets కావాలంటే - sort + fix one + two pointers $\rightarrow O(n^2)$
- Streaming data అయితే - వేరే approach అవసరం"

Stuck అయితే ఏం చేయాలి (అత్యంత ముఖ్యం)

1. బయటకి ఆలోచించండి - "నేను ఇలా ఆలోచిస్తున్నాను... కానీ ఇక్కడ ఆగిపోయాను"
2. చిన్న example తీసుకోండి - కాగితం మీద manually solve చేయండి
3. Brute force రాయండి - "మొదట పని చేసేది రాస్తాను, తర్వాత optimize"
4. Signals మళ్ళీ చూడండి - "sorted ఉంది... $O(1)$ space అన్నారు..."
5. Hint అడగండి - "ఈ దిశ సరైనదేనా?" (అడగడం బలహీనత కాదు)
6. సంబంధిత problem గుర్తు చేసుకోండి - "ఇది X లాంటిది... అక్కడ నేను Y వాడాను"

✗ సాధారణ interview తప్పులు

- | | |
|--|--|
| 1. వెంటనే code రాయడం (clarify లేకుండా) | → తప్పు problem solve చేస్తారు |
| 2. మౌనంగా ఉండటం | → interviewer కి మీ ఆలోచన తెలియదు |
| 3. Complexity చెప్పకపోవడం | → అడగకముందే చెప్పాలి |
| 4. Test చేయకపోవడం | → bugs interviewer కనుక్కుంటాడు |
| 5. Hints ని పట్టించుకోకపోవడం | → red flag |
| 6. Optimal solution కోసం ఆగిపోవడం | → brute force కూడా లేకుండా సమయం అయిపోతుంది |
| 7. Edge cases మర్చిపోవడం | → empty, single, duplicates, overflow |
| 8. "నాకు తెలియదు" అని ఆగిపోవడం | → "ఇలా ఆలోచిస్తాను..." అని ప్రయత్నించాలి |

Time management (45 నిమిషాల interview)

- 0-5 Clarify + examples
- 5-10 Approach + complexity + confirm
- 10-30 Code
- 30-38 Dry run + bugs fix
- 38-45 Complexity + follow-ups + questions

⚠ 15 నిమిషాలు దాటినా approach లేకపోతే → brute force రాయడం మొదలుపెట్టండి (ఏమీ లేకపోవడం కంటే working brute force మేలు)

43. Pattern Decision Tree (ఒక్క page cheat-sheet) ★

```

INPUT రకం ఏమిటి?
|
├─ ARRAY / STRING
|   |
|   ├─ Sorted అ?
|   |   |
|   |   └─ అవును → Two Pointers · Binary Search
|   |   └─ కాదు → Hash Map · Sort చేసి try
|   |
|   └─ "contiguous subarray/substring" అ?
|       |
|       ├─ Negatives లేవు → Sliding Window ★
|       └─ Negatives ఉన్నాయి → Prefix Sum + HashMap
|   |
|   └─ "next greater/smaller" అ? → Monotonic Stack

```

- └ Values $1..n$ అ? → Cyclic Sort
- └ Intervals అ? → Sort + Merge / Sweep Line
- └ "Kth" / "top K" అ? → Heap · QuickSelect
- └ "minimize max / maximize min" అ? → Binary Search on Answer ★★
- └ LINKED LIST
 - └ Cycle / middle / nth-from-end → Fast & Slow Pointers
 - └ Reverse (whole/part/k-group) → In-place Reversal
 - └ Head మారొచ్చు → Dummy Node
- └ TREE
 - └ Level-wise / shortest depth → BFS
 - └ Path / depth / property → DFS
 - └ Parent → child సమాచారం → Top-Down
 - └ Child → parent సమాధానం → Bottom-Up ★
 - └ BST అ? → Inorder (sorted) · Range validation
 - └ Construct / serialize → preorder + inorder / null markers
- └ GRAPH
 - └ Shortest path
 - └ Unweighted → BFS
 - └ Weighted non-negative → Dijkstra
 - └ Negative weights → Bellman-Ford
 - └ All pairs → Floyd-Warshall
 - └ Dependencies / ordering → Topological Sort
 - └ Connectivity (dynamic) → Union-Find
 - └ Grid → DFS/BFS (multi-source అ?)
 - └ Cycle detection → DFS (3-color / parent)
- └ "ALL POSSIBLE" / OPTIMIZATION
 - └ "అన్ని solutions కావాలి" → Backtracking ★
 - └ "max/min/count ways" + choices + overlapping → DP ★★
 - └ ఒక sequence → 1D DP
 - └ రెండు sequences → 2D String DP
 - └ Grid → 2D Grid DP
 - └ Items + capacity → Knapsack
 - └ $n \leq 20$ → Bitmask DP
 - └ Intervals → Interval DP
 - └ "ప్రతి step లో ఉత్తమమైనది" + నిరూపించగలను → Greedy

Constraints → Algorithm (మళ్ళీ)

$n \leq 10$	→ $O(n!)$	backtracking (permutations)
$n \leq 20$	→ $O(2^n)$	subsets, bitmask DP
$n \leq 100$	→ $O(n^3)$	Floyd-Warshall, interval DP
$n \leq 1,000$	→ $O(n^2)$	2D DP, nested loops
$n \leq 10^5$	→ $O(n \log n)$	sort, heap, binary search
$n \leq 10^6$	→ $O(n)$	two pointers, sliding window, hashing
$n \leq 10^9$	→ $O(\log n)$	binary search on answer, math

44. 8-Week Preparation Plan

వారం-వారీ ప్రణాళిక (రోజుకి 2-3 గంటలు)

వారం 1 – పునాదులు

- DSA_00_Foundations (complexity, recursion, data structures)
- Patterns 1-3: Two Pointers, Fast-Slow, Sliding Window
- 15-20 easy problems (patterns ని గుర్తించడం మీద దృష్టి)

వారం 2 – Arrays, Hashing, Stack

- Patterns 4-10: Prefix Sum, Kadane, Cyclic Sort, Intervals, Hash Map, Monotonic Stack
- 20 problems (easy → medium)

వారం 3 – Binary Search & Heap ★

- Patterns 11-14 (**Binary Search on Answer కి ఎక్కువ సమయం**)
- 20 problems

వారం 4 – Linked Lists & Trees

- Patterns 15-22
- Tree DFS top-down vs bottom-up ని పూర్తిగా అర్థం చేసుకోవడం
- 25 problems

వారం 5 – Graphs

- Patterns 23-27
- Grid problems, topological sort, union-find
- 20 problems

వారం 6 – Backtracking & Recursion

- Patterns 28-30
- Template ని బట్టి పట్టి, variants practice
- 15 problems

వారం 7 – Dynamic Programming ★★

- Patterns 31-37 (**అత్యధిక సమయం ఇక్కడ**)
- ప్రతి problem: recursion → memo → tabulation → space optimize
- 25 problems

వారం 8 – Revision + Mock Interviews

- Patterns 38-41 (greedy, bit, design)
- రోజుకి 2 mock interviews (timed, బయటకి మాట్లాడుతూ)
- తప్పు చేసిన problems ని మళ్ళీ (గుర్తుపెట్టుకోకుండా – solve చేయడం)

Practice నియమాలు (అత్యంత ముఖ్యం)

1. 🕒 Timer పెట్టుకోండి – 25 నిమిషాలు. దాటితే solution చూడండి, కానీ:
→ వెంటనే మూసేసి, స్వయంగా మళ్ళీ రాయండి
→ 3 రోజుల తర్వాత మళ్ళీ (అప్పుడు చూడకుండా రాగలగాలి)
2. 📝 ప్రతి problem తర్వాత ఒక వాక్యం రాయండి:
"ఇది [pattern] – [signal] వల్ల గుర్తించాను – కీలక అంతర్దృష్టి [X]"
→ ఇదే మీ personal revision doc అవుతుంది
3. 🎤 బయటకి మాట్లాడుతూ solve చేయండి (interview simulation)

4. 🔄 Spaced repetition: రోజు 1 → రోజు 3 → రోజు 7 → రోజు 21
5. 🌟 Quantity కంటే quality:
150 problems పూర్తిగా అర్థం చేసుకోవడం > 500 problems copy చేయడం
ప్రతి pattern నుండి 5-8 problems చాలు
6. ❌ Solutions ని బట్టి పట్టోద్దు – patterns ని అర్థం చేసుకోండి
(interview లో exact problem రాదు; pattern వస్తుంది)

తప్పకుండా solve చేయాల్సిన problems (pattern-wise minimum)

Two Pointers (5): Valid Palindrome · Two Sum II · 3Sum · Container With Most Water · Trapping Rain Water
 Sliding Window (5): Longest Substring Without Repeating · Min Size Subarray Sum · Longest Repeating Char Replacement · Permutation in String · Min Window Substring
 Hash Map (4): Two Sum · Group Anagrams · Longest Consecutive · Subarray Sum Equals K
 Stack (4): Valid Parentheses · Min Stack · Daily Temperatures · Largest Rectangle
 Binary Search (5): Binary Search · Search Rotated · Find Min Rotated · Koko Bananas ★ · Median Two Arrays
 Heap (4): Kth Largest · Top K Frequent · Merge K Lists · Find Median from Stream
 Linked List (5): Reverse · Merge Two · Reorder · Remove Nth · Reverse K-Group
 Trees (8): Max Depth · Same Tree · Invert · Diameter · Level Order · Validate BST · Kth Smallest BST · Max Path Sum
 Graphs (7): Number of Islands · Clone Graph · Course Schedule · Pacific Atlantic · Rotting Oranges · Word Ladder · Network Delay Time
 Backtracking (6): Subsets · Permutations · Combination Sum · Word Search · Palindrome Partitioning · N-Queens
 DP (12): Climbing Stairs · House Robber I/II · Coin Change · LIS · Word Break · Unique Paths · LCS · Partition Equal Subset · Edit Distance · Best Time to Buy/Sell IV · Burst Balloons
 Greedy (4): Jump Game I/II · Gas Station · Partition Labels
 Design (3): LRU Cache · Min Stack · Implement Trie

మొత్తం: ~75 core problems – ఇవి పూర్తిగా అర్థమైతే SDE2 interviews కి సిద్ధం ✅

45. Memory Tips – ఒకసారి చదివి గుర్తుపెట్టుకునే Framework

1. ఒక్క వాక్యంలో DSA interview

"కొత్త problem లేదు – నాకు తెలియని రూపంలో ఉన్న తెలిసిన pattern మాత్రమే."

2. ప్రతి problem కి 4 ప్రశ్నలు

1. Input/output/constraints ఏమిటి? → clarify
2. ఏ signals కనిపిస్తున్నాయి? → pattern

3. Brute force ఏమిటి, అందులో ఏం వృధా? → optimization దిశ
 4. n ఎంత? → ఏ complexity target? → algorithm class

3. Top 10 patterns (interviews లో 80%)

- | | |
|---------------------|---------------------------|
| 1. Two Pointers | 6. Tree DFS/BFS |
| 2. Sliding Window ★ | 7. Graph BFS/DFS (+ grid) |
| 3. Hash Map | 8. Backtracking |
| 4. Binary Search ★★ | 9. Dynamic Programming ★★ |
| 5. Heap / Top-K | 10. Monotonic Stack |

4. Analogy map

Pattern	Analogy
Two Pointers	బరువు తూచే త్రాసు
Fast & Slow	Circular track మీద ఇద్దరు పరుగెత్తడం
Sliding Window	రైలు కిటికీ
Prefix Sum	Highway milestone markers
Kadane	ప్రయాణంలో సహచరుడిని వదిలేయడం
Cyclic Sort	Theatre seat numbers
Monotonic Stack	వరుసలో ముందు చూడటం
Binary Search	నిఘంటువు
BS on Answer	సరైన size చెప్పులు ★
Heap / Top-K	3 కుర్చీలున్న పోటీ
Tree DFS	రాజ ఆజ్ఞ (top-down) / జనాభా లెక్కలు (bottom-up)
Tree BFS	Floor-by-floor announcement
Grid DFS	ఇంక చుక్క వ్యాపించడం
Topological Sort	వంట క్రమం
Union-Find	గ్రామాల మధ్య వంతెనలు
Backtracking	Maze + సుద్దముక్క (గుర్తులు చెరపడం)
DP	రద్దీ రోడ్డు – ఒకసారి లెక్కిసినది మళ్ళీ లెక్కించకపోవడం
Greedy	చిల్లర ఇవ్వడం (కానీ {1,3,4} ఉచ్చు!)

5. Debug checklist (code పని చేయకపోతే)

- Off-by-one? ($<$ vs $\leq$, i vs $i+1$, n vs $n-1$)
- Empty/single element **input**?
- Base case సరైనదా? (DP, recursion)
- Visited **mark** చేశానా? (**graph**, grid)
- **Copy** చేశానా? (backtracking `result.push([...current])`)
- Undo చేశానా? (backtracking, DFS)
- Overflow? (JS 32-bit bitwise, MAX_SAFE_INTEGER)
- **Sort** comparator ఇచ్చానా? (JS!)
- Memo key లో అన్ని state variables ఉన్నాయా?
- Negative numbers? (modulo, sliding **window**)

6. Complexity ని 10 సెకన్లలో చెప్పడం

ఒక loop	→ $O(n)$
Nested loops	→ $O(n^2)$
Loop + binary search	→ $O(n \log n)$
Sorting	→ $O(n \log n)$
Recursion tree (2 branches, depth n)	→ $O(2^n)$
DP	→ states $\times$ transitions
Heap operations	→ $O(\log n)$ each
Union-Find	→ $\sim O(1)$ amortized
Graph traversal	→ $O(V + E)$

7. చివరి మాట

DSA interview లో మీరు **solve చేసిన problems సంఖ్య ముఖ్యం కాదు** — మీరు గుర్తించగల **patterns సంఖ్య ముఖ్యం**.

500 problems copy-paste చేసిన వ్యక్తి, కొత్త problem ముందు నిస్సహాయుడు. **75 problems ని pattern దృష్టితో అర్థం చేసుకున్న వ్యక్తి**, ఎప్పుడూ చూడని problem ని కూడా విడగొట్టగలడు.

SDE2 అంటే — pattern గుర్తించి, సరైన solution ని clean code తో, complexity తో సహా రాయగలగడం. **SSE అంటే** — ఎందుకు ఈ **approach** అని justify చేయగలగడం; trade-offs (time vs space, simplicity vs optimality) మాట్లాడగలగడం; edge cases ముందే ఊహించడం; interviewer తో సహకారంగా ఆలోచించడం.

గుర్తుంచుకోండి: Interview లో మీరు **సమాధానం** కోసం మాత్రమే మూల్యాంకనం కావడం లేదు — **మీ ఆలోచనా విధానం** కోసం. తప్పు దారిలో వెళ్తూ బయటకి ఆలోచించే వ్యక్తి, మౌనంగా సరైన సమాధానం రాసే వ్యక్తి కంటే మంచి signal ఇస్తాడు (చాలా సందర్భాల్లో).

అనుబంధం — వేగవంతమైన Reference

Templates ఒక్క చూపులో

```
// Two Pointers (opposite)
let l = 0, r = n - 1;
```

```

while (l < r) { /* condition */ l++ || r--; }

// Sliding Window (variable)
let left = 0;
for (let right = 0; right < n; right++) {
  /* expand */
  while (invalid) { /* shrink */ left++; }
  /* update result */
}

// Binary Search (lowerBound)
let lo = 0, hi = n;
while (lo < hi) { const mid = lo + ((hi - lo) >> 1); if (cond(mid)) hi = mid; else lo = mid + 1; }

// Binary Search on Answer
let lo = minAns, hi = maxAns;
while (lo < hi) { const mid = lo + ((hi - lo) >> 1); if (feasible(mid)) hi = mid; else lo = mid + 1; }

// BFS (level)
const q = [start];
while (q.length) { const size = q.length; for (let i = 0; i < size; i++) { /* process */ } }

// DFS (grid)
const DIRS = [[0,1],[1,0],[0,-1],[-1,0]];
function dfs(r, c) { if (invalid) return; mark(); for (const [dr,dc] of DIRS) dfs(r+dr, c+dc); }

// Backtracking
function bt(state) {
  if (complete) { res.push([...state]); return; }
  for (const choice of choices) { state.push(choice); bt(state); state.pop(); }
}

// DP (memo)
function dp(...s) { if (base) return v; const k = s.join(","); if (memo.has(k)) return memo.get(k);
  let r = init; for (const c of choices) r = best(r, dp(...next)); memo.set(k, r); return r; }

// Monotonic Stack
for (let i = 0; i < n; i++) {
  while (stack.length && cond(nums[i], nums[stack.at(-1)])) { const j = stack.pop(); /* answer for j */ }
  stack.push(i);
}

// Union-Find
find(x) { if (p[x] !== x) p[x] = find(p[x]); return p[x]; }
union(x, y) { const rx = find(x), ry = find(y); if (rx === ry) return false; p[ry] = rx; return true; }

// Topological Sort (Kahn)
const q = nodes.filter(n => indegree[n] === 0);
while (q.length) { const n = q.shift(); order.push(n); for (const nb of adj[n]) if (--indegree[nb] === 0) q.push(nb); }

```

JavaScript DSA essentials

```
// Arrays
arr.sort((a, b) => a - b) // ⚠ comparator తప్పనిసరి!
[...arr].sort() / arr.toSorted() // non-mutating
arr.at(-1) // చివరిది
Array.from({length: n}, () => []) // 2D init (fill([]) ✗ same reference!)
new Array(n).fill(0)
arr.reduce((a, b) => a + b, 0)

// Map/Set
const m = new Map(); m.get(k) ?? 0; m.set(k, v); m.has(k); m.size
const s = new Set(arr); s.has(x); [...s]

// Strings
s.split(""); [...s]; s.charCodeAt(0) - 97; String.fromCharCode(97 + i)
s.slice(i, j) // substring

// Math
Math.floor(a / b) // integer division
((x % m) + m) % m // safe modulo
Number.MAX_SAFE_INTEGER // 2^53 - 1
Infinity / -Infinity

// Queue (shift O(n) - పెద్ద inputs కి index pointer)
let head = 0; const q = []; q.push(x); const item = q[head++];
```

Complexity cheat-sheet

Operation	Array	Map/Set	Heap	Tree (bal.)	Union-Find
Access	$O(1)$	—	$O(1)$ top	$O(\log n)$	—
Search	$O(n)$	$O(1)$	$O(n)$	$O(\log n)$	$O(\alpha)$
Insert	$O(n)$	$O(1)$	$O(\log n)$	$O(\log n)$	$O(\alpha)$
Delete	$O(n)$	$O(1)$	$O(\log n)$	$O(\log n)$	—

ఈ guide పూర్తి చేసినందుకు అభినందనలు! ఇప్పుడు చేయవలసినది — ప్రతి pattern నుండి 5 problems solve చేయడం ([DSA_01..10_Telugu.md](#) లో problem-by-problem walkthroughs ఉన్నాయి). ప్రతి problem తర్వాత "ఇది ఏ pattern, ఏ signal వల్ల గుర్తించాను?" అని ఒక వాక్యం రాయండి. 8 వారాల తర్వాత — ఏ problem చూసినా మీ మెదడు automatic గా pattern చెప్తుంది. అదే interview లో మీకు కావలసినది.

Companion docs: [DSA_00_Foundations_Telugu.md](#) (పునాదులు — ఇది ముందు) ·

[DSA_01..10_Telugu.md](#) (LeetCode Top-150 drill) · [JavaScript_Telugu.md](#) · [00PS_Telugu.md](#) ·

[HLD_Telugu.md](#) · [LLD_Telugu.md](#) · [SoftwareEngineering_Telugu.md](#)